Mikrocomputertechnik III

Christian Ege

Version v0.7.0: In Bearbeitung

Inhaltsverzeichnis

Über den Author . . . 2
Impressum . . . 3
Copyright 2026 – Urheberrechtshinweis . . . 4
Grundlagen der C++-Programmierung . . . 5
  Wie entsteht ein ausführbares Programm? . . . 5
  Der Linker . . . 7
Wiederholung der C++-Grundlagen . . . 9
  Interaktiver Compiler . . . 9
  Ein einfaches C++-Programm . . . 9
  Formatierte Ausgabe . . . 9
    Wie groß ist der Speicherbereich der Standarddatentypen? . . . 10
  Kommentare in C++ . . . 10
  Verzweigungen . . . 10
  Schleifen . . . 11
    For-Schleifen . . . 11
    While-Schleifen . . . 11
  Prä- und Postinkrement-Operatoren . . . 11
    Präinkrement-Operator . . . 11
    Postinkrement-Operator . . . 11
  Der Präprozessor . . . 11
    Hinweise . . . 12
    Die Include-Anweisung . . . 12
    Include-Guard . . . 12
    Bedingte Ersetzung . . . 13
  Zeiger . . . 13
    Pointer-Firewall . . . 13
  Strukturen . . . 14
    Deklaration und Verwendung . . . 14
    Zugriff auf Strukturattribute . . . 14
    typedef für Strukturen . . . 14
    Stack- vs. Heap-Allokation . . . 15
    Selbstreferenzierende Strukturen . . . 15
    Häufige Fehler . . . 15
    Padding und Alignment . . . 16
    Strukturen in der Datenübertragung und Hardware-Abbildung . . . 19
Halbleiter-Speicherbausteine . . . 22
  Überblick und Einteilung . . . 22
  ROM-Familie: Mask-ROM, PROM, EPROM, EEPROM . . . 22

Mask-ROM . . . 23
PROM . . . 23
EPROM . . . 23
EEPROM . . . 23
Flash-Speicher (NOR vs. NAND) . . . 23
NOR-Flash . . . 23
NAND-Flash . . . 23
Wear-Leveling . . . 23
XIP und Wait-States (STM32) . . . 24
SRAM . . . 24
DRAM . . . 24
Zeilen-/Spaltenprinzip . . . 24
Adressdekodierung und Memory-Map . . . 25
FIFO und Dual-Ported-RAM . . . 25
FIFO (First In, First Out) . . . 25
Dual-Ported-RAM . . . 26
Zusammenfassung . . . 26
Die Speicherverwaltung . . . 27
Speichersegmente . . . 28
Das Segment für den Programmcode `.text` . . . 29
Das Daten-Segment `.data` . . . 29
Das Segment der uninitialisierten Daten `.bss` . . . 30
Das Segment für konstante Daten `.rodata` . . . 30
Übersicht des Memory-Layouts . . . 31
Wo werden lokale Variablen abgelegt? . . . 31
Der Stack . . . 32
Maximalgröße des Stacks . . . 32
Der Call-Stack . . . 32
Aufrufparameter . . . 33
Rücksprungadresse . . . 34
Der Framepointer . . . 34
Der Stackpointer . . . 34
Register . . . 34
Lokale Variablen . . . 34
Rückgabewerte . . . 34
Fragen . . . 34
Der Heap . . . 35
Anfordern von Speicher . . . 35
CPU-Cache . . . 36
Der NVIDIA Tegra TX2 . . . 36
Cache-Hierarchie . . . 36

Instruction Caches . . . 37
Data Caches . . . 38
Cache-Schreibrichtlinien . . . 38
Cache-Kohärenz . . . 38
Schreiben cachefreundlicher Software . . . 38
1. Räumliche Lokalität (Spatial Locality) . . . 38
2. Zeitliche Lokalität (Temporal Locality) . . . 39
3. Datenstrukturen optimieren . . . 39
4. Cache-Line-Ausrichtung beachten . . . 39
5. „False Sharing“ vermeiden . . . 39
6. Prefetching nutzen . . . 39
7. Zusammenhängende Speicherlayouts bevorzugen . . . 39
Zusammenfassung . . . 40
Rechnerarchitektur . . . 41
RISC-Architektur . . . 41
CISC-Architektur . . . 41
Load/Store-Architektur . . . 42
Die Blackfin AAU . . . 42
Der STM32F103 Mikrocontroller . . . 44
Pipeline . . . 45
Die Cortex-M3-Pipeline . . . 45
Effizientes Schreiben von Code ohne Conditionals . . . 46
Warum vermeiden wir Sprünge? . . . 46
Beispiel: Absolute Wertberechnung . . . 46
Beispiel: Clamping (Begrenzen eines Werts) . . . 47
Beispiel: Auswahl zwischen zwei Werten ohne if . . . 47
Bitmaskierung als Alternative . . . 48
Schleifen optimieren . . . 48
Lookup-Tabellen als Ersatz für Bedingungslogik . . . 48
Wann Branchless Code sinnvoll ist? . . . 49
Wann Branchless Code NICHT sinnvoll ist? . . . 49
Zusammenfassung . . . 49
System- und Schnittstellenbausteine des STM32F103 . . . 50
Peripherie im Adressraum . . . 50
Read-Modify-Write und Bit-Banding . . . 50
Read-Modify-Write beim Cortex-M3 . . . 51
I/O-Schnittstellen und Peripheriebusse . . . 58
Datenstrukturen . . . 70
Einfach verkettete Listen . . . 71
Binärbäume . . . 75
Das BMP-Dateiformat . . . 79

Funktionszeiger in C 81
Alles hat einen Zustand 83
Anhang A: Linker-Skript des STM32F103 89
Anhang B: Start-up-Code des STM32F103 93
Anhang C: Versionsverwaltung 98
Anhang D: Testen 103

Moderne CPU Architekturen aus dem Blickwinkel der Softwareentwicklung betrachtet.

Über den Author

Christian Ege arbeitet als Softwarearchitekt für Eingebettete Systeme bei der ifm syntron gmbh im Bereich Industrielle Bildverarbeitung und Robotik.

Du erreichst Christian Ege unter folgender E-Mail Adresse ch@ege.io. Christian freut sich über Fragen und Anregungen.

CE

Impressum

Version: v0.7.0

Version Date: 2026-10-14

Version Notes: In Bearbeitung

Copyright 2026 – Urheberrechtshinweis

Alle Inhalte dieses Werkes, insbesondere Texte, Fotografien und Grafiken, sind urheberrechtlich geschützt. Das Urheberrecht liegt, soweit nicht ausdrücklich anders gekennzeichnet, bei Christian Ege. Bitte fragen Sie mich, falls Sie Inhalte dieses Werkes verwenden möchten.

```
Unter der "Creative-Commons-Lizenz" veröffentlichte Inhalte sind als solche
gekennzeichnet. Sie dürfen entsprechend den angegebenen Lizenzbedingungen verwendet
werden.
```

Wer gegen das Urheberrecht verstößt (z. B. Bilder oder Texte unerlaubt kopiert), macht sich gem. §§ 106 ff. UrhG strafbar, wird zudem kostenpflichtig abgemahnt und muss Schadensersatz leisten (§ 97 UrhG).

Grundlagen der C++-Programmierung

Dieses Kapitel soll zur Wiederholung der wichtigsten Konzepte und Sprachmittel der Programmiersprache C++ dienen.

Wie entsteht ein ausführbares Programm?

Die folgende schematische Darstellung beschreibt den Ablauf bei der Erzeugung des ausführbaren Maschinencodes.

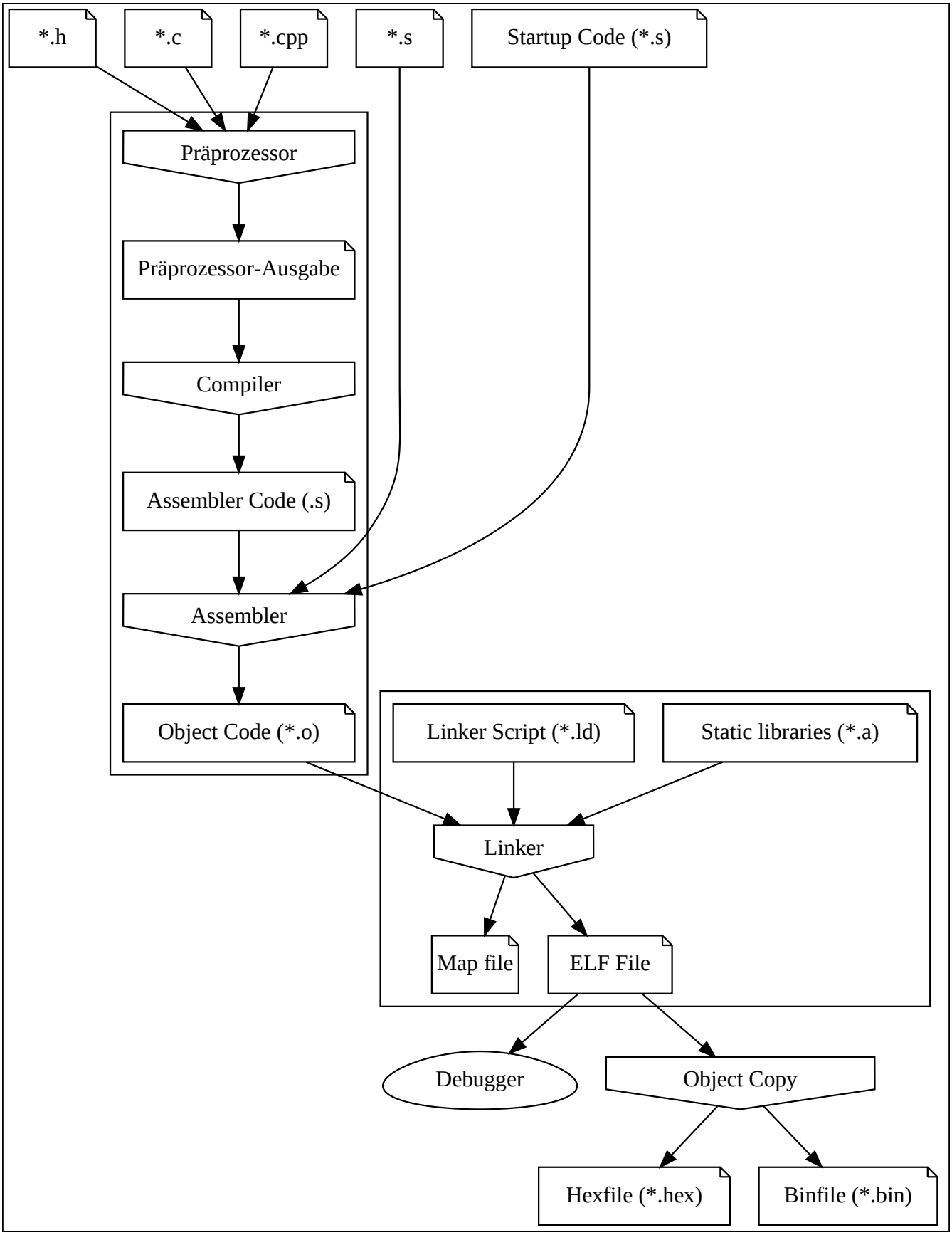


Abbildung 1. Ablauf vom Quelltext zur ausführbaren Datei

Bei der Übersetzung von Quellcode in ausführbaren Maschinencode werden mehrere Stufen durchlaufen. In der ersten Stufe werden die C-, C++- und Header-Dateien durch den sogenannten Präprozessor (Der Präprozessor) verarbeitet. Dabei entsteht eine temporäre Datei, in die sämtliche `#include`-Anweisungen eingefügt werden. Damit Variablen und Funktionen nicht mehrfach definiert werden, ist die Verwendung eines Include-Guard notwendig.

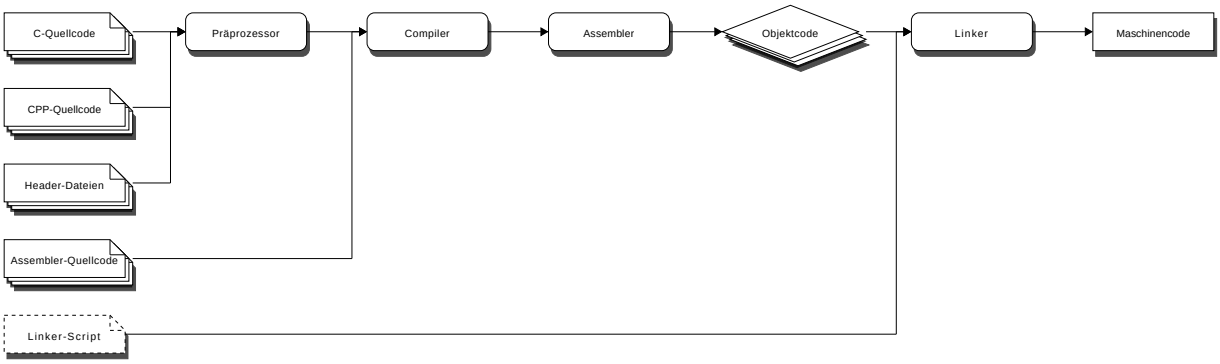


Abbildung 2. Sequenzieller Ablauf vom Quelltext zur ausführbaren Datei

Der Linker

Der Linker hat die Aufgabe, aus dem vom Assembler erzeugten Objektcode eine ausführbare Datei (z. B. `.exe` oder `.elf`) zu erzeugen. Im Falle eines Mikrocontrollers wie dem Der STM32F103 Mikrocontroller kommt hierbei ein sogenanntes Linker-Skript des STM32F103 zum Einsatz. Dieses definiert die Größe der einzelnen Speicherbereiche sowie deren Offsets.

Speicheraufbau des STM32F103

[1] image::img/stm32f103-memory.png[]

```
/* Memories definition */
MEMORY
{
  RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 20K
  ROM (rx)  : ORIGIN = 0x08000000, LENGTH = 128K
}
```

Neben den Speichergrößen und deren Anfangsadressen definiert das Linker-Skript auch die Position und die Größe des Stacks:

```
/* Highest address of the user-mode stack */
_estack        = 0x20005000; /* End of RAM */
_Min_Stack_Size = 0x400;      /* Required amount of stack */
```

Darüber hinaus legt das Linker-Skript die Speichersegmente (Speichersegmente) fest:

```
/* Sections */
SECTIONS
{
  .isr_vector : { /* ... */ } >ROM
  .text       : { /* ... */ } >ROM
  .rodata     : { /* ... */ } >ROM
  .data       : { /* ... */ } >RAM AT>ROM
```

```
  . = ALIGN(4);
  .bss : { /* ... */ } >RAM
}
```

Zusätzlich benötigt der STM32F103 einen Start-up-Code des STM32F103, der unter anderem das `.bss`-Segment initialisiert:

```
/* Zero-fill the bss segment */
FillZerobss:
    movs r3, #0
    str  r3, [r2]
    adds r2, r2, #4

LoopFillZerobss:
    ldr r3, = _ebss
    cmp r2, r3
```

[1] STM32F103 Datenblatt: https://www.st.com/resource/en/datasheet/stm32f103c8.pdf

Wiederholung der C++-Grundlagen

Die Grundlagen der C++-Entwicklung haben Sie bereits in den vergangenen Semestern kennengelernt. Dieses Kapitel dient der Auffrischung und Wiederholung.

Interaktiver Compiler

Code kann direkt im interaktiven Compiler ausprobiert werden:

http://gcc.godbolt.org/

Ein einfaches C++-Programm

Ein einfaches C++-Programm – nicht das allereinfachste, aber ein guter Einstieg.

Erste Versuche in C++

```
int main(int argc, char *argv[])
{
    puts("Hello world.");
    return 0;
}
```

Wie lautet die Signatur der Funktion `puts` und wo ist diese deklariert?

- puts

Zur Verwendung muss der passende Header eingebunden werden:

```
#include <stdio.h>
```

Formatierte Ausgabe

Da die Verwendung von `printf` nicht typsicher ist, verwenden wir im Verlauf der Vorlesung die `{fmt}`-Bibliothek.

Formatierte Ausgabe mittels {fmt}

```
#include <fmt/core.h>

int main() {
    int age = 30;
    int height = 190;

    fmt::print("Ich bin {} Jahre alt.\n", age);
    fmt::print("Ich bin {} cm groß.\n", height);
```

```
}
```

Wodurch unterscheiden sich die Programme?

- `main` wird anders deklariert.
- Es werden Variablen verwendet.

Wie lautet die Signatur der Funktion `fmt::print`?

- fmt::print
- Format String Syntax

Wie groß ist der Speicherbereich der Standarddatentypen?

Die Größe eines Datentyps kann mit dem Operator `sizeof` ermittelt werden:

```
fmt::print("Die Größe von char: {}\n", sizeof(char));
```



Probieren Sie dies beispielhaft für verschiedene Datentypen aus.

Kommentare in C++

Mehrzeilige Kommentare werden mit `/* ... */` notiert, einzeilige Kommentare mit `//`.

Verzweigungen

if-Abfragen können wie folgt definiert werden:

```
#include <stdio.h>

int main()
{
    int foo = 1;

    if (0 == foo) {
        fmt::print("foo == 0\n");
    } else {
        fmt::print("foo != 0\n");
    }

    return 0;
}
```

Schleifen

For-Schleifen

```
for (int i = 0; i < 14; i++) {
    fmt::print("i: {}\n", i);
}
```

While-Schleifen

```
int i = 0;
while (i < 20) {
    fmt::print("i: {}\n", i);
    i++;
}
```

Prä- und Postinkrement-Operatoren

Der Präinkrement-Operator erhöht den Operanden um eine Einheit **vor** seiner Rückgabe. Beim Postinkrement erfolgt die Rückgabe zuerst, anschließend die Erhöhung.

Präinkrement-Operator

```
int i = 0;
while (i < 20) {
    fmt::print("i: {}\n", ++i);
}
```

Postinkrement-Operator

```
int i = 0;
while (i < 20) {
    fmt::print("i: {}\n", i++);
}
```

Der Präprozessor

Der Präprozessor ist die erste Stufe eines C-Compilers. Er wertet Makros aus, verarbeitet `#include`- und `#define`-Anweisungen und erzeugt daraus eine neue Datei, die an den eigentlichen Compiler übergeben wird.

```
#define LAENGE 360
```

Hinweise

- Minimieren Sie die `#include`-Anweisungen auf das Nötigste, um die Kompilierzeit zu reduzieren.
- Ein Präprozessor-Makro beginnt mit `#`. Davor dürfen nur Leerzeichen stehen.
- Der Präprozessor ersetzt das Makro überall im Quelltext.
- Innerhalb von Zeichenkettenkonstanten (`"..."`) findet keine Ersetzung statt.
- Befindet sich die Zeichenfolge im Inneren eines Namens (z. B. `dieLAENGEinMeter`), findet ebenfalls keine Ersetzung statt.
- Der Präprozessor setzt vor und nach einem Ersetzungstext jeweils ein Leerzeichen.

```
a + LAENGE .0
```

Ergibt:

```
a + 360 .0
```

Makros können auch per Kommandozeile definiert werden:

```
gcc -DMY_TEST=100 -o test test.c
```

Die Include-Anweisung

Bei dieser Präprozessor-Anweisung wird der Inhalt der eingebundenen Datei in den Quelltext kopiert. Die resultierende Datei wird anschließend an den Compiler übergeben.

Include-Guard

Ein Include-Guard verhindert das mehrfache Einbinden derselben Header-Datei und somit Mehrfachdefinitionen.

```
#ifndef EIN_EINDEUTIGER_NAME
#define EIN_EINDEUTIGER_NAME

/* Variablen */
/* Funktionen */

#endif /* EIN_EINDEUTIGER_NAME */
```

Bedingte Ersetzung

Mit `#if`, `#ifdef`, `#ifndef`, `#else`, `#elif` und `#endif` können Teile des Codes abhängig von Makrowerten kompiliert oder ausgelassen werden.

```
#ifdef WIN32
# include <windows.h>
#else
# include <unistd.h>
#endif
```

```
#if VERBOSE >= 2
    fmt::print("Kontrollausgabe\n");
#endif
```

Zeiger

Zeiger werden wie folgt deklariert:

```
int foo = 42;
int* pFoo = &foo;
```

Pointer-Firewall

Bei Verwendung von Zeigern empfiehlt es sich, diese auf Gültigkeit zu prüfen.

> Thou shalt not follow the NULL pointer, for chaos and madness await thee at its end.
>
> — Henry Spencer, The Ten Commandments for C Programmers

```
void doSomething(int* data) {
    if (data != NULL) {
        *data += 2;
    }
}
```

Strukturen

Strukturen dienen dazu, mehrere zusammengehörige Werte unter einem gemeinsamen Datentyp zusammenzufassen. Sie sind besonders nützlich für komplexere Datensätze wie Personen, Punkte, Sensordaten oder Listenknoten.

Deklaration und Verwendung

```
struct person {
    int   alter;
    float gewicht;
    char  name[25];
} adam, eva;
```

```
struct person paul;
paul.alter = 13;
paul.gewicht = 47.8;
```

In C++ (und in C nach einem typedef) kann die Struktur auch ohne das Präfix `struct` verwendet werden:

```
person paula;
```

Zugriff auf Strukturattribute

- `objekt.feld` – Zugriff über Wert
- `ptr->feld` – Zugriff über Zeiger

```
person p;
p.alter = 25;

person* ptr = &p;
ptr->alter = 26;
```

typedef für Strukturen

```
typedef struct ListNode {
    unsigned int data;
    struct ListNode* pNext;
} ListNode_t;

ListNode_t node;
```

Stack- vs. Heap-Allokation

Stack

```
ListNode_t n;
n.data = 42;
```

Heap

```
ListNode_t* n = malloc(sizeof(ListNode_t));
n->data = 42;
```

Selbstreferenzierende Strukturen

```
typedef struct ListNode {
    unsigned int data;
    struct ListNode* pNext;
} ListNode_t;
```

Häufige Fehler

Fehler	Ursache	Lösung
`node.data` statt `node->data`	Wert- statt Zeigerzugriff	`->` verwenden
`malloc(sizeof(pointer))`	falscher `sizeof`-Operand	`sizeof(ListNode_t)` verwenden
fehlendes `free()`	Speicherleck	Speicher in Aufräumfunktion freigeben
Stack statt Heap	Lebensdauerproblem	Objekt dynamisch auf dem Heap anlegen
leere Liste nicht geprüft	Sonderfall übersehen	auf `pHead == NULL` prüfen

Padding und Alignment

Beim Arbeiten mit Strukturen ist es wichtig zu verstehen, dass der Compiler unter Umständen zusätzliche Bytes einfügt, um eine für die Architektur optimale Ausrichtung sicherzustellen. Dieser Vorgang wird als **Padding** (beziehungsweise **Structure Padding**) bezeichnet.

Warum entsteht Padding?

Viele Prozessorarchitekturen können bestimmte Datentypen nur effizient lesen, wenn sie an Speicheradressen ausgerichtet sind, die zur Größe des Typs passen:

- `int` → typischerweise 4-Byte-Ausrichtung
- `float` → 4-Byte-Ausrichtung
- `double` → 8-Byte-Ausrichtung

Der Compiler fügt daher automatisch **Auffüllbytes** ein, damit jedes Feld der Struktur korrekt ausgerichtet ist. Dies kann dazu führen, dass die Struktur größer ist als die Summe ihrer Felder.

Beispiel: Padding innerhalb einer Struktur

```
struct Example {
    char   a;   // 1 Byte
    int    b;   // 4 Byte
    char   c;   // 1 Byte
};
```

Auf einer 32-Bit- oder 64-Bit-Architektur ergibt sich typischerweise folgende Anordnung:

```
Offset  Inhalt        Größe
0       a             1 Byte
1-3     Padding       3 Byte
4-7     b             4 Byte
8       c             1 Byte
9-11    Padding       3 Byte
```

Gesamtgröße: **12 Byte**, nicht 6.

Reihenfolge der Felder beeinflusst Padding

```
struct ExampleOpt {
    int    b;
    char   a;
    char   c;
};
```

Jetzt:

```
Offset  Inhalt        Größe
0-3     b             4 Byte
4       a             1 Byte
5       c             1 Byte
6-7     Padding       2 Byte
```

Gesamtgröße: **8 Byte** statt 12.

Warum Feldausrichtung wichtig ist

- reduziert Speicherbedarf bei großen Arrays von Strukturen
- verbessert Cache-Lokalität
- reduziert die Anzahl benötigter Speicherzugriffe
- vermeidet unaligned access penalties (insbesondere ARM Cortex-A/Cortex-M)

explizite Ausrichtung mit `alignas`

```
struct alignas(16) Vector4 {
    float x, y, z, w;
};
```

Dies ist insbesondere für:

- SIMD-Optimierungen
- Hardwarebeschleunigung
- DMA-Transfers

relevant.

Packed-Strukturen

```
struct __attribute__((packed)) Packed {
    char a;
    int  b;
    char c;
};
```

Dies führt jedoch zu:

- unaligned memory accesses
- potenziell deutlich schlechterer Performance
- möglichen Ausrichtungsfehlern auf strengen Architekturen

Packed-Strukturen sollten nur verwendet werden, wenn zwingend notwendig, z. B. bei Protokoll-Headern oder Hardware-Registern.

Praktische Empfehlung

- Felder nach absteigender Größe sortieren
- Padding prüfen (z. B. mit `sizeof`)
- Für Performance-kritische Strukturen Ausrichtung bewusst wählen
- Packed nur dann einsetzen, wenn absolut notwendig

Strukturen in der Datenübertragung und Hardware-Abbildung

Beim Einsatz von Strukturen für die Serialisierung von Daten oder beim Abbilden von Hardware-Registern ist besondere Vorsicht geboten. Die interne Speicheranordnung einer Struktur ist nicht vollständig durch den C- oder C++-Standard definiert. Compiler und Plattform können sich daher unterschiedlich verhalten.

Die Reihenfolge der Felder ist nicht standardisiert

Der C/C++-Standard garantiert lediglich, dass die Reihenfolge der Felder **relativ zueinander** erhalten bleibt. Er definiert jedoch **nicht vollständig**, wie viel Padding eingefügt wird oder wie die Struktur im Speicher ausgerichtet ist.

Das bedeutet:

- Die Strukturgröße ist plattform- und compilerabhängig.
- Padding kann zwischen Feldern oder am Ende auftreten.
- Unterschiedliche Compiler können dieselbe Struktur unterschiedlich packen.
- Optimierungseinstellungen beeinflussen das Layout.

Warum das direkte Übertragen einer Struktur problematisch ist

Viele Systeme senden Daten so:

```
send(&myStruct, sizeof(myStruct));
```

Dieses Verfahren ist fehleranfällig, da:

- **Padding-Bytes** übertragen werden, deren Inhalt undefiniert ist.
- Der Empfänger eine **andere Ausrichtung oder anderes Padding** haben kann.
- Das Layout z. B. zwischen ARM und x86 erheblich variieren kann.
- Die Strukturgröße auf verschiedenen Architekturen unterschiedlich ist.
- Unterschiedliche **Endianness** (Little/Big Endian) auftreten kann.

Beispiel: Eine 32-Bit-Zahl wird auf einer Big-Endian-Architektur komplett anders im Speicher abgelegt als auf einem Little-Endian-System.

Ergebnis: **Die Daten kommen beim Empfänger falsch oder unbrauchbar an.**

Empfehlung für Serialisierung

- Niemals Strukturen binär übertragen.
- Immer ein **explizites Datenformat** definieren:
  - einzelne Felder übertragen
  - definierte Byte-Reihenfolge (meist Little Endian)
  - keine Padding-Bytes
  - explizite Typgrößen (z. B. `uint32_t`, `uint16_t`)
  - Versionierung des Datenformats
- Geeignete Werkzeuge:
  - Protobuf / FlatBuffers
  - CBOR / MessagePack
  - manuell definierte Byte-Streams

Strukturen über Hardware-Register legen (Memory-Mapped I/O)

Eine häufige, aber riskante Technik ist:

```
struct RegMap {
    uint32_t CTRL;
    uint16_t STATUS;
    uint32_t VALUE;
};

volatile RegMap* const REG = (volatile RegMap*)0x40000000;
```

Problem:

- Der Compiler darf Padding einfügen → die Struktur passt **nicht** mehr über die Register.
- Der Compiler darf **Feldreihenfolge nicht verändern**, aber Padding trotzdem einfügen.
- Lese- und Schreibzugriffe werden eventuell zu **Split Loads / Split Stores**.
- Registerzugriffe müssen exakt der Hardwarebreite entsprechen.

Auch die Ausrichtung ist kritisch:

Ein 16-Bit-Register darf z. B. **nicht auf einer ungeraden Adresse** liegen.

Empfehlung für Registerabstraktion

- Register **immer einzeln** adressieren oder
- exakt spezifizierte Strukturen mit:

```
struct __attribute__((packed, aligned(1))) RegMap { ... };
```

Beachte:

- `packed` unterdrückt Padding
- kann aber unaligned memory access erzeugen
- was auf ARM-Cortex-A im schlimmsten Fall einen Fault auslöst

Die meisten Embedded-Hersteller liefern daher:

- CMSIS-konforme Registerstrukturen,
- exakt definierte `volatile`-Typen,
- Garantien über Packing,
- Compiler-spezifische Attribute.

Eigene Strukturen sollten nur verwendet werden, wenn klar ist, wie:

- Alignment
- Busbreite
- Endianness
- Registergröße

zusammenwirken.

Zusammenfassung

- Strukturen sind **nicht portabel binär übertragbar**.
- Padding, Alignment und Endianness machen eine 1:1-Speicherübertragung gefährlich.
- Die Reihenfolge der Felder ist zwar definiert, aber **der Speicherabstand nicht**.
- Für Serialisierung sollten explizite Datenformate genutzt werden.
- Für Memory-Mapped-Register sind streng definierte, gepackte Strukturen oder herstellerspezifische Header zu bevorzugen.

Halbleiter-Speicherbausteine

Dieses Kapitel behandelt die physikalischen und technologischen Grundlagen typischer Speicherbausteine in eingebetteten Systemen. Ziel ist, die Brücke zwischen Bausteineigenschaften und Softwareverhalten zu schlagen (z. B. Startup-Kopie von `.data` nach SRAM, XIP aus Flash, Cache-Effekte).

Überblick und Einteilung

Speicherbausteine lassen sich nach mehreren Kriterien einteilen:

- **Flüchtig** vs. **nichtflüchtig**
- **Schreibbar** vs. **nur lesbar**
- **Wahlfreier Zugriff** vs. **sequenzieller Zugriff**
- **Bandbreite, Latenz, Dichte, Energiebedarf**
- **Lebensdauer** (Schreib-/Löschzyklen)

Wichtige Kenngrößen:

- **Zugriffszeit:** Zeit vom Adressanlegen bis zu gültigen Daten
- **Lese-/Schreibbandbreite:** Datenmenge pro Zeit
- **Speicherdichte:** Bits pro Fläche/Chip
- **Endurance:** erlaubte Anzahl von Programmier-/Löschzyklen
- **Retention:** Datenhaltezeit ohne Versorgung

Typ	Flüchtig	Byteweise schreibbar	Typische Endurance	Typische Nutzung
ROM/Mask-ROM	nein	nein	n/a	Festes Programm/Tabellen
EEPROM	nein	ja	10^5 bis 10^6	Parameter, Kalibrierung
Flash	nein	seiten-/sektorweise	10^3 bis 10^5	Programm, Massendaten
SRAM	ja	ja	praktisch unbegrenzt	Laufzeitdaten, Cache
DRAM	ja	ja	praktisch unbegrenzt	Hauptspeicher hoher Dichte

ROM-Familie: Mask-ROM, PROM, EPROM, EEPROM

Mask-ROM

Mask-ROM wird bei der Fertigung dauerhaft programmiert. Änderungen sind nach der Produktion nicht mehr möglich. Vorteil sind geringe Stückkosten bei hohen Volumina und sehr stabile Datenhaltung.

PROM

PROM (**Programmable ROM**) lässt sich einmalig programmieren. Danach ist der Inhalt fest.

EPROM

EPROM (**Erasable PROM**) kann gelöscht und neu programmiert werden. Das Löschen erfolgt klassisch per UV-Licht über ein Quarzfenster im Gehäuse.

EEPROM

EEPROM (**Electrically Erasable PROM**) kann elektrisch gelöscht/programmiert werden, oft byteweise. Das macht EEPROM ideal für selten geänderte, persistente Konfigurationsdaten.

Flash-Speicher (NOR vs. NAND)

Flash ist heute der dominierende nichtflüchtige Programmspeicher in Mikrocontrollern.

Wesentliche Eigenschaften:

- Lesen ist schnell und oft direkt ausführbar (XIP bei NOR-Flash)
- Schreiben erfolgt blockorientiert (Page/Word-Programmierung)
- Löschen erfolgt in größeren Blöcken (Sektoren/Pages)
- Schreib-/Löschzyklen sind begrenzt (Verschleiß)

NOR-Flash

- schneller Random-Read
- gut für Execute-In-Place (Code direkt aus Flash)
- typischer Einsatz: Programmspeicher von Mikrocontrollern

NAND-Flash

- sehr hohe Dichte und günstiger pro Bit
- blockorientierter Zugriff, Fehlerkorrektur (ECC) wichtig
- typischer Einsatz: Massenspeicher (z. B. SD-Karten, SSD)

Wear-Leveling

Da Flash-Zellen nur eine begrenzte Anzahl von Löschzyklen erlauben, verteilt Wear-Leveling

Schreibvorgänge über mehrere Blöcke. So wird vorzeitiger Ausfall einzelner Blöcke vermieden.

XIP und Wait-States (STM32)

Bei vielen Cortex-M-Systemen wird Code direkt aus internem NOR-Flash ausgeführt (XIP). Steigt die CPU-Taktfrequenz, sind häufig zusätzliche Wait-States nötig, weil der Flash langsamer als der Kern ist. Das erklärt, warum sich die Laufzeit eines identischen Codes bei Takt- oder Flash-Setup-Änderungen unterscheiden kann.

SRAM

SRAM (**Static RAM**) speichert Bits in bistabilen Flip-Flop-Zellen (klassisch 6 Transistoren pro Bit). Solange Versorgung anliegt, bleibt der Zustand ohne Refresh erhalten.

Eigenschaften:

- sehr kurze Zugriffszeiten
- einfache Ansteuerung (kein Refresh erforderlich)
- geringere Dichte und höherer Flächenbedarf als DRAM

Typische Einsätze:

- Laufzeitdaten (`.data`, `.bss`, Heap, Stack)
- CPU-Caches (L1/L2)
- schnelle Puffer

DRAM

DRAM (**Dynamic RAM**) speichert jedes Bit als Ladung in einem Kondensator (1T1C-Zelle). Die Ladung läuft mit der Zeit aus, daher ist periodischer Refresh notwendig.

Eigenschaften:

- sehr hohe Dichte
- günstiger pro Bit als SRAM
- komplexere Steuerung (Refresh, Zeilen-/Spaltenadressierung)
- höhere Latenz als SRAM

Zeilen-/Spaltenprinzip

DRAM adressiert Speicher typischerweise in zwei Schritten:

1. Zeile aktivieren (Row/Wordline)
2. Spalte auswählen (Column/Bitline)

Diese Architektur ist Grundlage für SDRAM/DDR-Familien, die in leistungsstarken Systemen als

Hauptspeicher eingesetzt werden.

Adressdekodierung und Memory-Map

Eine CPU adressiert Speicher über Adressleitungen. Die externe/interne Adressdekodierung entscheidet, welcher Baustein auf eine Adresse antwortet.

Typische Signale:

- Adressbus
- Datenbus
- Steuerleitungen (RD/WR/OE/WE)
- Chip-Select (CS)

Prinzipbeispiel:

- `0x0800_0000 .. 0x0801_FFFF` → interner Flash (STM32F103)
- `0x2000_0000 .. 0x2000_4FFF` → SRAM
- `0x4000_0000 .. ...` → Peripherie

Hinweis zum STM32F103: Der Bereich ab `0x0000_0000` ist ein Alias, dessen Abbildung vom Boot-Modus abhängt (über BOOT-Pins, z. B. Flash, System Memory oder SRAM). Für die eindeutige Flash-Adressierung wird deshalb typischerweise `0x0800_0000` verwendet.

Diese Aufteilung (Memory-Map) ist die Grundlage für Memory-Mapped I/O und Registerzugriffe in C.

FIFO und Dual-Ported-RAM

FIFO (First In, First Out)

FIFO-Bausteine puffern Datenströme in Einfügereihenfolge. Sie entkoppeln zwei Taktdomänen oder Verarbeitungsgeschwindigkeiten (Producer/Consumer).

Typische Signale:

- `empty` / `full`
- `read` / `write`
- optional `almost_empty` / `almost_full`

Einsatzfälle:

- UART-/SPI-Empfangspuffer
- Audio-/Videodatenströme
- DMA-Kopplung zwischen Peripherie und Speicher

Dual-Ported-RAM

Dual-Port-RAM erlaubt den gleichzeitigen Zugriff über zwei unabhängige Ports. Damit können zwei Busmaster (z. B. CPU und DMA oder zwei Prozessoren) mit geringer Kopplung auf einen gemeinsamen Speicherbereich zugreifen.

Nutzen:

- deterministische Übergabe von Datenblöcken
- weniger Kopieraufwand
- effizient für Kommunikations- und Echtzeitszenarien

Zusammenfassung

- Die Eigenschaften der Speichertechnologie bestimmen direkt das Softwareverhalten.
- Flash ist nichtflüchtig und dicht, aber schreibzyklusbegrenzt und blockweise löschbar.
- SRAM ist schnell und ideal für Laufzeitdaten; DRAM ist dichter, benötigt aber Refresh.
- Adressdekodierung und Memory-Map sind Basis von Speichersegmenten und Peripheriezugriff.
- FIFO und Dual-Ported-RAM sind zentrale Hardware-Bausteine zur Entkopplung von Datenströmen.

Die Speicherverwaltung

Ein C/C++-Programm nutzt während der Ausführung mehrere klar voneinander abgegrenzte Speicherbereiche. Die wichtigsten davon sind:

- **Programmspeicher** (Maschinencode)
- **globaler Speicher** für globale und statische Variablen
- **Heap** für dynamische Speicherverwaltung
- **Stack** für Funktionsaufrufe und lokale Variablen

In den folgenden Abschnitten wird der Aufbau des Programmspeichers schrittweise am Beispiel der GCC-Toolchain erläutert.

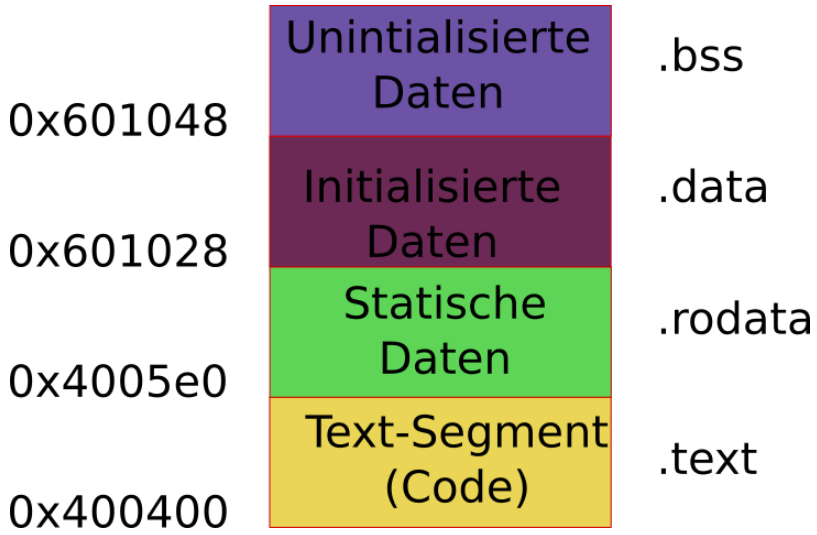


Abbildung 3. Speicherorganisation eines C/C++-Programms mit statischen Segmenten

Speichersegmente

Beim Start eines Programms wird der ausführbare Code aus dem ROM/Flash oder von der Festplatte in den Arbeitsspeicher geladen. Dort ist er in logisch getrennte Segmente gegliedert, die jeweils unterschiedliche Arten von Daten enthalten.

In diesem Kapitel analysieren wir mittels GCC und den GNU-Binutils die Speicherorganisation eines kleinen Programms. Alle Beispiele in diesem Kapitel funktionieren auf Linux, macOS (über Homebrew oder MacPorts) und Windows (MinGW).

Ein Minimalprogramm:

```
int main(int argc, char** argv)
{
    return 0;
}
```

übersetzen wir zunächst:

```
g++ -c memory_layout.cpp
```

Dies erzeugt die Datei `memory_layout.o`. Diese enthält bereits Teile des Programms, ist aber noch nicht ausführbar. Die Struktur lässt sich mit `objdump` untersuchen:

```
objdump -x memory_layout.o
```

Die Ausgabe zeigt die verschiedenen Segmente (Sections) einer Objektdatei:

- `.text` – ausführbarer Code
- `.data` – initialisierte globale/statische Variablen
- `.bss` – nicht initialisierte globale/statische Variablen
- `.rodata` – konstante Daten
- `.comment`, `.eh_frame`, `.note.GNU-stack` – zusätzliche Metadaten

Für das Minimalprogramm ist `.text` sehr klein, `.data` und `.bss` sind leer.

Fügen wir globale Variablen hinzu:

```
int  integer_g = 100;
char character_g;

int main(int argc, char** argv)
{
    return 0;
```

```
}
```

Nun enthält `.data` vier Bytes (das `int`) und `.bss` ein Byte (das `char`):

```
0000000000000000 g     O .data 0000000000000004 integer_g
0000000000000000 g     O .bss  0000000000000001 character_g
```

Globale Variablen mit **initialisiertem Wert** wandern in `.data`. Globale Variablen **ohne Initialisierung** wandern in `.bss`.

Erweitern wir das Programm um eine zusätzliche Funktion:

```
int  integer_g = 100;
char character_g;

int add(int a, int b)
{
    return a + b;
}

int main(int argc, char** argv)
{
    int a = 100;
    int b = 200;
    int c = 0;
    c = add(a,b);
    return 0;
}
```

Hier wächst `.text`, da mehr ausführbarer Code vorhanden ist. `.data` und `.bss` bleiben unverändert.

Das Segment für den Programmcode `.text`

Das `.text`-Segment enthält den ausführbaren Programmcode. Es ist als **read-only** markiert, da Maschinencode zur Laufzeit nicht verändert werden soll.

Auf Mikrocontrollern wird `.text` häufig direkt aus dem Flash ausgeführt (XIP – Execute In Place).

Das Daten-Segment `.data`

Hier werden alle globalen oder statischen Variablen abgelegt, die mit einem Initialwert versehen sind und zur Laufzeit verändert werden können.

Beispiele:

```
int counter = 10;
```

```
float factor = 1.5f;
```

Diese Werte müssen beim Programmstart in den RAM kopiert werden, weshalb sie im ELF-Dateiformat im ROM abgelegt und beim Start des Programms initialisiert werden.

Das Segment der uninitialisierten Daten `.bss`

Das `.bss`-Segment enthält alle globalen und statischen Variablen, die **nicht** explizit initialisiert sind:

```
static int buffer[1024];   // im .bss
int x;                     // im .bss
```

Diese Variablen werden vom Startup-Code des Programms vor dem Eintritt in `main` automatisch **auf Null gesetzt**.

Der Name `.bss` steht historisch für **Block Started by Symbol**.

Das Segment für konstante Daten `.rodata`

Konstanten, die mit `const` deklariert werden, landen im `.rodata`-Segment:

```
const float PI = 3.14159265359f;
```

Konstante Daten können – sofern die Plattform dies unterstützt – direkt im ROM verbleiben. Sie müssen nicht in den RAM kopiert werden.

Ein Beispielprogramm zur Veranschaulichung:

```
#include <stdio.h>

int a;            // .bss
int c = 0;        // .data
int b = 100;      // .data
const int d = 42; // .rodata

int main() {
    printf("Address of a: %p  value: %d\n", &a, a);
    printf("Address of b: %p  value: %d\n", &b, b);
    printf("Address of c: %p  value: %d\n", &c, c);
    printf("Address of d: %p  value: %d\n", &d, d);
    printf("sizeof(int): %lu\n", sizeof(int));
    return 0;
}
```

Übersicht des Memory-Layouts

Das folgende Diagramm zeigt einen typischen Aufbau des virtuellen Adressraums eines Prozesses mit Stack, Heap und den wichtigsten Segmenten:

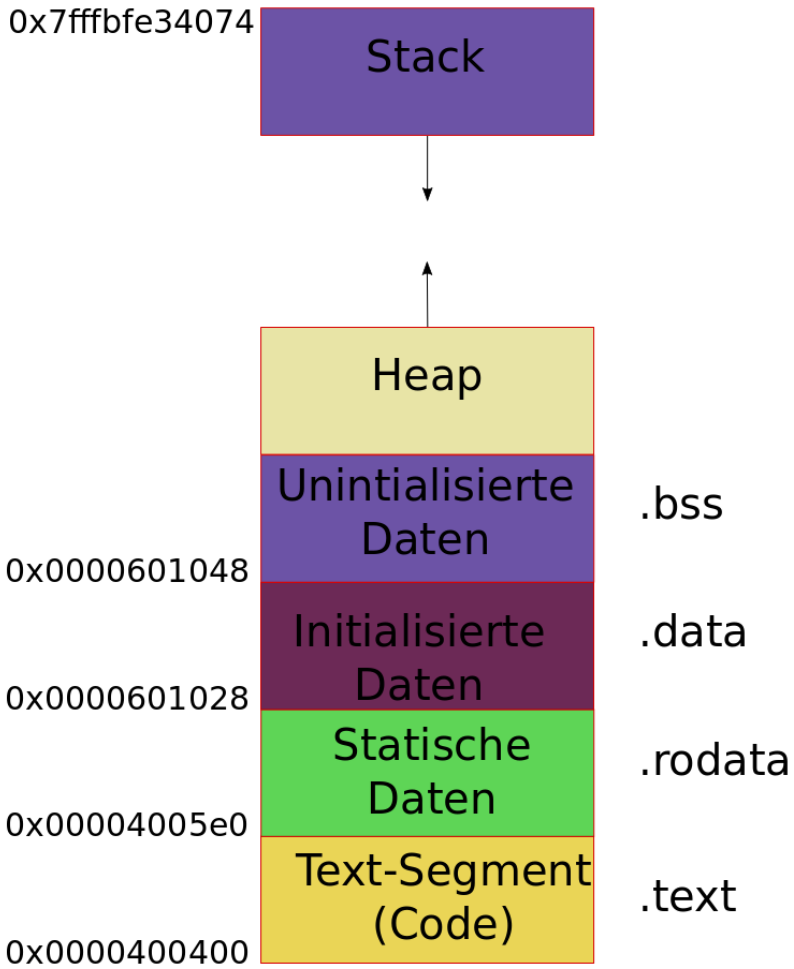


Abbildung 4. Speicherorganisation eines C/C++-Programms mit Stack, Heap und statischen Segmenten

Wo werden lokale Variablen abgelegt?

Nun stellt sich die Frage: **Wo landen Variablen, die innerhalb von Funktionen definiert werden?** Sie befinden sich weder in `.data` noch in `.bss`.

Die Antwort: **auf dem Stack**. Dies wird im nächsten Abschnitt erläutert.

Der Stack

Der Stack ist ein Speicherbereich, der nach dem LIFO-Prinzip (Last In – First Out) organisiert ist. Er dient zur Verwaltung von:

- lokalen Variablen
- Funktionsparametern
- Rücksprungadressen
- gesicherten Registern
- temporären Daten

Jede Funktion besitzt einen sogenannten **Stack Frame**, der beim Betreten der Funktion angelegt und beim Verlassen wieder freigegeben wird.

Maximalgröße des Stacks

Typische Standardwerte:

- Visual Studio: **1 MB**
- GCC unter Linux: **8 MB**

Viele Compiler erlauben das Anpassen der Stackgröße. Bei Mikrocontrollern ist die Stackgröße oft im Linkerskript fest definiert und sehr begrenzt (z. B. 2–16 KB).

Der Call-Stack

Beim Aufruf einer Funktion werden mehrere Dinge auf dem Stack gespeichert:

- Funktionsparameter
- Rücksprungadresse
- ggf. der Framepointer
- gesicherte CPU-Register
- lokale Variablen

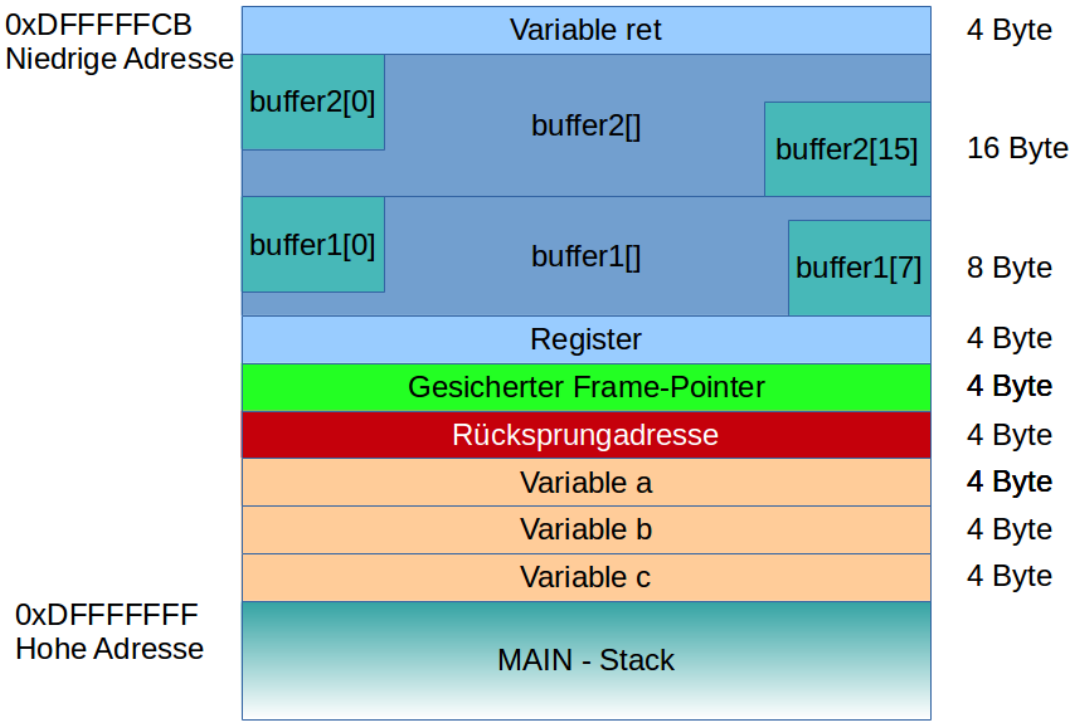
Beispielprogramm:

```
#include <stdio.h>

void function(int a, int b, int c) {
    char buffer1[8];
    char buffer2[16];
    int* ret;
    ret = (int*)(buffer2 + 16);
    (*ret) += 7;
}
```

```c
int main(void) {
    int x = 0;
    function(1, 2, 3);
    x = 1;
    printf("x = %d\n", x);
    return 0;
}
```

Der zugehörige Stack-Frame ist schematisch in der folgenden Abbildung dargestellt:



Aufrufparameter

Parameter werden entweder:

- auf dem Stack übergeben (klassisch), oder
- über Register (z. B. ARM EABI: R0–R3 für die ersten vier Parameter)

Parameter, die per Kopie übergeben werden, verlieren ihren Gültigkeitsbereich, sobald die Funktion endet. Für große Objekte empfiehlt sich die Übergabe per Zeiger oder Referenz.

Rücksprungadresse

Vor dem Sprung in die Funktion speichert die CPU die Adresse der nächsten auszuführenden Anweisung im Stack. Der Rücksprungbefehl kehrt anschließend genau an diese Stelle zurück.

Der Framepointer

Manche Compiler verwenden ein Framepointer-Register, um auf lokale Variablen und Parameter innerhalb des aktuellen Stackframes leichter zugreifen zu können. Wird ein Framepointer genutzt, muss er ebenfalls gesichert und wiederhergestellt werden.

Der Stackpointer

Der Stackpointer zeigt auf die aktuelle Spitze des Stacks. Bei den meisten Architekturen zu niedrigeren Adressen hin.

Register

Wenn ein Unterprogramm CPU-Register benötigt, müssen diese gesichert und beim Verlassen wiederhergestellt werden. Dies geschieht üblicherweise automatisch gemäß der Calling Convention der jeweiligen Plattform.

Lokale Variablen

Lokale Variablen leben ausschließlich innerhalb der Lebensdauer des jeweiligen Stackframes. Nach Verlassen der Funktion ist der Speicher formal wieder freigegeben und darf nicht mehr verwendet werden.

Rückgabewerte

Rückgabewerte werden je nach Architektur

- in einem Register (z. B. `R0`),
- oder über den Stack

an den Aufrufer übergeben. Ein Rückgabewert, der als Zeiger auf eine lokale Variable implementiert ist, ist **immer ungültig**.

Fragen

- Was passiert, wenn eine Funktion rekursiv ist?
- Wie werden große Objekte effizient an eine Funktion übergeben?
- Welche Gefahren bestehen, wenn die Rücksprungadresse überschrieben wird?

Der Heap

Der Heap dient zur dynamischen Speicherverwaltung. Er wird verwendet, wenn:

- die benötigte Speichergröße zur Kompilierzeit unbekannt ist,
- große Datenmengen reserviert werden sollen, oder
- Speicher über Funktionsgrenzen hinweg existieren muss.

In C:

```
float* p = malloc(sizeof(float));
if (p != NULL) {
    /* ... */
    free(p);
}
```

In C++:

```
float* p = new float;
delete p;
```

Regel: Niemals `malloc/free` und `new/delete` mischen.

Anfordern von Speicher

Die Laufzeitbibliothek verwaltet den Heap in Blöcken unterschiedlicher Größe. Dadurch kann Fragmentierung entstehen: Gesamtfreier Speicher ist vorhanden, aber nicht als zusammenhängender Block.

Dies führt zu potenziell **nicht deterministischem Verhalten**, weshalb in Echtzeitsystemen dynamische Speicherverwaltung oft eingeschränkt oder komplett verboten ist.

Kann kein Speicher zugeteilt werden:

- `malloc()` liefert `NULL` zurück.
- `new` wirft standardmäßig eine Exception (`std::bad_alloc`), sofern nicht `nothrow` verwendet wird.

CPU-Cache

> There are 2 hard problems in computer science: cache invalidation, naming things, and off-by-1 errors.
>
> — Leon "Zero-Width-Space-Character" Bambrick, Twitter @SecretGeek

Der NVIDIA Tegra TX2

Die Funktionsweise und Bedeutung moderner CPU-Caches betrachten wir exemplarisch am NVIDIA Jetson TX2 Tegra „Parker“ SoC. Dieses enthält sowohl Denver-2- als auch ARM-Cortex-A57-Kerne sowie eine integrierte Pascal-GPU.

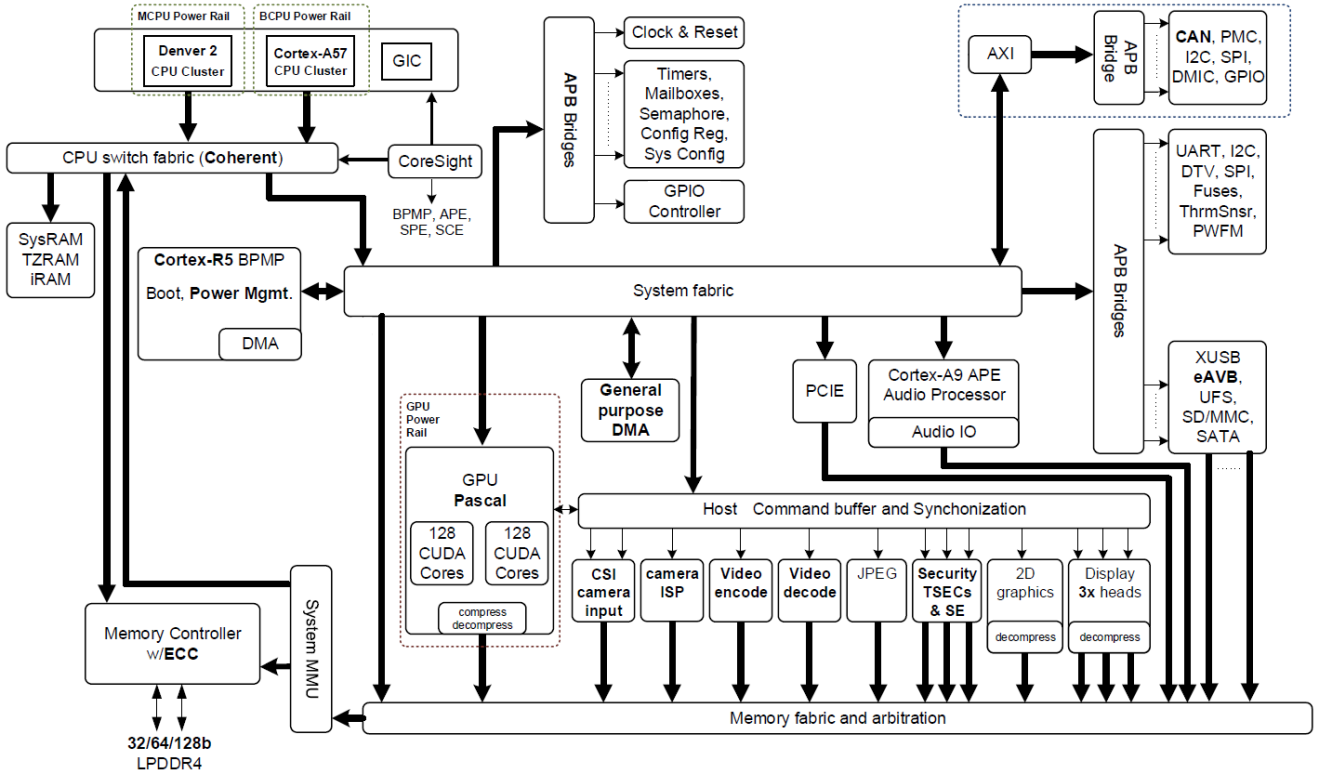


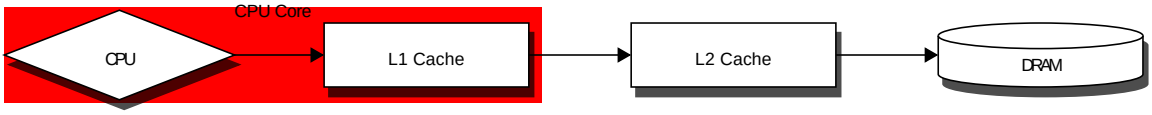
Abbildung 5. NVIDIA Jetson TX2 Tegra „Parker“ SoC block diagram featuring integrated NVIDIA Pascal GPU, NVIDIA Denver 2 + ARM Cortex-A57 CPU clusters, and multimedia acceleration engines [1]

Cache-Hierarchie

Der Cortex-A57-CPU-Komplex des Jetson TX2 verfügt pro CPU-Kern über:

- **48 KB Instruction Cache (I-Cache)**
- **32 KB Data Cache (D-Cache)**

Diese bilden den **L1-Cache**. Alle vier Cortex-A57-Kerne teilen sich außerdem einen gemeinsamen **L2-Cache mit 2 MB Kapazität**.



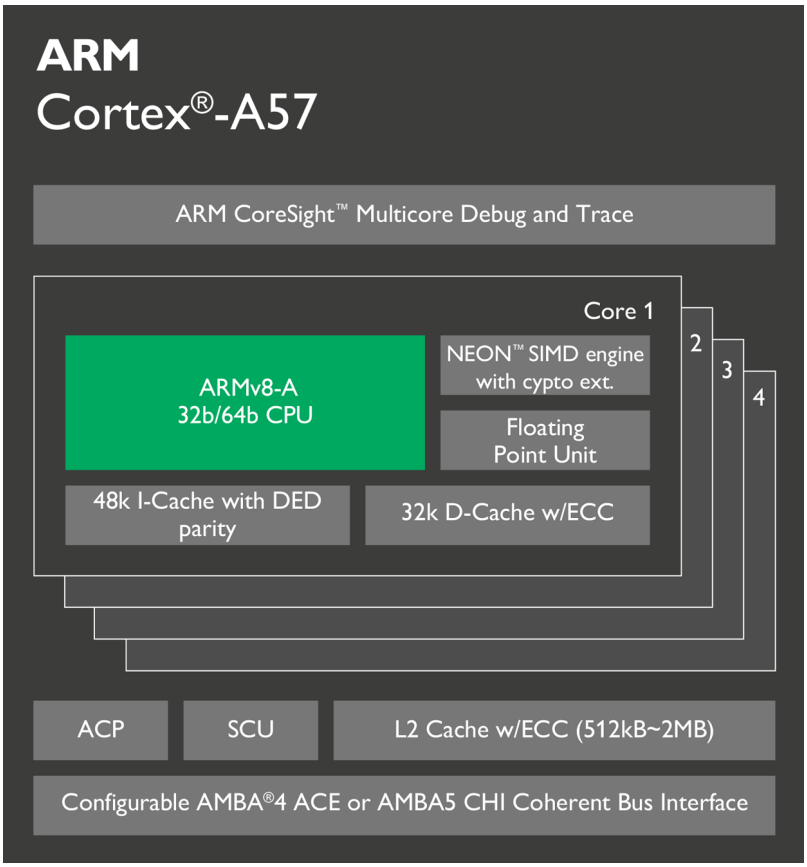


Abbildung 6. Der ARM Cortex-A57 CPU-Komplex [2]

Instruction Caches

Instruction Caches (I-Caches) speichern ausführbaren Programmcode. Sie sorgen dafür, dass Instruktionen mit möglichst geringer Latenz geladen werden können.

Wesentliche Eigenschaften:

- ausschließlich lesend aus Sicht der CPU
- sehr geringe Zugriffszeiten (wenige Zyklen)
- optimiert für sequentielle Programmausführung
- eng verzahnt mit **Branch Prediction** und **Prefetching**

Ein **I-Cache-Miss** führt zu einem Nachladen aus dem L2-Cache oder aus dem DRAM. Dies kann die Pipeline der CPU deutlich verlangsamen.

Typische Situationen, die von I-Caches profitieren:

- kleine, häufig genutzte Funktionen
- kompakte Schleifen (Loop Bodies)

- deterministische Ablaufpfade
- optimierte Bibliotheksfunktionen

Data Caches

Data Caches (D-Caches) speichern häufig benutzte Daten. Sie arbeiten mit **Cache-Lines**, die beim Cortex-A57 typischerweise **64 Byte** groß sind.

Cache-Schreibrichtlinien

Der Cortex-A57 verwendet standardmäßig:

- **Write-Back**: Schreibzugriffe verbleiben zunächst im Cache; das Rückschreiben ins RAM erfolgt verzögert.
- optional **Write-Through**: Jeder Schreibzugriff landet sofort im RAM (langsamer, aber einfacher).

Cache-Kohärenz

Da mehrere Kerne auf denselben Speicher zugreifen, sorgt ein MESI-basiertes Kohärenzprotokoll dafür, dass jeder Kern konsistente Daten erhält.

Ein **Cache-Miss** tritt auf, wenn Daten:

- noch nicht im Cache vorhanden sind,
- aus dem Cache verdrängt wurden, oder
- durch DMA oder andere Kerne invalidiert wurden.

Ein Miss erzeugt typischerweise eine Verzögerung von **100 bis 300 CPU-Zyklen**.

Schreiben cachefreundlicher Software

Die Leistung moderner CPUs hängt stark davon ab, wie gut Software auf das Cache-Verhalten abgestimmt ist. Die folgenden Grundregeln helfen, Programme cachefreundlicher zu gestalten.

1. Räumliche Lokalität (Spatial Locality)

Daten sollten möglichst **linear im Speicher** angeordnet sein.

```
for (int i = 0; i < n; i++) {
    sum += a[i];   // optimal: sequentieller Zugriff
}
```

Ungünstig sind große Sprünge:

```
for (int i = 0; i < n; i += 64) {
```

```
    sum += a[i];   // schlecht: kann jede Cache-Line überspringen
}
```

2. Zeitliche Lokalität (Temporal Locality)

Daten, die mehrfach benötigt werden, sollten nah beieinander liegen und wiederholt genutzt werden, damit sie im Cache bleiben.

3. Datenstrukturen optimieren

- Arrays statt verketteter Listen (bessere Lokalität)
- kleine Strukturen bevorzugen
- bei Vektoroperationen ggf. SoA (Structure of Arrays) statt AoS (Array of Structures) verwenden

4. Cache-Line-Ausrichtung beachten

Viele Architekturen verwenden eine **Ausrichtung auf 64 Bit oder 64 Byte**. Falsche Ausrichtung kann zu geteilten Lese- oder Schreibzugriffen führen.

5. „False Sharing“ vermeiden

Wenn zwei Threads unterschiedliche Variablen verwenden, die in derselben Cache-Line liegen, führt dies zu unnötigem Kohärenzverkehr.

Lösung:

```
alignas(64) int counter1;
alignas(64) int counter2;
```

6. Prefetching nutzen

Einige Compiler unterstützen explizites Prefetching:

```
__builtin_prefetch(&a[i + 64]);
```

Dies kann insbesondere bei großen Datenmengen hilfreich sein.

7. Zusammenhängende Speicherlayouts bevorzugen

- `std::vector` statt `std::list`
- statische Arrays fördern gute räumliche Lokalität
- Pointer-basierte Strukturen nur, wenn unbedingt notwendig

Zusammenfassung

- Der Jetson TX2 besitzt pro Core 48 KB I-Cache und 32 KB D-Cache sowie einen gemeinsamen 2 MB großen L2-Cache.
- **Cache-Hits** sind sehr schnell, **Cache-Misses** extrem teuer.
- Gute Speicherlokalität ist ein wesentlicher Faktor für die Performanz.
- Durch geeignete Datenstrukturen und Zugriffsstrategien kann Software erheblich beschleunigt werden.

[1] Quelle: https://devblogs.nvidia.com/jetson-tx2-delivers-twice-intelligence-edge/ Abgerufen: 20.10.2019

[2] Quelle: https://developer.arm.com/ip-products/processors/cortex-a/cortex-a57 Abgerufen: 20.10.2019

Rechnerarchitektur

RISC-Architektur

RISC ist die Abkürzung für **R**educed **I**nstruction **S**et. RISC-Prozessoren verfolgen die Strategie, dass wenige Befehle, die mit wenigen CPU-Zyklen auskommen, zu einer höheren Performance führen.

Typische Vertreter der RISC-Architektur sind die folgenden Prozessorfamilien:

- Blackfin
- ARM
- PowerPC
- MIPS

CISC-Architektur

CISC ist die Abkürzung für **C**omplex **I**nstruction **S**et. Die Philosophie der CISC-Prozessoren ist es, komplexere Befehle bereitzustellen, die in mehrere kleine Schritte unterteilt werden können. Eine Lösung dieser Aufgabenstellung bestand beispielsweise in der Mikroprogrammierung. Dabei werden komplexe Befehle mittels sogenannter Mikroprogramme aufgebaut. Im Laufe der Zeit wurden die CPUs dadurch jedoch immer komplexer und schwieriger weiterzuentwickeln. Daher kommt es vor, dass ein Prozessorhersteller den Mikrocode einer CPU patchen muss. Dies war zum Beispiel bei einem Bug in Intels Skylake-Architektur notwendig. "Die Fähigkeit, solche Updates nachzuladen, hat Intel mit dem Pentium Pro (P6) eingeführt. Der berühmte FDIV-Bug im Pentium hatte auf harte Weise gelehrt, dass man unbedingt nachträgliche Patch-Möglichkeiten im internen Betriebsablauf braucht." Quelle: Andreas Stiller, Prozessor-Patches, c’t 2016, Heft 7. Ein Überblick über aktuelle Mikrocode-Patches findet sich in der folgenden Abbildung:

Microcode-Updates für neuere Prozessoren								
	Windows 8.1		Windows 10		Linux		UBU	
Intel								
Datei	mcupdate_GenuineIntel.dll		mcupdate_GenuineIntel.dll		microcode.dat		UBU v1.49 Update 2.1	
Datum	12. 5. 2015		5. 11. 2015		6. 11. 2015		6. 2. 2016	
Ivy Bridge 306A9	1Bh	29. 5. 2014	1Bh	29. 5. 2014	1Ch	26. 2. 2015	19h	13. 6. 2013
Haswell core i5/ i7 306C3	1Ch	29. 5. 2014	1Eh	13. 8. 2015	1Eh	13. 8. 2015	19h	16. 7. 2015
Broadwell Core i5 306D4	1Fh	27. 2. 2015	1Fh	27. 2. 2015	22h	11. 9. 2015	–	–
Ivy Bridge EP 306E4	428h	29. 5. 2014	428h	29. 5. 2014	428h	29. 5. 2014	–	–
Broadwell Y 306E7	70Dh	8. 5. 2014	70Dh	28. 5. 2014	70Dh	28. 5. 2014	–	–
Haswell-E 306F2	2Eh	23. 2. 2015	36h	20. 8. 2015	36h	20. 8. 2015	36h	2. 10. 2015
Haswell-E 306F4	05h	9. 3. 2015	9h	17. 7. 2015	9h	17. 7. 2015	–	–
Haswell ULT 40651	1Ch	3. 7. 2014	1Dh	13. 8. 2015	1Dh	13. 8. 2015	–	–
Crystal Well 40661	12h	3. 7. 2014	12h	3. 7. 2014	–		–	–
Broadwell H 40671	–		–		13h	3. 8. 2015	13h	3. 8. 2015
Skylake 506E3	–		–		–		7Ch	6. 2. 2016
AMD								
Datei	mcupdate_ AuthenticAMD.dll		mcupdate_ AuthenticAMD.dll		amd-ucode-latest.tar			
Datum	20. 8. 2013		10. 7. 2015		25. 2. 2015			
Fam 14 5010	29h	21. 1. 2013	29h	21. 1. 2013	29h	21. 1. 2013		
Fam 14 5020	119h	18. 1. 2013	119h	18. 1. 2013	119h	18. 1. 2013		
Fam 15 6012	624h	21.10. 2011	624h	21. 10. 2011	63Dh	6. 3. 2013		
Fam 15 6020	–		–		832h	23. 7. 2014		
Fam 15 6101	1116h	3. 5. 2012	1116h	3. 5. 2012	1119h	13. 7. 2012		
Fam 16	–		–		10Fh	29. 2. 2014		

Load/Store-Architektur

Eine Load/Store-Architektur unterteilt die Maschinenbefehle in zwei Gruppen:

- Operationen für den Speicherzugriff (Load und Store zwischen Speicher und Registern)
- ALU-Operationen, die nur auf Registern arbeiten.

Die Load/Store-Architektur ist bei RISC-Architekturen sehr weit verbreitet.

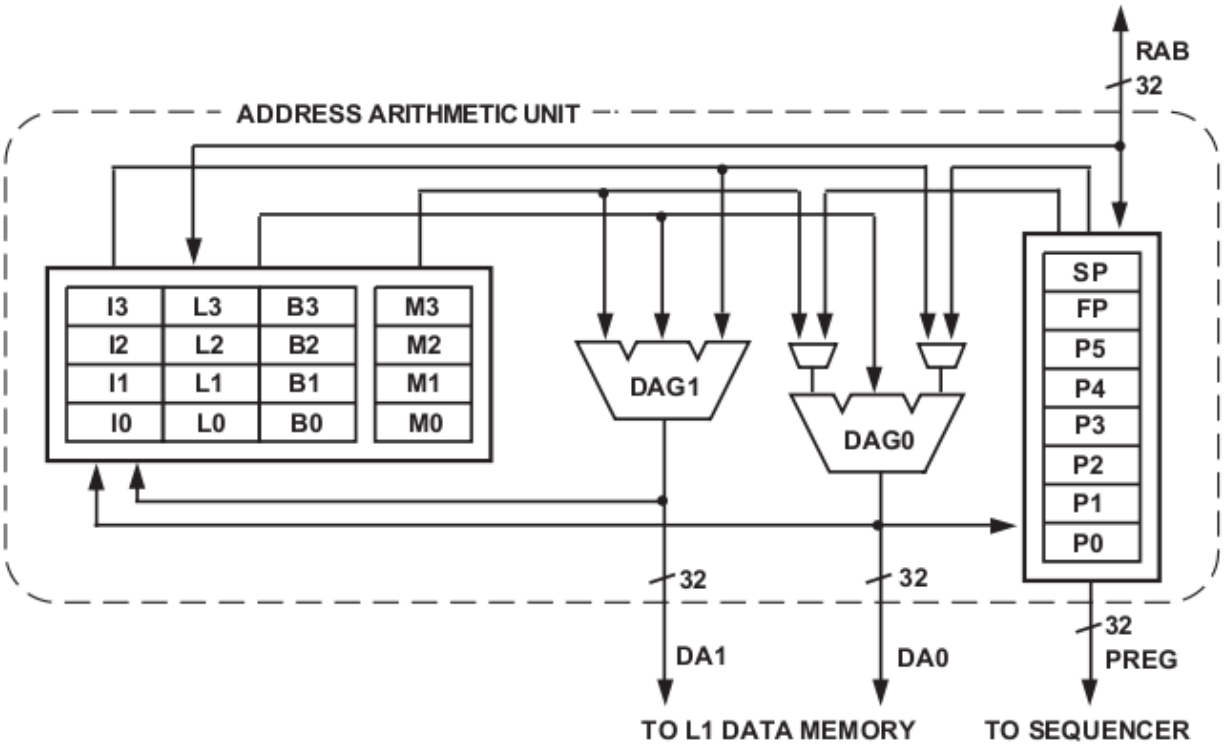
Die Blackfin AAU

Wie die meisten DSPs besitzt auch die Blackfin-CPU-Familie von Analog Devices eine Load/Store-Architektur. Operanden und Ergebnisse werden immer durch Register repräsentiert. Die Operanden werden aus dem Speicher in Register geladen. Dasselbe gilt für die Ergebnisse: Diese werden mittels eines expliziten `move`-Befehls in den Hauptspeicher geschrieben.

Für den Transport der Daten aus und in den Hauptspeicher ist die *AAU* (**A**ddress **A**rithmetic **U**nit) zuständig. Diese sorgt für einen effizienten Transfer der Daten.

Mit einer separaten Arithmetikeinheit für die Adressberechnungen wird verhindert, dass die Verarbeitungseinheit durch die Adressberechnung belastet wird. Dadurch ist es nicht nur möglich, Lade- und Speicheroperationen parallel zu Datenberechnungen durchzuführen, sondern auch die

Berechnung der Speicheradressen auszulagern. Die *AAU* verwendet sogenannte **D**aten-**A**dressen-**G**eneratoren (DAGs), um die Adressen für Speicheroperationen zu berechnen. Die AAU ist in der folgenden Abbildung dargestellt.



Eine AAU besteht aus zwei DAG-Einheiten, neun Zeiger-Registern, vier Index-Registern und vier kompletten Sätzen an Modifizier-, Basis- und Längen-Registern. Diese Register enthalten Werte, auf die die DAGs für die Adressberechnung zurückgreifen.

- Index-Register *I[0:3]*: 32-Bit-Register, die einen Adresszeiger enthalten. So lädt die Operation `R3 = [I0]` den Inhalt der Adresse, die in `I0` gespeichert ist, in das Register `R3`.
- Modifizier-Register *M[0:3]*: Diese vorzeichenbehafteten 32-Bit-Register stellen einen Inkrementwert beziehungsweise eine Schrittweite dar.
- Basis- und Längen-Register *B[0:3]*: Sie repräsentieren die Basis und die Länge eines Puffers. Jedes Paar aus `B`- und `L`-Registern ist mit dem passenden `I`-Register gekoppelt.
- Zeiger-Register *P[0:5], FP, SP*: Diese 32-Bit-Register enthalten jeweils einen Zeiger auf einen Speicherbereich. Sie sind jedoch nicht an die `B`-, `L`- und `I`-Register gekoppelt.

Der STM32F103 Mikrocontroller

Beim STM32F103 handelt es sich um einen Arm-Cortex-M3-32-Bit-RISC-Core. Damit gehört diese CPU zu den Vertretern der RISC-Architektur. Der Baustein kann mit bis zu 72 MHz getaktet werden. Er ist in unterschiedlichen Speicherausbauvarianten erhältlich; maximal stehen 128 KByte Flash und 20 KByte SRAM zur Verfügung.

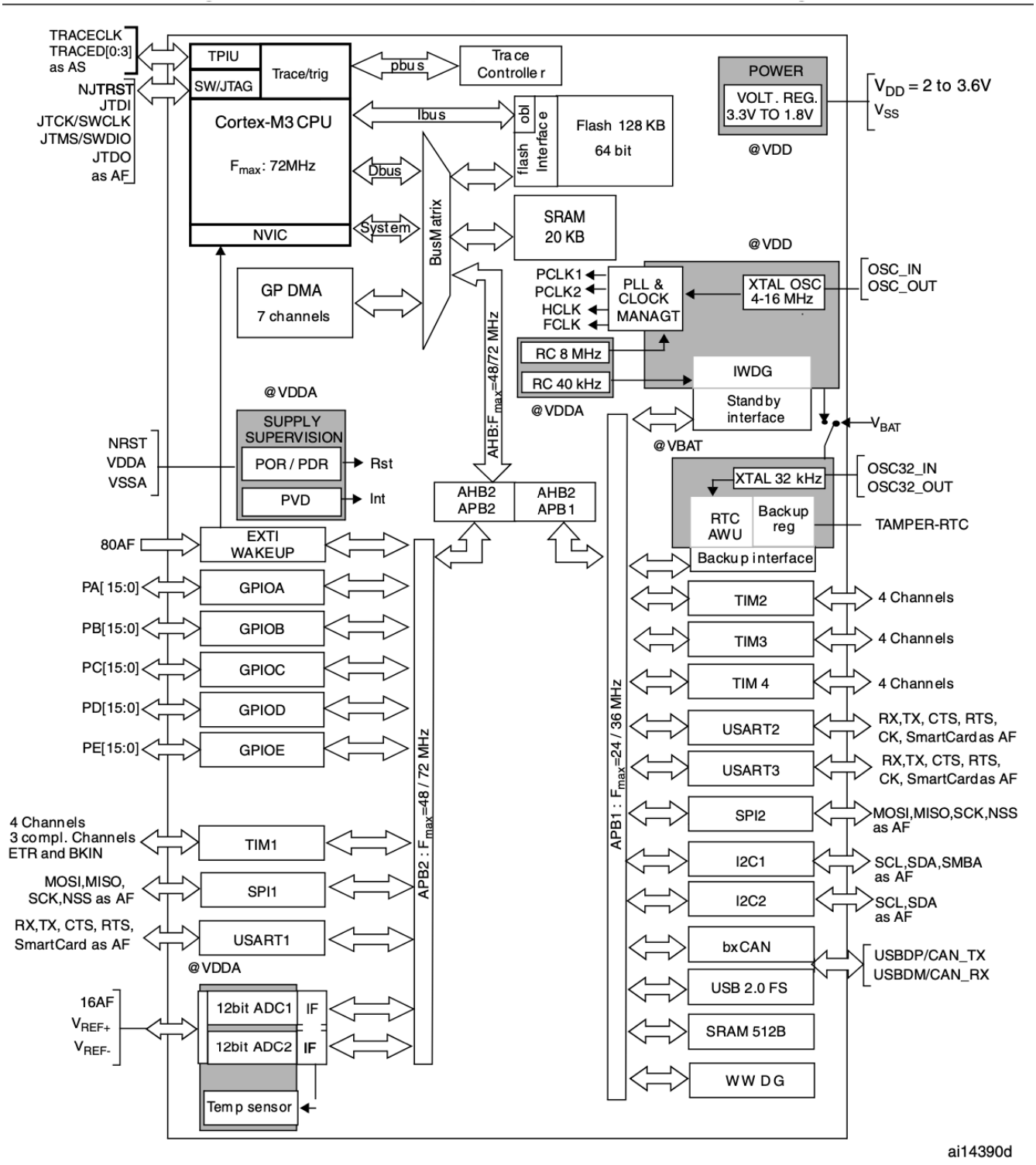


Abbildung 7. Blockdiagramm des STM32F103 Quelle: [1 - Grundlagen der C++-Programmierung]

Der ARM Cortex M3 verfügt über eine dreistufige Befehlspipeline.

Pipeline

Wikipedia Die Pipeline (auch Befehls-Pipeline oder Prozessor-Pipeline) bezeichnet bei Mikroprozessoren eine Art „Fließband“, mit dem die Abarbeitung der Maschinenbefehle in Teilaufgaben zerlegt wird, die für mehrere Befehle parallel durchgeführt werden. Dieses Prinzip, oft auch kurz Pipelining genannt, stellt eine weit verbreitete Mikroarchitektur heutiger Prozessoren dar.

Statt eines gesamten Befehls wird während eines Taktzyklus des Prozessors nur jeweils eine Teilaufgabe abgearbeitet, allerdings werden die verschiedenen Teilaufgaben mehrerer Befehle dabei gleichzeitig bearbeitet. Da diese Teilaufgaben einfacher (und somit schneller) sind als die Abarbeitung des gesamten Befehls am Stück, kann durch Pipelining die Effizienz der Taktfrequenz des Mikroprozessors gesteigert werden. Insgesamt benötigt ein einzelner Befehl nun mehrere Takte zur Ausführung, da aber durch die quasi parallele Bearbeitung mehrerer Befehle in jedem Zyklus ein Befehl „fertiggestellt“ wird, wird der Gesamtdurchsatz durch dieses Verfahren erhöht.

Die einzelnen Teilaufgaben einer Pipeline nennt man Pipeline-Stufen, Pipeline-Stages oder auch Pipeline-Segmente. Diese Stufen werden durch getaktete Pipeline-Register getrennt. Neben einer Befehls-Pipeline kommen in modernen Systemen verschiedene weitere Pipelines zum Einsatz, beispielsweise eine Arithmetik-Pipeline in der Gleitkommaeinheit. **Wikipedia**

Die Cortex-M3-Pipeline

Die Cortex-M3-Familie der ARM-Mikroprozessoren, wie sie zum Beispiel im STM32F103 zum Einsatz kommt, verfügt über eine dreistufige Pipeline. Diese dreistufige Pipeline ist in der folgenden Abbildung dargestellt.

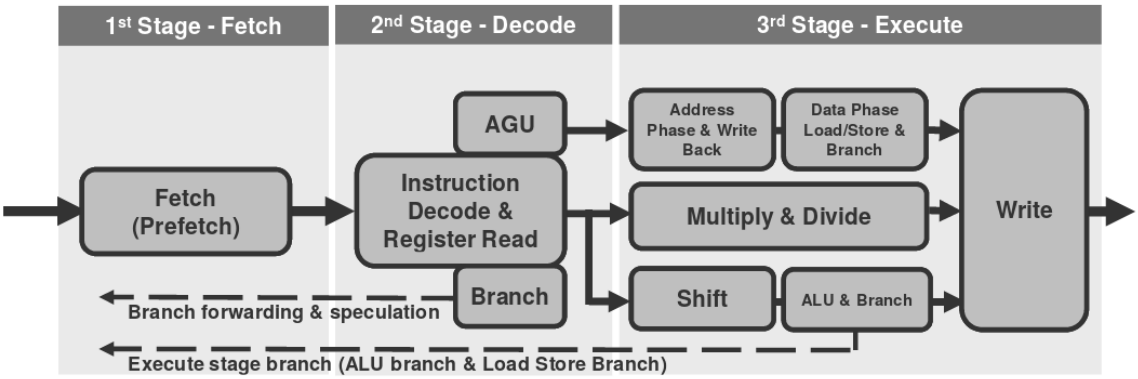


Abbildung 8. Cortex-M3-Pipeline Quelle: [1]

Eine optimale Pipelineausnutzung ist in der Abbildung dargestellt. Bei einer optimalen Ausnutzung der Pipeline dauert es n Schritte, bis die Pipeline gefüllt ist. Anschließend wird mit jedem Takt ein Ergebnis erzeugt. Alle Operationen finden auf Registern statt. Dieses Beispiel benötigt 6 Takte für die Abarbeitung. .Pipeline-Abarbeitung ohne Sprung Quelle: [1] image::img/Cortex-M3-Pipeline-jump.png[]

Findet ein Sprung in der Befehlsabarbeitung statt, so benötigt dies 3 Takte, um den Sprung

abzuarbeiten. Dies ist in der folgenden Abbildung dargestellt.

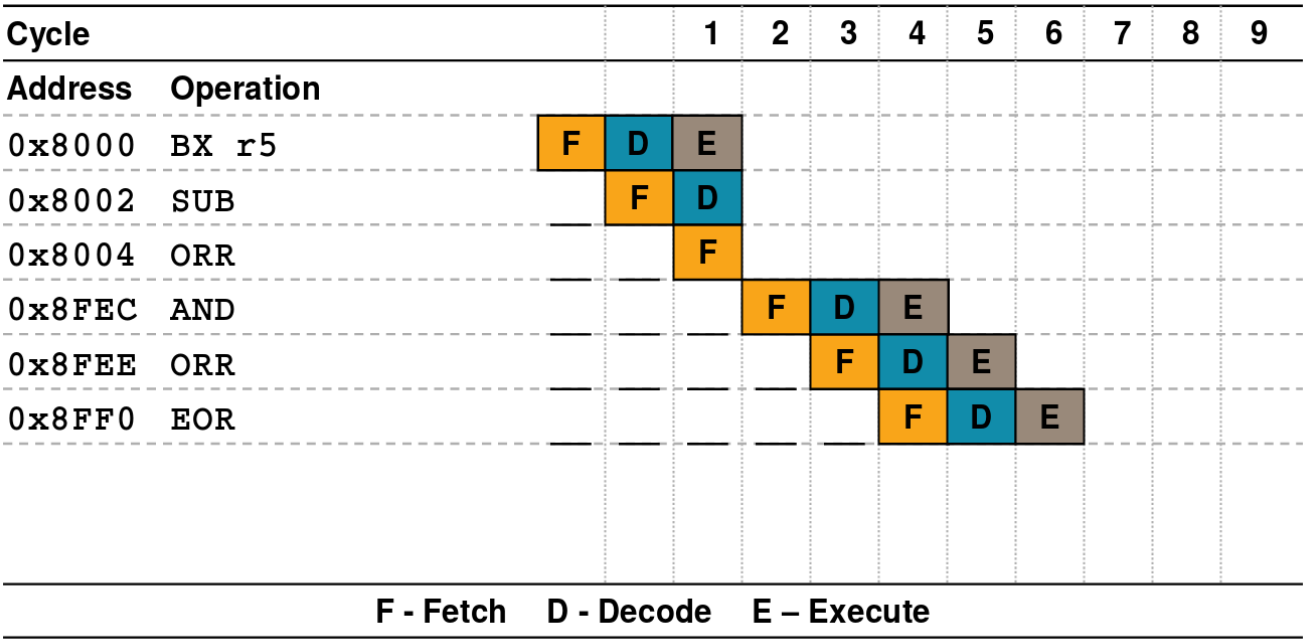


Abbildung 9. Pipeline-Abarbeitung mit Sprung Quelle: [1]

Effizientes Schreiben von Code ohne Conditionals

Da der ARM Cortex-M3 im STM32F103 eine dreistufige Pipeline besitzt, sind Sprungbefehle (Branches) vergleichsweise teuer. Jeder bedingte Sprung (`if`, `switch`, `?:`, Schleifen mit komplexen Bedingungen) kann zu einem sogenannten **Pipeline Flush** führen. Dadurch muss die Pipeline neu gefüllt werden, was typischerweise **2–3 zusätzliche Takte** kostet.

Um diese Verzögerungen zu vermeiden, kann in vielen Fällen **branchless code** geschrieben werden, also Code ohne bedingte Verzweigungen.

Warum vermeiden wir Sprünge?

- jeder bedingte Sprung kann die Pipeline leeren
- die Cortex-M3-Architektur besitzt **keine Branch Prediction** im klassischen Sinn
- falsch eingeschätzte Sprünge können 3 Takte Verlust bedeuten
- bitweise Operationen und arithmetische Tricks sind oft schneller
- Compiler können manche Bedingung nicht optimal optimieren

Beispiel: Absolute Wertberechnung

Schlechter Ansatz (mit Sprung):

```
int abs_val(int x) {
    if (x < 0) {
        return -x;
    }
```

```
    return x;
}
```

Besser (ohne Sprung):

```
int abs_val(int x) {
    int mask = x >> 31;      // ergibt 0x00000000 oder 0xFFFFFFFF
    return (x ^ mask) - mask;
}
```

Ergebnis:

- **kein Sprung**, keine Pipeline-Unterbrechung
- konstante Laufzeit
- ideal für zeitkritische Routinen

Beispiel: Clamping (Begrenzen eines Werts)

Mit Verzweigungen:

```
if (val < 0)      val = 0;
else if (val > 255) val = 255;
```

Branchless:

```
val = val * (val >= 0);
val = 255 - ((255 - val) * (val <= 255));
```

Oder lesbarer (je nach Compiler werden diese Ausdrücke in bedingte Instruktionen ohne Verzweigung übersetzt):

```
val = (val < 0) ? 0 : val;
val = (val > 255) ? 255 : val;
```

→ Moderne Compiler wandeln diese ternären Operatoren häufig in **branchless Maschinenbefehle** um.

Beispiel: Auswahl zwischen zwei Werten ohne if

```
int select(int cond, int a, int b) {
    return (cond * a) + (!cond * b);
}
```

Hier erzeugt `cond` entweder:

- `1 * a + 0 * b` oder
- `0 * a + 1 * b`

→ völlig ohne Sprung.

Bitmaskierung als Alternative

Viele Operationen können über bitweise Masken geschrieben werden:

```
uint32_t out = (mask & a) | (~mask & b);
```

Dies ermöglicht:

- kein Sprung
- konstante Ausführungszeit
- ideal für ISR, DSP- oder Echtzeitanwendungen

Schleifen optimieren

Auch in Schleifen können Branches reduziert werden:

- konstante Schleifenlängen bevorzugen
- inneren Schleifenkörper bedingungslos halten
- bedingte Teile nach außen ziehen

Beispiel (schlecht):

```
for (int i = 0; i < N; i++) {
    if (data[i] != 0) {
        sum += data[i];
    }
}
```

Besser (branchless):

```
for (int i = 0; i < N; i++) {
    int m = (data[i] != 0);
    sum += data[i] * m;
}
```

Lookup-Tabellen als Ersatz für Bedingungslogik

Beispiel: Zustandsmaschine

```
next_state = transition_table[current_state][input_symbol];
```

Vorteile:

- keine Sprünge
- extrem schnell
- ideal für Parser, Protokolle, Steuerungslogik

Wann Branchless Code sinnvoll ist?

- zeitkritische ISR oder Loops
- Berechnungen mit kleinen Ganzzahlen
- Signalverarbeitung / Filter
- Compileroptimierungen nicht ausreichen
- deterministische Laufzeit benötigt wird

Wann Branchless Code NICHT sinnvoll ist?

- wenn Lesbarkeit wesentlich leidet
- wenn der Compiler selbst bereits optimal optimiert
- wenn branchless Berechnungen mehr Takte benötigen als der Sprung

Der Cortex-M3 ist gutmütig bei normalen `if`-Pfaden. Erst in inneren Schleifen, oder wenn Sprungabhängigkeiten eng aufeinander folgen, lohnt sich branchless-Optimierung.

Zusammenfassung

- Der Cortex-M3 hat keine klassische Branch Prediction → Sprünge kosten Takte.
- Branchless Programming kann Pipeline-Stalls vermeiden.
- Bitmasken, Arithmetik-Tricks und Lookup-Tabellen helfen, Sprünge zu vermeiden.
- Für zeitkritische Bereiche kann dies erhebliche Performancevorteile bringen.

[1] ARM Cortex - M3 Introduction ARM University Relations

System- und Schnittstellenbausteine des STM32F103

Dieses Kapitel ergänzt die Architekturgrundlagen um die wichtigsten Mikrocontroller-Bausteine, die in realen Embedded-Systemen täglich verwendet werden. Im Fokus stehen dabei der STM32F103 und die ARM-Cortex-M3-Plattform.

Peripherie im Adressraum

Der STM32F103 verwendet **Memory-Mapped I/O**: Peripheriebausteine werden über normale Speicheradressen angesprochen.

Typische Adressbereiche:

- Flash / Programmspeicher (STM32F103 typischerweise ab `0x0800_0000`)
- SRAM (ab `0x2000_0000`)
- Peripherie (ab `0x4000_0000`, z. B. APB1, APB2, AHB)
- Systembereich (z. B. NVIC, SysTick, SCB)

Der Bereich `0x0000_0000` ist beim STM32F103 bootabhängig aliasiert und zeigt je nach BOOT-Konfiguration auf Flash, System Memory oder SRAM.

Ein Registerzugriff sieht in C wie ein Speicherzugriff aus, wirkt aber auf Hardware:

```
typedef struct {
    volatile uint32_t CRL;
    volatile uint32_t CRH;
    volatile uint32_t IDR;
    volatile uint32_t ODR;
    volatile uint32_t BSRR;
    volatile uint32_t BRR;
    volatile uint32_t LCKR;
} GPIO_TypeDef;

#define GPIOA ((GPIO_TypeDef*)0x40010800U)
```

Wichtig ist das Schlüsselwort `volatile`: Der Compiler darf diese Zugriffe nicht „wegoptimieren“, da sich Registerwerte auch ohne direkten C-Code ändern können (z. B. durch Hardware, DMA oder Interrupts).

Read-Modify-Write und Bit-Banding

Die folgenden Details zu atomaren Bitoperationen, Side-Effects bei Registerzugriffen und Bit-Banding werden im folgenden Unterabschnitt vertieft:

Read-Modify-Write beim Cortex-M3

Bei Mikrocontrollern der ARM-Cortex-M-Familie ist der Zugriff auf einzelne Bits innerhalb von Peripherieregistern ein häufiger Anwendungsfall. Typische Operationen wie „ein Bit setzen“, „ein Bit löschen“ oder „ein Bit toggeln“ werden in C meist als klassische Read-Modify-Write-Operationen (RMW) implementiert:

```
REG |= (1 << 5);   // Bit 5 setzen
REG &= ~(1 << 2);  // Bit 2 löschen
```

Was trivial aussieht, hat auf Hardwareebene jedoch einige Besonderheiten.

Was passiert bei einer RMW-Operation?

Eine Read-Modify-Write-Sequenz besteht aus mehreren Schritten:

1. **Read** – der aktuelle Wert des Registers wird vom Bus geholt.
2. **Modify** – der Prozessorkern verändert den Wert im Registerfile.
3. **Write** – der neue Wert wird in das Peripherieregister zurückgeschrieben.

Der Cortex-M3 kann solche Operationen nicht atomar ausführen – er benötigt mehrere Buszyklen. Währenddessen kann Folgendes passieren:

- ein Interrupt tritt zwischen Read und Write auf
- andere Hardware verändert das gleiche Register
- die Peripherie reagiert empfindlich auf Mehrfachzugriffe
- ein Statusbit wird durch Lesen gelöscht („read-to-clear“)

Dies kann zu unvorhersehbarem oder fehlerhaftem Verhalten führen.

Beispielproblem: Seiteneffekte bei Statusregistern

Nicht jedes Statusregister verwendet dasselbe Löschverhalten. Beim STM32 arbeiten viele Timer-Statusbits (z. B. `TIMx_SR`) als **rc_w0** (durch Schreiben von `0` löschen), nicht als **read-to-clear**. Ein unkritisches Beispiel wäre:

```
uint32_t status = TIM2->SR;
TIM2->SR = (uint16_t)~TIM_SR_UIF;   // UIF gezielt löschen (rc_w0)
```

Ein echtes **read-to-clear**-Muster findet sich hingegen bei anderen Modulen, beispielsweise bei USART-Statusabläufen (z. B. `SR` lesen, danach `DR` lesen). Die genaue Semantik muss deshalb immer im jeweiligen Referenzmanual geprüft werden.

Bitbanding – die Besonderheit des Cortex-M3

Der ARM Cortex-M3 bietet ein Feature, das RMW-Probleme vollständig vermeiden kann: das

sogenannte **Bit-Banding**. Es ermöglicht, einzelne Bits des SRAM oder bestimmter Peripheriebereiche wie separate Speicheradressen anzusprechen.

Eine spezielle Adresse im Bitband-Aliasbereich entspricht genau **einem** Bit im realen Speicher. Ein Schreibzugriff auf diese Adresse setzt oder löscht das Bit **atomar** – ganz ohne RMW.

Beispiel: Ein Bit im SRAM lässt sich wie eine Variable behandeln:

```
// Beispiel: Bit 7 des Bytes an Adresse 0x20000000 (byte_offset = 0)
// Adresse = BITBAND_SRAM_BASE + (byte_offset * 32) + (bit_number * 4)
uint32_t* bit7 = (uint32_t*)(0x22000000 + (0 * 32) + (7U * 4U));
*bit7 = 1;  // setzt Bit 7 atomar
```

Viele CMSIS-Treiber abstrahieren diese Adressberechnung bereits.

Vorteile des Bitbandings

- einzelne Bits können **atomar** gesetzt oder gelöscht werden
- keinerlei RMW notwendig
- perfekt für Flags, Semaphoren und ISR-Interaktion
- besonders geeignet für:
  - GPIO-Toggles
  - Statusbits
  - Software-Locks
  - einfache Synchronisierung

Nachteile des Bitbandings

- nicht alle Peripherieregister werden unterstützt
- kostet zusätzliche Adressberechnung („Alias-Bereich“)
- auf Cortex-M4 ist Bit-Banding implementationsabhängig, auf Cortex-M7 in der Regel nicht vorhanden

Moderne Controller verwenden dafür häufig „Bit-Set/Clear-Register“.

Alternativen zum Bitbanding

Viele Mikrocontroller (STM32F103 eingeschlossen) bieten eigene Register zum Setzen und Löschen einzelner Bits:

```
// Bit 5 setzen:
GPIOA->BSRR = (1 << 5);

// Bit 2 löschen:
```

```
GPIOA->BRR = (1 << 2);
```

Vorteile:

- **atomare** Hardware-Operation
- kein Lesen des Registers notwendig
- keine RMW-Sequenz
- funktioniert für alle GPIOs

Ein Vorteil gegenüber Bit-Banding: **GPIO-BSRR/BRR sind speziell für schnellen Peripheriezugriff optimiert.**

Warum klassische RMW-Probleme gefährlich sind

- Interrupts können Flags verändern
- Lesezugriffe können Side-Effects auslösen
- Peripherieregister sind oft write-sensitive
- einige Register dürfen niemals „teilweise“ beschrieben werden
- manche Register besitzen Bits, die nur schreibbar sind, wenn andere Bits unverändert bleiben müssen

Best Practices für Registerzugriffe beim Cortex-M3

- **GPIOs immer über BSRR/BRR** schalten → niemals über ODR RMW
- wenn verfügbar: **SET/CLR-Register** nutzen (z. B. bei Timern, DMA, EXTI)
- **Bit-Banding** nur dort einsetzen, wo es vom Hersteller vorgesehen ist
- **niemals** komplette Register lesen und zurückschreiben, wenn Flags enthalten sind
- Dokumentation prüfen: „read-to-clear“, „write-1-to-clear“ oder „write-only“ beachten
- bei konkurrierendem Zugriff (ISR/Threads):
  - Atomics
  - interruptfreie Bereiche
  - Memory barriers (`__DMB()`, `__DSB()`)

Zusammenfassung

- Read-Modify-Write ist auf dem Cortex-M3 nicht atomar.
- Zwischen Read und Write können Side-Effects auftreten.
- Bit-Banding erlaubt atomare Bitoperationen ohne RMW.
- Viele Peripherieblöcke bieten atomare Set/Reset-Register.
- RMW sollte bei Peripherieregistern möglichst **vermieden** werden. :leveloffset: 0

Takterzeugung (HSI, HSE, PLL, RCC)

Ohne korrekte Taktkonfiguration arbeitet der Controller zwar, aber meist nicht mit der gewünschten Peripheriegeschwindigkeit.

Wichtige Taktquellen:

- **HSI**: interner RC-Oszillator
- **HSE**: externer Quarz/Oszillator
- **PLL**: Vervielfachung eines Eingangstakts

Der Taktbaum wird über das RCC-Modul konfiguriert. Typischer Ablauf:

1. HSE oder HSI aktivieren und auf Stabilität warten.
2. PLL konfigurieren und aktivieren.
3. Systemtaktquelle auf PLL umschalten.
4. Bus-Teiler (AHB/APB) korrekt setzen.
5. Peripherietakte gezielt freischalten.

```
/* Beispiel: GPIOA-Takt auf APB2 freischalten */
RCC->APB2ENR |= (1U << 2);   /* IOPAEN */
```

Ohne diesen Schritt bleiben Registerzugriffe oft wirkungslos, obwohl der Code syntaktisch korrekt ist.

GPIO-Grundlagen

GPIO-Pins unterstützen mehrere Betriebsarten, z. B.:

- Input Floating
- Input Pull-up / Pull-down
- Push-Pull Output
- Open-Drain Output
- Alternate Function (z. B. UART, SPI, Timer)

Beim STM32F103 erfolgt die Konfiguration über `CRL` (Pins 0..7) und `CRH` (Pins 8..15), jeweils mit Modus- und Konfigurationsbits pro Pin.

Entprellen als Einstieg

Mechanische Taster prellen. Eine robuste Software-Entprellung kann z. B. mit einem Timer-Tick (1 ms) erfolgen:

- Rohsignal einlesen
- Zustandsautomat aktualisieren

- erst nach stabilen Folgemessungen als „gültig“ übernehmen

Damit lässt sich das Thema GPIO direkt mit Zustandsautomaten und Timern verbinden.

Interrupts (NVIC, Prioritäten, Vektortabelle)

Interrupts erlauben eine ereignisgesteuerte Verarbeitung ohne permanentes Polling.

Kernbegriffe:

- **Vektortabelle**: Zuordnung von IRQ-Nummern zu ISR-Einsprungadressen
- **NVIC**: Verwaltung, Priorisierung und Freigabe der Interrupts
- **Prioritäten**: kleinere Zahl bedeutet höhere Priorität
- **Latenz**: Zeit zwischen Ereignis und ISR-Beginn

Typische Schritte:

1. Interruptquelle im Peripheriemodul konfigurieren.
2. Pending-Flags löschen.
3. NVIC-Interrupt freischalten.
4. ISR schlank halten.

```
NVIC_SetPriority(EXTI0_IRQn, 5);
NVIC_EnableIRQ(EXTI0_IRQn);
```

ISR-Regeln in der Praxis:

- keine langen Blockaden
- keine unnötigen Schleifen
- keine nicht-deterministischen Wartezeiten
- Datenübergabe an Main-Loop über Flags, Ringbuffer oder Queue

Timer und Counter

Timer sind zentrale Bausteine für Zeitmessung, Periodik und Signalerfassung.

Typische Funktionen:

- Zeitbasis (z. B. 1 ms Systemtick)
- Output Compare
- Input Capture
- Encoder- und PWM-Betrieb

Grundprinzip:

- Prescaler teilt den Eingangstakt.
- Counter zählt bis Auto-Reload (`ARR`).
- Bei Ereignissen werden Flags gesetzt oder Interrupts ausgelöst.

Für Input-Capture wird der aktuelle Zählerstand bei einer Flanke gespeichert. Aus Differenzen zweier Capture-Werte lässt sich z. B. die Periodendauer eines Signals berechnen.

PWM (Pulsweitenmodulation)

PWM erzeugt ein periodisches digitales Signal mit einstellbarem Tastgrad (Duty-Cycle). Viele Anwendungen (LED-Dimmung, Motorsteuerung, Servoansteuerung) basieren darauf.

Grundformeln:

- `f_PWM = f_TIM / ((PSC + 1) * (ARR + 1))`
- `Duty = CCR / (ARR + 1)`

Beispielidee:

- Timer auf 1 kHz konfigurieren
- `CCR` zwischen 0 und `ARR` variieren
- LED-Helligkeit ändert sich proportional zum Tastgrad

Watchdog (IWDG/WWDG)

Watchdogs erhöhen die Ausfallsicherheit. Bleibt die Software hängen, folgt ein kontrollierter Reset.

Im STM32F103 relevant:

- **IWDG** (Independent Watchdog): eigener Low-Speed-Takt, läuft unabhängig
- **WWDG** (Window Watchdog): Refresh nur in definiertem Zeitfenster

Typische Fehler:

- Watchdog zu spät oder zu früh füttern
- Timeout unrealistisch kurz wählen
- Refresh in einer ISR verstecken, obwohl der Main-Loop steht

Best Practice: Der Watchdog wird nur dann zurückgesetzt, wenn alle gesundheitsrelevanten Teilfunktionen ihren Zyklus erfolgreich abgeschlossen haben.

DMA (Direct Memory Access)

DMA überträgt Daten zwischen Speicher und Peripherie ohne aktive CPU-Kopierlast. Das reduziert Latenz und entlastet den Kern.

Bausteine:

- Kanal-Konfiguration (Quelle, Ziel, Länge)
- Transfermodus (normal oder zirkulär)
- Ereignisse/Interrupts (Transfer Complete, Half Transfer, Error)

Typischer Einsatz:

- ADC-Samples in Ringpuffer
- USART/SPI-Datenströme mit geringer CPU-Last
- periodische Peripherieaktualisierung

Beispiel: Speicher → USART-DR für TX-Burst, Abschluss per DMA-Interrupt signalisieren.

Die Hardware-Grundprinzipien zu FIFO-Puffern und Dual-Ported-RAM als Entkopplungsbausteine wurden bereits im Kapitel FIFO und Dual-Ported-RAM eingeführt.

Serielle Basis-Schnittstellen

USART als Einstieg

USART ist meist die erste aktiv genutzte Schnittstelle für Debug-Ausgaben und einfache Protokolle.

Grundschritte:

1. GPIO-Pins auf Alternate Function konfigurieren
2. Baudrate setzen
3. TX/RX aktivieren
4. optional Interrupt oder DMA für Empfang/Versand verwenden

Damit verbindet USART unmittelbar die Themen Registerzugriff, Interrupts und DMA.

SPI und I²C im Überblick

Neben USART sind in vielen Embedded-Projekten zwei weitere serielle Standardschnittstellen zentral:

- **SPI**: synchron, schnell, Master/Slave, getrennte Daten-/Taktleitung
- **I²C**: synchron, adressbasiert, zwei Leitungen (SCL/SDA), Multi-Device-Bus

SPI wird häufig für Displays, ADCs und Flash-Bausteine genutzt, I²C typischerweise für Sensorik, RTCs oder Konfigurations-EEPROMs.

Zusammenfassung

- Peripherie wird über Memory-Mapped I/O und Registerzugriffe gesteuert.
- RCC und Clock-Tree sind Grundlage für jede funktionierende Peripherie.
- GPIO, Timer, Interrupts und PWM bilden den Kern vieler Embedded-Anwendungen.
- Watchdog und DMA erhöhen Robustheit und Effizienz.

- USART, SPI und I²C sind die wichtigsten seriellen Basisschnittstellen.

I/O-Schnittstellen und Peripheriebusse

Im vorigen Kapitel haben wir die Peripherie des STM32F103 kennengelernt, die *auf* dem Chip sitzt: GPIO, Timer, Interrupts, DMA und die ersten Schnittstellenbausteine. Dieses Kapitel verlässt den Chip und betrachtet die Verbindungen *nach außen*: Wie kommen Bits über ein Kabel zu einem anderen Gerät, und welche Standards haben sich dafür etabliert?

Wir betrachten die klassischen seriellen Schnittstellen (RS-232, RS-422, RS-485), die parallele Schnittstelle (Centronics) und die Peripheriebusse USB und Firewire. Der STM32F103 dient wieder als roter Faden: Wo er eine Schnittstelle direkt unterstützt, zeigen wir die Konfiguration, wo nicht, zeigen wir, wie sie sich mit GPIO und Timern nachbilden lässt.

Grundbegriffe

Bevor wir einzelne Standards betrachten, klären wir die Begriffe, mit denen sich alle Schnittstellen beschreiben lassen.

Schichten einer Schnittstelle

Eine Schnittstelle besteht fast immer aus mehreren Schichten, die oft verwechselt werden:

Schicht	Aufgabe	Beispiele
Elektrische Ebene	Spannungspegel, Leitungsart, Steckverbinder	RS-232, RS-485, USB-Pegel
Übertragungsebene	Rahmenformat, Bitreihenfolge, Taktung	UART-Rahmen (Start-, Daten-, Stoppbit)
Protokollebene	Bedeutung der Daten, Adressierung, Fehlerbehandlung	Modbus RTU, NMEA, eigene Telegramme



RS-485 legt zum Beispiel *nur* die elektrische Ebene fest. Ob darüber ein UART-Rahmen und Modbus oder ein eigenes Protokoll läuft, ist eine völlig andere Frage.

Parallel und seriell

Bei der **parallelen** Übertragung liegt für jedes Bit eines Datenworts eine eigene Leitung bereit, alle Bits werden gleichzeitig übertragen. Bei der **seriellen** Übertragung werden die Bits nacheinander über eine einzelne Leitung (pro Richtung) geschickt.

Parallel wirkt auf den ersten Blick schneller, hat aber bei höheren Taktraten und längeren Kabeln Probleme:

- **Skew:** Die Signale auf den einzelnen Leitungen laufen unterschiedlich schnell und kommen zeitversetzt an.
- **Übersprechen:** Benachbarte Leitungen beeinflussen sich gegenseitig.
- **Aufwand:** Viele Adern, große Stecker, teure Kabel.

Serielle Schnittstellen kommen mit wenigen Leitungen aus und erreichen darum auch bei längeren Strecken hohe Taktraten. Das ist der Grund, warum fast alle modernen Schnittstellen seriell sind.

Synchron und asynchron

Bei der **synchronen** Übertragung bekommt der Empfänger den Takt mit:

- über eine eigene Taktleitung (SPI, I²C) oder
- eingebettet im Datenstrom (USB, Manchester-Codierung).

Bei der **asynchronen** Übertragung gibt es keinen gemeinsamen Takt. Beide Seiten haben vorab eine Übertragungsgeschwindigkeit vereinbart, und der Empfänger synchronisiert sich bei jedem Zeichen neu auf eine Startflanke. Das ist das Prinzip des UART.

Richtungsbetrieb

Betrieb	Bedeutung
Simplex	Nur eine Richtung (z. B. Sensor sendet, Empfänger hört nur zu)
Halbduplex	Beide Richtungen, aber abwechselnd auf denselben Leitungen (z. B. RS-485 mit zwei Adern)
Vollduplex	Beide Richtungen gleichzeitig auf getrennten Leitungen (z. B. RS-232, USART mit TX und RX)

Baudrate und Bitrate

Die **Baudrate** (Einheit Bd) gibt die Anzahl der Schritte (Symbole) pro Sekunde an, die **Bitrate** (Einheit bit/s) die Anzahl der Bits pro Sekunde.

Bei Schnittstellen mit zwei Signalzuständen (UART, RS-232, RS-485) trägt jeder Schritt genau ein Bit, hier sind beide Werte gleich. Bei mehrwertigen Symbolen gilt:

```
Bitrate = Baudrate × Bits pro Symbol
```

Die Bitrate ist außerdem nicht die **Nutzdatenrate**: Start-, Stopp- und Paritätsbits werden mitgezählt, tragen aber keine Nutzdaten. Bei 8N1 (siehe unten) sind nur 8 von 10 übertragenen Bits Nutzdaten, also 80 %.

Topologien

Topologie	Beschreibung
Punkt-zu-Punkt	Genau zwei Teilnehmer (RS-232)
Multidrop / Bus	Mehrere Teilnehmer teilen sich dieselben Leitungen (RS-485)
Stern / Baum	Zentrale Verteiler (Hubs) verbinden die Teilnehmer (USB)
Daisy-Chain	Teilnehmer sind der Reihe nach verkettet (Firewire)

Single-ended und differenziell

Bei der **single-ended** Übertragung wird eine Signalspannung gegen eine gemeinsame Masse gemessen. Jede Störung auf der Signalleitung und jeder Unterschied im Massepotential der beiden Geräte verfälscht direkt den Pegel.

Bei der **differenziellen** Übertragung läuft das Signal über zwei Leitungen, die gegenläufig angesteuert werden. Der Empfänger bewertet nur die *Differenz* der beiden Spannungen. Störungen, die beide Leitungen gleichermaßen treffen (Gleichtaktstörungen), heben sich dadurch heraus.

Fehlererkennung

Übertragungen können gestört werden. Schnittstellen und Protokolle setzen darum Prüfverfahren ein:

- **Paritätsbit:** Ein zusätzliches Bit ergänzt die Anzahl der Einsen im Zeichen auf eine gerade (even) oder ungerade (odd) Zahl. Es erkennt jeden Fehler mit einer ungeraden Anzahl falscher Bits, zwei falsche Bits fallen aber nicht auf.
- **Prüfsumme:** Einfache Summe über die Daten.
- **CRC (Cyclic Redundancy Check):** Wesentlich robuster, erkennt auch Mehrbitfehler und Bündelfehler. Wird z. B. bei Modbus RTU (CRC-16) und USB eingesetzt.

UART und RS-232

Die serielle Schnittstelle am PC, das „COM-Port“, war jahrzehntelang der Standardweg, um Geräte anzubinden. Dahinter stecken zwei Dinge, die wir sauber trennen müssen: der **UART** als Übertragungsverfahren und **RS-232** als elektrischer Standard.

Der UART-Rahmen

Ein UART (*Universal Asynchronous Receiver Transmitter*) überträgt Zeichen asynchron. Der STM32 nennt seinen Baustein *USART*, weil er zusätzlich einen synchronen Betrieb mit Taktleitung kann. Für die asynchrone Betriebsart sind nur zwei Signalleitungen nötig, `TX` und `RX`, plus die gemeinsame Masse.

Im Ruhezustand (Idle) liegt die Leitung auf logisch 1 (High). Ein Zeichen wird in einem **Rahmen** (Frame) übertragen:

1. **Startbit:** Die Leitung geht für eine Bitzeit auf 0. Diese fallende Flanke synchronisiert den Empfänger.
2. **Datenbits:** 5 bis 9 Bits, üblich sind 8. Das niederwertigste Bit (LSB) wird zuerst gesendet.
3. **Paritätsbit (optional):** Even, odd oder weggelassen.
4. **Stoppbit(s):** 1, 1,5 oder 2 Bitzeiten auf logisch 1. Das stellt den Ruhepegel wieder her und gibt dem Empfänger Zeit.

Die Konfiguration wird kurz in der Form `8N1` notiert: 8 Datenbits, keine (None) Parität, 1 Stoppbit. `8E1` bedeutet 8 Datenbits, gerade (Even) Parität, 1 Stoppbit.

Tabelle 1. Beispiel: Das Zeichen `'A'` (0x41 = 0100 0001) bei 8N1

Position	Idle	Start	D0	D1	D2	D3	D4	D5	D6	D7	Stopp
Pegel	1	0	1	0	0	0	0	0	1	0	1



Das LSB geht zuerst auf die Leitung, deshalb erscheint 0x41 hier als `1 0 0 0 0 0 1 0`, also "rückwärts" gegenüber der üblichen Schreibweise.

Baudrate und Zeitverhalten

Die Bitzeit ist der Kehrwert der Baudrate:

Baudrate	Bitzeit	Zeichenzeit (8N1, 10 Bit)	Zeichen pro Sekunde
9600 Bd	ca. 104 µs	ca. 1,04 ms	960
115200 Bd	ca. 8,68 µs	ca. 86,8 µs	11520

Diese Zahlen sind wichtig, wenn wir Software entwerfen:

Wenn die Software ein empfangenes Zeichen nicht schnell genug abholt, geht es verloren (siehe Fehlerflags weiter unten). Genau dafür gibt es Interrupts, Ringpuffer und DMA.

Toleranz der Baudrate

Es gibt keinen gemeinsamen Takt, also laufen Sender und Empfänger mit leicht unterschiedlichen Taktfrequenzen. Der Empfänger tastet jedes Bit in der Mitte der Bitzeit ab, gemessen ab der Flanke des Startbits. Das letzte Bit eines 8N1-Rahmens, das Stoppbit, wird nach 9,5 Bitzeiten abgetastet. Der Abtastpunkt darf bis zu einer halben Bitzeit verrutschen, das ergibt:

```
0,5 / 9,5 ≈ 5,3 %
```

Das ist die theoretische Grenze für die *gesamte* Abweichung beider Seiten zusammen. In der Praxis bleibt man deutlich darunter (Richtwert: wenige Prozent), weil Flankensteilheit und Rauschen zusätzlich Reserve kosten.



Ein interner RC-Oszillator (z. B. HSI beim STM32) ist deutlich ungenauer als ein Quarz. Bei hohen Baudraten oder über den Temperaturbereich kann das zu Übertragungsfehlern führen. Das erklärt manches mysteriöse "Zeichensalat"-Problem.

Die Baudrate beim STM32F103

Der USART des STM32F103 erzeugt die Baudrate aus dem Peripherietakt `f_PCLK` mit einem Teiler. Das Register `BRR` enthält den Teilerwert `USARTDIV` im Festkommaformat (12 Bit Mantisse, 4 Bit Nachkommaanteil in Sechzehnteln):

```
USARTDIV = f_PCLK / (16 x Baudrate)
```

```
BRR      = (Mantisse << 4) | Nachkommaanteil
```

Beispiel: USART1 hängt am APB2 und läuft mit 72 MHz. Gewünscht sind 115200 Bd:

```
USARTDIV = 72 000 000 / (16 x 115 200) = 39,0625
Mantisse = 39
Nachkomma = 0,0625 x 16 = 1
BRR = (39 << 4) | 1 = 0x0271
```

Hier geht die Rechnung exakt auf, der Baudratenfehler ist 0 %. Bei anderen Takten oder Baudraten bleibt ein Rest. Für USART2 und USART3 am APB1 (max. 36 MHz) ergibt sich z. B. `USARTDIV = 19,53125`; der nächste darstellbare Wert weicht nur um etwa 0,16 % von 115200 Bd ab, das ist unkritisch.

Konfiguration und Zeichenausgabe

Das folgende Beispiel konfiguriert USART1 mit 115200 Bd, 8N1, `TX` auf PA9 und `RX` auf PA10:

```
/* Takt für GPIOA (IOPAEN, Bit 2) und USART1 (USART1EN, Bit 14) */
RCC->APB2ENR |= (1U << 2) | (1U << 14);

/* PA9  (TX): Alternate Function Push-Pull, 50 MHz -> MODE=11, CNF=10 -> 0xB */
/* PA10 (RX): Input Floating                       -> MODE=00, CNF=01 -> 0x4 */
GPIOA->CRH &= ~((0xFU << 4) | (0xFU << 8));
GPIOA->CRH |=  ((0xBU << 4) | (0x4U << 8));

/* Baudrate: 72 MHz / (16 * 39,0625) = 115200 Bd */
USART1->BRR = (39U << 4) | 1U;

/* UE (Bit 13), TE (Bit 3), RE (Bit 2) */
USART1->CR1 = (1U << 13) | (1U << 3) | (1U << 2);
```

Ein Zeichen senden und empfangen (Polling):

```
void uart_putc(char c)
{
    while ((USART1->SR & (1U << 7)) == 0) { }   /* TXE: Datenregister leer */
    USART1->DR = (uint8_t)c;
}

int uart_getc(void)
{
    while ((USART1->SR & (1U << 5)) == 0) { }   /* RXNE: Zeichen empfangen */
    return (int)(USART1->DR & 0xFFU);           /* Lesen von DR löscht RXNE */
}
```



Beim Lesen von DR wird das Flag RXNE automatisch gelöscht. Das ist ein echtes Read-to-Clear-Verhalten, wie wir es im Abschnitt zu Read-Modify-Write kennengelernt haben.

Fehlerflags

Das Statusregister SR meldet Übertragungsfehler:

Flag	Bedeutung
PE (Parity Error)	Die Parität des empfangenen Zeichens stimmt nicht.
FE (Framing Error)	Das Stoppbit wurde nicht als 1 erkannt. Typische Ursachen: falsche Baudrate, falsche Rahmenkonfiguration, abgezogenes Kabel, Break-Zustand.
NE (Noise Error)	Rauschen während der Abtastung erkannt.
ORE (Overrun Error)	Ein neues Zeichen kam an, bevor das alte aus DR abgeholt wurde. Das alte oder das neue Zeichen ist verloren.



Ein häufiger Fehler beim Einstieg ist FE durch eine falsch berechnete Baudrate, etwa weil der Systemtakt nach der Initialisierung der PLL doch nicht 72 MHz beträgt. Erst Takt prüfen, dann den UART.

Zur Vermeidung von ORE bei höheren Baudraten gibt es drei Strategien, die zu den Kapiteln über Interrupts und DMA zurückführen:

1. **Interrupt:** ISR holt jedes Zeichen sofort ab und legt es in einen Ringpuffer.
2. **DMA:** Der DMA-Controller überträgt empfangene Zeichen selbstständig in einen Speicherbereich, die CPU wird nur bei einem Halb- oder Voll-Puffer informiert.
3. **Höhere Priorität:** Der UART-Interrupt bekommt eine hohe Priorität, damit er nicht von längeren ISRs blockiert wird.

RS-232: Der elektrische Standard

Ein UART am Mikrocontroller liefert Pegel von 0 V und 3,3 V (*TTL/CMOS-Pegel*). Der klassische Standard **RS-232** (heute korrekt TIA/EIA-232) definiert andere Pegel, die für längere Kabel und Störfestigkeit ausgelegt wurden:

	UART mit TTL-Pegel (3,3 V)	RS-232
Logisch 1 (Mark, Idle)	ca. 3,3 V	−3 V … −15 V (Sender: typisch −5 V … −15 V)
Logisch 0 (Space)	ca. 0 V	+3 V … +15 V (Sender: typisch +5 V … +15 V)
Undefinierter Bereich	zwischen den Schwellen	−3 V … +3 V

Zwei Dinge fallen auf:

1. Die Logik ist **invertiert**: Eine 1 ist eine *negative* Spannung, eine 0 eine *positive*.
2. Die Pegel liegen weit außerhalb des Bereichs, den ein 3,3-V-Mikrocontroller verträgt.



Ein RS-232-Signal darf niemals direkt an einen STM32-Pin angeschlossen werden. Die Spannungen können den Eingang zerstören. Dazwischen gehört ein **Pegelwandler** (z. B. ein MAX232 oder MAX3232). Dieser erzeugt aus der Versorgungsspannung mit Ladungspumpen die benötigten positiven und negativen Spannungen.

Weitere Eigenschaften von RS-232:

- **Single-ended:** Alle Signale beziehen sich auf eine gemeinsame Masse (GND).
- **Punkt-zu-Punkt:** Genau zwei Teilnehmer, Vollduplex.
- **Reichweite:** Die Spezifikation nennt etwa 15 m bei bis zu 20 kbit/s. In der Praxis sind bei niedrigeren Raten auch längere Strecken möglich, die Störanfälligkeit steigt aber.



Viele Geräte sprechen von einer "seriellen Schnittstelle", haben aber nur einen UART mit TTL-Pegel. Diesen kann man **nicht** direkt mit einem RS-232-Anschluss verbinden. Prüfe immer, welche Pegel gemeint sind.

Steckverbinder und Leitungen

Am PC wird ein 9-poliger Sub-D-Stecker verwendet. Der PC ist ein **DTE** (Data Terminal Equipment), ein Modem oder Gerät ein **DCE** (Data Communication Equipment).

Pin	Signal	Bedeutung (aus Sicht des DTE)
1	DCD	Data Carrier Detect, Trägersignal erkannt
2	RxD	Empfangsdaten
3	TxD	Sendedaten
4	DTR	Data Terminal Ready, DTE ist bereit
5	GND	Signalmasse
6	DSR	Data Set Ready, DCE ist bereit
7	RTS	Request To Send, DTE will senden
8	CTS	Clear To Send, DCE ist sendebereit
9	RI	Ring Indicator, Klingelsignal

Wenn zwei DTEs direkt miteinander verbunden werden (z. B. zwei PCs), müssen `TxD` und `RxD` über Kreuz verdrahtet werden. Ein solches Kabel heißt **Nullmodem-Kabel**. Meist werden dabei auch `RTS` mit `CTS` und `DTR` mit `DSR` über Kreuz verbunden, `GND` wird 1:1 durchgeschleift.

Flusskontrolle (Handshake)

Wenn der Empfänger Daten langsamer verarbeitet als der Sender sie liefert, braucht es eine Flusskontrolle:

- **Hardware-Handshake (RTS/CTS):** Zusätzliche Leitungen signalisieren, ob der Empfänger bereit ist. Zuverlässig, aber zwei Leitungen mehr.
- **Software-Handshake (XON/XOFF):** Der Empfänger sendet die Steuerzeichen `XOFF` (0x13) zum Anhalten und `XON` (0x11) zum Fortsetzen im normalen Datenstrom. Nicht geeignet für Binärdaten, da diese Werte auch in den Nutzdaten vorkommen können.

Die USART-Bausteine des STM32F103 unterstützen den Hardware-Handshake über `CTS`/`RTS`.

USB-UART-Wandler

Moderne PCs haben keinen COM-Port mehr. Stattdessen werden USB-UART-Wandler (z. B. FT232, CP210x, CH340) verwendet. Diese Bausteine erzeugen auf der USB-Seite einen virtuellen COM-Port und liefern auf der anderen Seite einen UART mit **TTL-Pegel**. Das ist die übliche Debug-Verbindung zu Mikrocontroller-Boards.

RS-422 und RS-485

RS-232 stößt bei Reichweite, Störfestigkeit und Teilnehmerzahl an Grenzen. Industrielle Anwendungen brauchen oft Kabel über mehrere hundert Meter, durch störungsreiche Umgebungen und mit vielen Teilnehmern. Dafür wurden **RS-422** (TIA/EIA-422) und **RS-485** (TIA/EIA-485) entwickelt.



Beide Standards legen nur die **elektrische Ebene** fest, also Pegel und Leitungstechnik. Das Rahmenformat liefert wieder der UART, das Protokoll darüber wird getrennt festgelegt.

Differenzielle Übertragung

Beide Standards arbeiten differenziell über ein **verdrilltes Aderpaar** (Twisted Pair). Die Leitungen heißen üblicherweise `A` und `B`. Der Sender erzeugt zwischen `A` und `B` eine Differenzspannung, der Empfänger wertet nur diese Differenz aus:

	RS-422	**RS-485**
Sender-Ausgangsspannung (differenziell)	mindestens ±2 V	mindestens ±1,5 V
Empfängerschwelle	±200 mV	±200 mV
Gleichtaktbereich	−7 V … +7 V	−7 V … +12 V

Zwei Eigenschaften machen die Schnittstellen robust:

- **Gleichtaktunterdrückung:** Eine Störung, die beide Adern gleich trifft, ändert die Differenz nicht. Durch die Verdrillung koppeln Störungen auf beide Adern nahezu gleich ein.
- **Massepotentialunterschiede:** Die Geräte dürfen sich im Massepotential um mehrere Volt unterscheiden (innerhalb des Gleichtaktbereichs), ohne dass der Empfang gestört wird.



Die Bezeichnungen `A` und `B` sind bei Herstellern nicht einheitlich (manche verwenden `D+` und `D`□ oder tauschen die Zuordnung). Bei einer Kommunikation, die

> nicht funktioniert, sind vertauschte Adern eine häufige Fehlerursache. Im Ruhezustand (Idle, logisch 1) ist laut Standard `B` positiv gegenüber `A`.

Reichweite und Datenrate

Mit steigender Leitungslänge sinkt die erreichbare Datenrate. Als Faustwerte gelten:

- bis etwa **1200 m** bei Datenraten bis ca. 100 kbit/s
- bis etwa **10 Mbit/s** bei Leitungslängen von wenigen Metern (ca. 12 m)

Diese Werte sind Größenordnungen aus den Datenblättern, die tatsächlich erreichbare Strecke hängt von Kabel, Umgebung und Transceiver ab.

RS-422

RS-422 ist für Punkt-zu-Mehrpunkt-Verbindungen ausgelegt:

- **Ein** Sender, bis zu **zehn** Empfänger an einem Aderpaar
- **Vollduplex** durch zwei getrennte Aderpaare (4-Draht)
- Einsatz z. B. für schnelle Punkt-zu-Punkt-Strecken, Encoder-Signale, Datenverteilung

RS-485

RS-485 erweitert RS-422 um einen echten **Bus** mit mehreren Sendern:

- Mehrere Teilnehmer teilen sich **dasselbe Aderpaar** (Multipoint)
- Meist **Halbduplex** über zwei Adern (2-Draht), alternativ Vollduplex über vier Adern
- Die Sender haben einen **Tri-State-Ausgang** (hochohmig, wenn sie nicht senden). Es darf immer nur **ein** Teilnehmer gleichzeitig senden.
- Standardmäßig bis zu **32 Teilnehmer** (Unit Loads), mit Transceivern geringer Last (z. B. 1/8 Unit Load) bis zu 256

	RS-232	**RS-422**	**RS-485**
Signalart	single-ended	differenziell	differenziell
Teilnehmer	2 (Punkt-zu-Punkt)	1 Sender, bis zu 10 Empfänger	bis zu 32 (256 mit 1/8-UL-Bausteinen)
Betrieb	Vollduplex	Vollduplex (4-Draht)	Halbduplex (2-Draht) oder Vollduplex (4-Draht)
Max. Länge (Richtwert)	ca. 15 m	ca. 1200 m	ca. 1200 m
Max. Datenrate (Richtwert)	ca. 20 kbit/s (Spezifikation)	bis 10 Mbit/s	bis 10 Mbit/s
Typischer Einsatz	Debug, Modem, Messgeräte	Encoder, schnelle Verbindungen	Feldbusse, Gebäudetechnik, Modbus

Busaufbau bei RS-485

Auf einem RS-485-Bus treten Leitungseffekte auf, die bei kurzen RS-232-Strecken keine Rolle spielen.

Linienstruktur: Der Bus wird als Linie aufgebaut, von Teilnehmer zu Teilnehmer. Stichleitungen (Stubs) zu den Geräten müssen möglichst kurz sein, eine Sternverdrahtung ist ungeeignet, weil sie Reflexionen erzeugt.

Abschlusswiderstände: Am Ende einer langen oder schnellen Leitung treten Reflexionen auf, wenn die Leitung nicht mit ihrem Wellenwiderstand abgeschlossen ist. Übliche Kabel haben einen Wellenwiderstand von ca. 120 Ω. Deshalb kommt an **beide Enden** des Busses je ein Widerstand von 120 Ω, nicht in die Mitte. Bei kurzen Leitungen und niedrigen Datenraten kann man oft darauf verzichten.

Fail-Safe-Biasing: Wenn kein Teilnehmer sendet, sind alle Treiber hochohmig und die Differenzspannung auf der Leitung ist undefiniert. Der Empfänger könnte durch Rauschen zufällige Zeichen erkennen. Mit einem Pull-up-Widerstand an `B` und einem Pull-down-Widerstand an `A` wird ein definierter Ruhepegel erzwungen. Dies wird typischerweise an einer Stelle des Busses (oft am Master) eingebaut, die Widerstandswerte richten sich nach den Vorgaben des Transceiver-Herstellers. Viele moderne Transceiver haben bereits einen eingebauten Fail-Safe-Empfänger.

Anbindung an den STM32

Ein UART liefert TTL-Pegel. Zwischen UART und Bus sitzt ein **RS-485-Transceiver** (z. B. MAX485 oder Bausteine der SN65HVD-Familie). Er hat vier Signale zum Mikrocontroller:

Signal	Funktion
`DI` (Driver Input)	Eingang des Senders, wird mit `TX` des UART verbunden.
`RO` (Receiver Output)	Ausgang des Empfängers, wird mit `RX` des UART verbunden.
`DE` (Driver Enable)	Aktiviert den Sender, wenn High.
`/RE` (Receiver Enable)	Aktiviert den Empfänger, wenn Low.

Bei Halbduplex-Betrieb werden `DE` und `/RE` meist zusammengeschaltet und von **einem** GPIO-Pin gesteuert. High bedeutet Senden, Low bedeutet Empfangen.



Neuere STM32-Serien haben im USART einen Hardware-Support für die `DE` -Steuerung. Beim F103 erfolgt die Umschaltung typischerweise per GPIO in Software.

Bus-Turnaround

Die Umschaltung zwischen Senden und Empfangen ist der kritischste Punkt der Software. Wird der Sender zu früh abgeschaltet, geht das letzte Zeichen verloren. Wird er zu spät abgeschaltet, blockiert er den Bus für den Antwortenden.

Das Statusflag `TXE` zeigt nur an, dass das *Datenregister* leer ist, das letzte Zeichen steckt dann aber noch im Schieberegister und wird noch ausgesendet. Erst das Flag `TC` (Transmission Complete, Bit 6

in SR) zeigt an, dass auch das Schieberegister geleert wurde und das letzte Stoppbit auf der Leitung war.

```
/* PA8 ist als Push-Pull-Ausgang konfiguriert und steuert DE und /RE */
void rs485_send(const uint8_t *buf, size_t len)
{
    GPIOA->BSRR = (1U << 8);                /* Senden aktivieren (PA8 = High) */

    for (size_t i = 0; i < len; i++)
    {
        uart_putc((char)buf[i]);
    }

    while ((USART1->SR & (1U << 6)) == 0) { } /* TC: letztes Bit ist draußen */

    GPIOA->BSRR = (1U << (8 + 16));         /* zurück auf Empfangen (PA8 = Low) */
}
```

Das Beispiel wartet blockierend und dient nur zur Illustration. In einer echten Anwendung würde man den Sendevorgang über DMA oder Interrupt abwickeln und DE im *Transmission-Complete-Interrupt* zurücknehmen.

Kollisionen vermeiden: Master/Slave

Weil nur ein Teilnehmer gleichzeitig senden darf, braucht ein RS-485-Bus eine Regel, wer wann sendet. Das einfachste und häufigste Verfahren ist **Master/Slave**:

- Ein **Master** sendet Anfragen.
- Jeder **Slave** hat eine Adresse und antwortet **nur**, wenn er angesprochen wurde.

So kommt es konstruktionsbedingt zu keinen Kollisionen. Multi-Master-Systeme brauchen dagegen eine Buszugriffsregelung (Arbitrierung), die RS-485 nicht mitbringt. Bussysteme wie CAN lösen das bereits in der Hardware.

Beispielprotokoll: Modbus RTU

Das verbreitetste Protokoll auf RS-485 ist **Modbus RTU**. Ein Telegramm besteht aus:

Adresse	Funktionscode	Daten	Prüfsumme
1 Byte	1 Byte	n Bytes	CRC-16 (2 Byte, niederwertiges Byte zuerst)

Die Slave-Adressen liegen im Bereich 1 bis 247, Adresse 0 ist Broadcast.

Beispiel: Der Master liest zwei Register ab Adresse 0 vom Slave 1 (Funktionscode 0x03, *Read Holding Registers*):

```
01 03 00 00 00 02 C4 0B
|  |  |---|  |---|  |---|
|  |   |     |       +-- CRC-16 (low, high)
|  |   |     +---------- Anzahl der Register (2)
|  |   +---------------- Startadresse (0)
|  +-------------------- Funktionscode (Read Holding Registers)
+----------------------- Slave-Adresse (1)
```

Das Ende eines Telegramms erkennt der Empfänger **nicht** an einem Endezeichen, sondern an einer **Pause**:

- Zwischen zwei Zeichen innerhalb eines Telegramms darf höchstens eine Pause von 1,5 Zeichenzeiten liegen.
- Eine Pause von mindestens 3,5 Zeichenzeiten markiert das Telegrammende.

Ein Zeichen besteht bei Modbus RTU meist aus 11 Bit (1 Start, 8 Daten, 1 Parität, 1 Stopp). Bei 9600 Bd sind das etwa 1,7 ms (1,5 Zeichen) beziehungsweise etwa 4,0 ms (3,5 Zeichen). Oberhalb von 19200 Bd werden feste Zeiten von 750 µs und 1750 µs empfohlen.



Hier schließt sich der Kreis zum Timer-Abschnitt. Die Erkennung des Telegrammendes ist eine klassische Aufgabe für einen Timer, der bei jedem empfangenen Zeichen neu gestartet wird und nach 3,5 Zeichenzeiten ein Ereignis auslöst.

Die CRC-16-Berechnung lässt sich bitweise einfach umsetzen:

```
uint16_t crc16_modbus(const uint8_t *data, size_t len)
{
    uint16_t crc = 0xFFFFU;

    for (size_t i = 0; i < len; i++)
    {
        crc ^= data[i];
        for (int bit = 0; bit < 8; bit++)
        {
            if (crc & 0x0001U)
            {
                crc = (uint16_t)((crc >> 1) ^ 0xA001U);
            }
            else
            {
                crc = (uint16_t)(crc >> 1);
            }
        }
    }
    return crc;
}
```

Für die Berechnung des Beispieltelegramms `01 03 00 00 00 02` liefert die Funktion `0x0BC4`. Gesendet wird das niederwertige Byte zuerst, also `C4 0B`.

Die Schleife über die Bits enthält eine Verzweigung. Eine tabellengestützte Variante, wie sie im Abschnitt zu Lookup-Tabellen als Ersatz für Bedingungslogik beschrieben ist, ist schneller und vermeidet Sprünge.

Zusammenfassung

- Eine Schnittstelle besteht aus elektrischer Ebene, Übertragungsebene und Protokoll. RS-232 und RS-485 legen nur die elektrische Ebene fest.
- Der UART überträgt asynchron in Rahmen aus Start-, Daten-, optionalem Paritäts- und Stoppbit. Die zulässige Baudratenabweichung liegt theoretisch bei etwa 5 %, praktisch deutlich darunter.
- RS-232 arbeitet single-ended mit invertierter Logik und Pegeln von ±3 V bis ±15 V. Ein Pegelwandler zum Mikrocontroller ist Pflicht.
- RS-422 und RS-485 übertragen differenziell über verdrillte Leitungen und erreichen Strecken bis etwa 1200 m. RS-485 ist ein Multipoint-Bus mit Halbduplex.
- Bei RS-485 sind Abschlusswiderstände, Fail-Safe-Biasing und eine korrekte Umschaltung zwischen Senden und Empfangen (`TC` statt `TXE`) entscheidend.
- Modbus RTU zeigt, wie auf der elektrischen Ebene ein Master/Slave-Protokoll mit CRC und zeitbasierter Telegrammerkennung aufgebaut wird.

Verständnisfragen

1. Warum wird bei einem UART das LSB zuerst gesendet? Welche Bitfolge geht beim Zeichen `0x53` bei 8N1 auf die Leitung?
2. Berechne für USART1 bei 72 MHz Peripherietakt den Wert für `BRR` bei 9600 Bd. Wie groß ist der Baudratenfehler?
3. Warum darf ein RS-232-Signal nicht direkt an einen STM32-Eingang angeschlossen werden? Was leistet ein MAX3232?
4. Warum ist ein RS-485-Bus ohne Fail-Safe-Biasing im Leerlauf störanfällig?
5. Warum muss bei der `DE`-Umschaltung auf `TC` und nicht auf `TXE` gewartet werden?
6. Ein Modbus-RTU-Telegramm wird bei 19200 Bd mit 11 Bit pro Zeichen übertragen. Wie lang ist die Pause für 3,5 Zeichenzeiten?

Datenstrukturen

Datenstrukturen bestimmen, wie Informationen im Speicher abgelegt und verarbeitet werden. Sie sind entscheidend für die Leistungsfähigkeit eines Programms.

Dieses Kapitel behandelt zwei fundamentale dynamische Datenstrukturen:

- **einfach verkettete Listen** – lineare Strukturen zum flexiblen Einfügen und Löschen
- **Binärbäume** – hierarchische Strukturen mit effizienten Suchoperationen

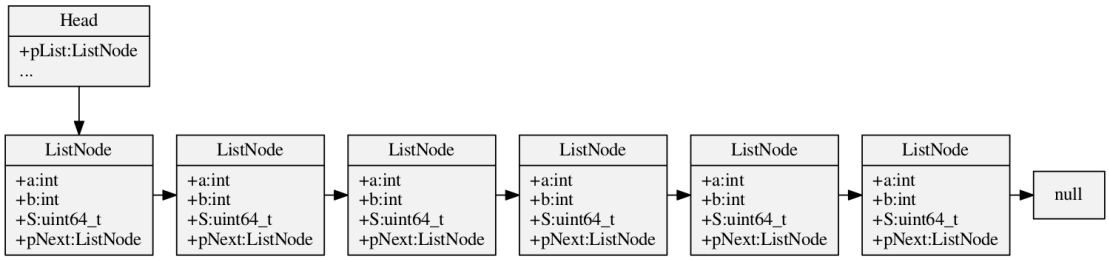
Diese Beispiele dienen als praktische Einführung in Pointer, dynamischen Speicher und rekursive Strukturierung.

Einfach verkettete Listen

Die einfach verkettete Liste ist eine Datenstruktur, die es dem Anwender ermöglicht, nahezu beliebige Daten anzufügen. Sie ist lediglich durch die zur Verfügung stehende Menge an Arbeitsspeicher begrenzt. Das Grundelement stellt eine Struktur dar. Diese Struktur wird auch als Knoten oder auf Englisch als **Node** bezeichnet.

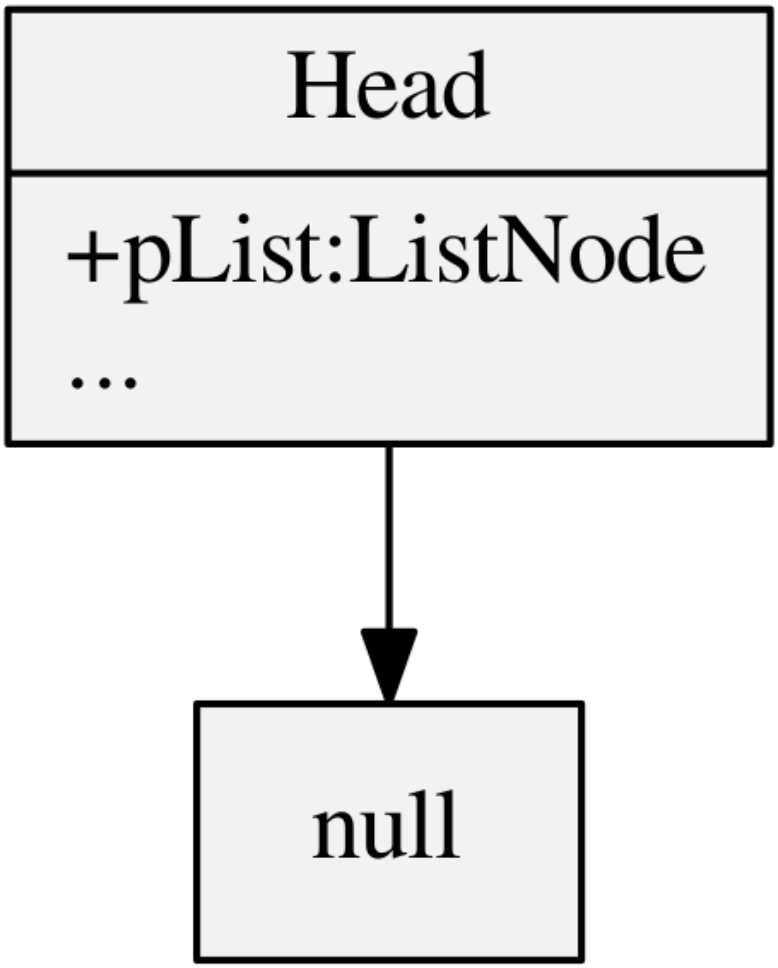
```
typedef struct ListNode {
    int operand_a;
    int operand_b;
    struct ListNode* pNext;
} ListNode_t;
```

Das Attribut `struct ListNode* pNext;` ermöglicht es, beliebige Instanzen der Struktur `ListNode_t` miteinander zu verknüpfen. In der Regel verwendet man einen sogenannten **Head**-Zeiger, der den Anfang einer Liste beinhaltet. Das Ende der Liste wird dadurch gekennzeichnet, dass der Zeiger `pNext` des letzten Elements auf `NULL` zeigt. Dies ist in der Abbildung dargestellt.



Die leere Liste

Der Sonderfall „leere Liste“ wird dadurch dargestellt, dass der *Head*-Zeiger direkt auf `NULL` zeigt. Dies ist in der Abbildung dargestellt.

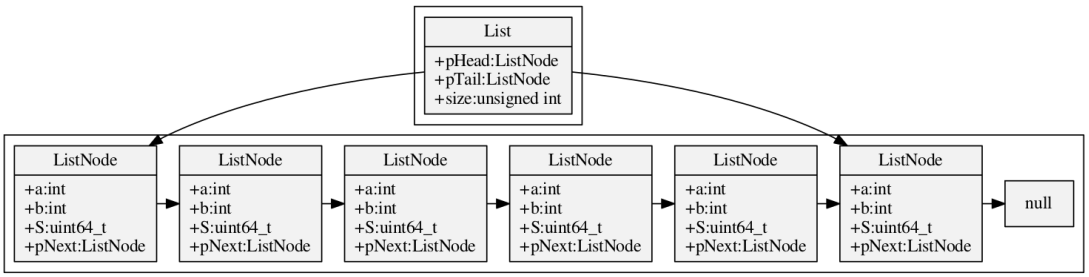


Das Ende der Liste

Zur schnelleren Navigation kann nicht nur der Anfang der Liste im *Head*-Zeiger abgespeichert werden; man kann zusätzlich noch das Ende, den sogenannten *Tail*, abspeichern. Dieser ermöglicht es, Elemente schnell am Ende anzufügen. Die Abbildung verdeutlicht dies.

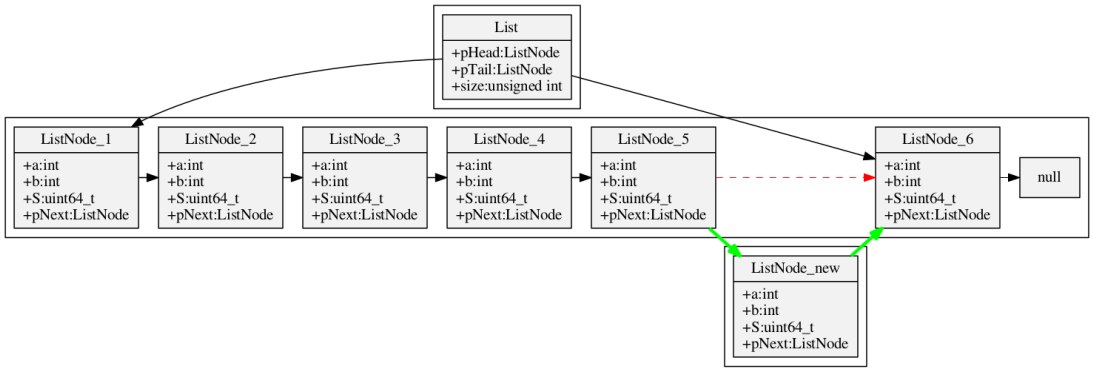
```
typedef struct List {
    ListNode_t* pHead;
    ListNode_t* pTail;
    unsigned int size;
```

```
} List_t;
```



Ein Element einfügen

Die Liste hat gegenüber einem Array den großen Vorteil, dass Elemente an beliebiger Stelle eingefügt werden können. Die Funktion `int insert_after(List* list, ListNode_t* loc, ListNode* elem)` fügt ein neues Element nach einem bereits vorhandenen Element ein. Hierbei müssen die folgenden Dinge überprüft werden:



- Ist die Liste leer?
- Wird am Ende eingefügt? (siehe Abbildung)
- Wird ein Element innerhalb der Liste eingefügt? (siehe Abbildung)

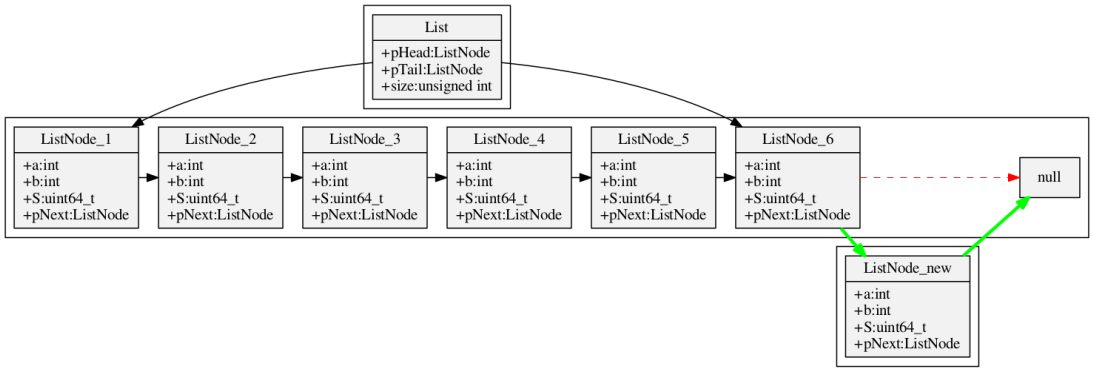
```
int insert_after(List* list, ListNode_t* loc, ListNode* elem)
{

    if((NULL != list) && (loc != NULL) && (elem !=NULL))
    {
```

```
        if(list->pTail == NULL)
        {
            /* empty list */
            list->pHead = elem;
            list->pTail = elem;
            elem->pNext = NULL;
        }
        else if  (list->pTail == elem)
        {
            /* append at the end */
            loc->pNext = elem;
            list->pTail = elem;
            elem->pNext = NULL;

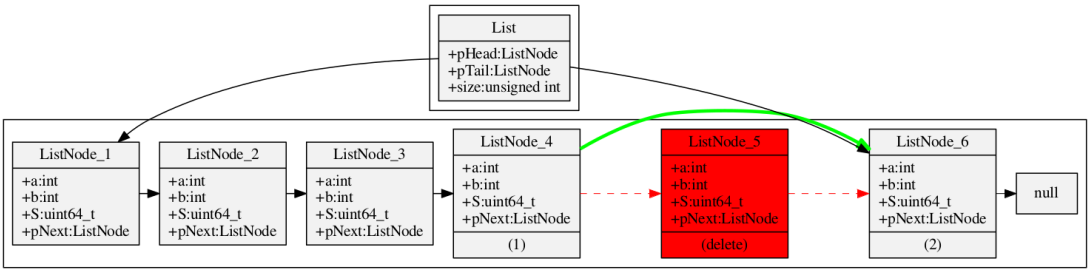
        }
        else
        {
            /* in the list */
            elem->pNext = loc->pNext;
            loc->pNext = elem;
        }
        list->size = list->size + 1;
    }

}
```



Ein Element aus der Liste löschen

In der Abbildung ist das Löschen eines Knotens aus der Liste dargestellt. Das Element `(delete)` soll aus der Liste gelöscht werden. Zunächst muss der Zeiger auf das nachfolgende Element `(2)` des zu löschenden Knotens gesichert werden. Anschließend kann das Element `(delete)` gelöscht werden und der Zeiger des Vorgängerknotens `(1)` auf das nachfolgende Element `(2)` gesetzt werden.



Zusammenfassung

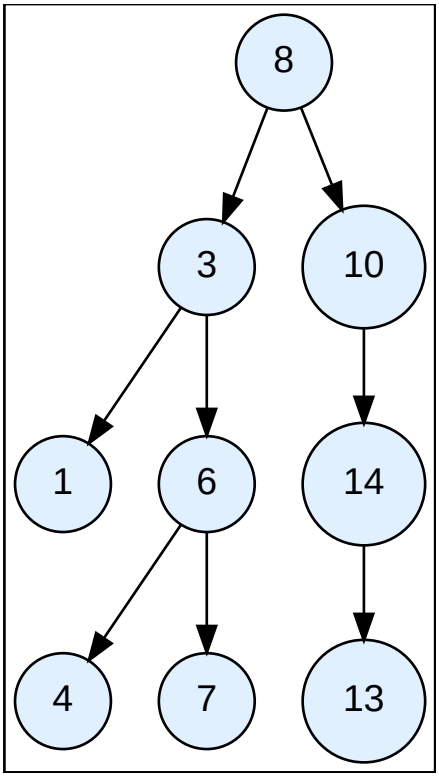
Die einfach verkettete Liste stellt eine Möglichkeit dar, mit einer dynamischen Anzahl an Elementen umzugehen. Es können leicht neue Elemente eingefügt werden und Elemente herausgelöscht werden. Dies ist bei einem Array nicht so einfach möglich. Soll auf ein Element in der Liste zugegriffen werden, muss die Liste zuerst nach dem passenden *Schlüssel* durchsucht werden. Bei einem Array kann direkt über den Index zugegriffen werden.

Binärbäume

Binärbäume sind hierarchische Datenstrukturen, deren Knoten maximal zwei Kinder besitzen:

```
typedef struct TreeNode {
    int value;
    struct TreeNode* left;
    struct TreeNode* right;
} TreeNode_t;
```

Einfacher Binärbaum



Binärer Suchbaum (BST)

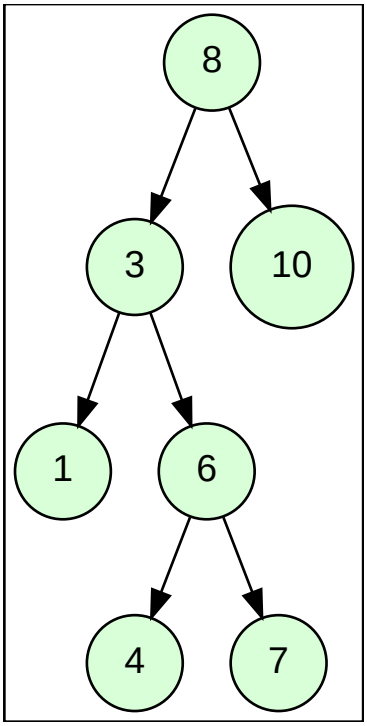
Ein binärer Suchbaum erfüllt die Ordnungseigenschaft:

```
left->value < parent->value <= right->value
```

```
TreeNode_t* insert(TreeNode_t* root, int value)
{
    if (root == NULL) {
        TreeNode_t* n = malloc(sizeof(TreeNode_t));
        n->value = value;
        n->left = n->right = NULL;
        return n;
    }

    if (value < root->value)
        root->left = insert(root->left, value);
    else
        root->right = insert(root->right, value);

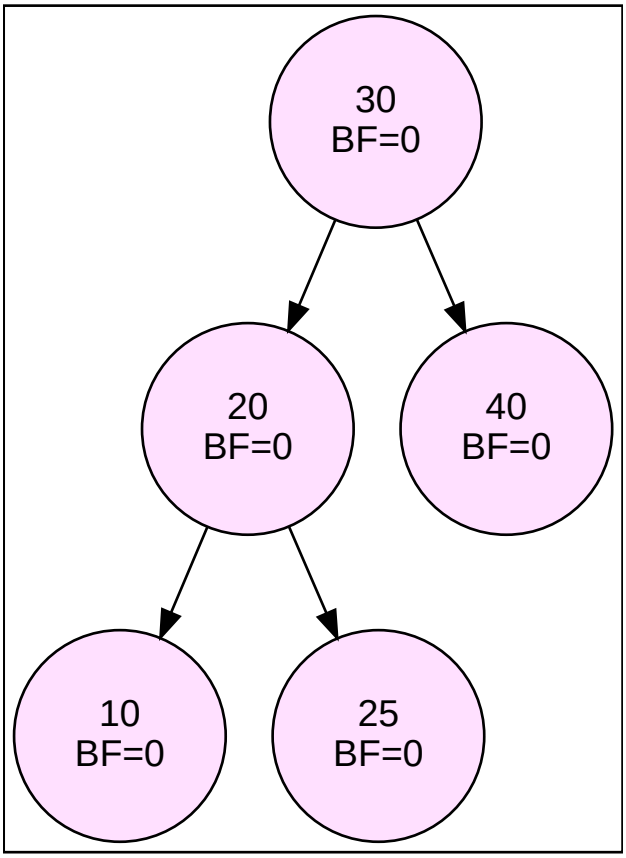
    return root;
}
```



AVL-Bäume

AVL-Bäume sind selbstbalancierende Binärbäume. Sie garantieren O(log n) für Suche, Einfügen und Löschen.

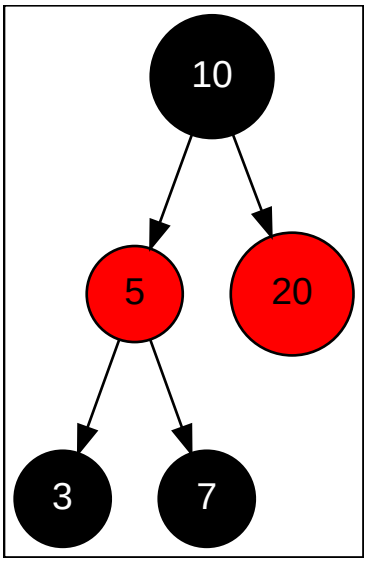
Wikipedia: https://de.wikipedia.org/wiki/AVL-Baum



Rot-Schwarz-Bäume

Rot-Schwarz-Bäume sind ebenfalls selbstbalancierend. Sie werden häufig in Standardbibliotheken verwendet (z. B. C++ `std::map`).

Wikipedia: https://de.wikipedia.org/wiki/Rot-Schwarz-Baum



Vergleich Liste vs. Binärbaum

Eigenschaft	Liste	Binärbaum
Zugriff	sequentiell	logarithmisch
Einfügen/Löschen	O(1) (wenn Position bekannt)	O(log n)
Struktur	linear	hierarchisch
Worst-Case	robust	kann degenerieren

Das BMP-Dateiformat

Bei `BMP` handelt es sich um ein sogenanntes Bitmap-Dateiformat. Der Vorteil von BMP liegt in der Verfügbarkeit von Programmen zur Darstellung von `BMP`-Dateien. So können sowohl der Browser Chrome als auch Mozillas Firefox mit BMP-Dateien umgehen.

Der Bitmap-Dateikopf

Die Größe des Dateikopfes bei BMP beträgt 14 Bytes.

Offset	Format	Größe	Inhalt
0	uint16_t	2 Byte	Dieses Feld identifiziert BMP- oder DIB-Dateien. Es enthält `0x42 0x4D` in hexadezimaler Schreibweise. Dies entspricht in ASCII den Buchstaben `BM`. Die folgenden Einträge sind möglich: • BM – Windows 3.1x, 95, NT, … etc. • BA – OS/2 struct bitmap array • CI – OS/2 struct color icon • CP – OS/2 const color pointer • IC – OS/2 struct icon • PT – OS/2 pointer
2	uint32_t	4 Byte	Die Größe der BMP-Datei in Bytes
6	uint32_t	4 Byte	Reserviert
10	uint32_t	4 Byte	Der Offset, an dem die Pixeldaten beginnen. Dieser Wert **muss** zwingend ausgewertet werden, da nicht immer sichergestellt ist, dass er 54 beträgt (14 Bytes + 40 Bytes).

Quelle: [@wiki:bmp]

Der Bitmap-Informationsblock

Offset	Format	Größe	Inhalt
2	uint32_t	4 Byte	Größe der BITMAPINFOHEADER-Struktur in Byte
6	int32_t	4 Byte	Breite der Bitmap in Pixel

Offset	Format	Größe	Inhalt
10	int32_t	4 Byte	Der Betrag gibt die Höhe der Bitmap in Pixel an. Ist der Wert *positiv*, so ist die Bitmap eine sogenannte „bottom-up“-Bitmap (die Bilddaten beginnen mit der untersten und enden mit der obersten Bildzeile). Dies ist die gebräuchlichste Variante. Ist der Wert *negativ*, so ist die Bitmap eine „top-down“-Bitmap (die Bilddaten beginnen mit der obersten und enden mit der untersten Bildzeile).
0	uint16_t	2 Byte	Inhalt: 1 (Stand in einigen älteren Formaten wie PCX für die Anzahl der Farbebenen, wird aber für BMP nicht verwendet)
0	uint16_t	2 Byte	**biBitCount**: Gibt die Farbtiefe der Bitmap in bpp an; muss einer der folgenden Werte sein: 1, 4, 8, 16, 24 oder 32. Bei 1, 4 und 8 bpp sind die Farben indiziert.
10	uint32_t	4 Byte	**biCompression** Einer der folgenden Werte: • 0 (BI_RGB): Bilddaten sind unkomprimiert. • 1 (BI_RLE8): Bilddaten sind lauflängenkodiert für 8 bpp. Nur erlaubt, wenn biBitCount=8 und biHeight positiv ist. • 2 (BI_RLE4): Bilddaten sind lauflängenkodiert für 4 bpp. Nur erlaubt, wenn biBitCount=4 und biHeight positiv ist. • 3 (BI_BITFIELDS): Bilddaten sind unkomprimiert und benutzerdefiniert (mittels Farbmasken) kodiert. Nur erlaubt, wenn biBitCount=16 oder 32.
10	uint32_t	4 Byte	• Wenn `biCompression=BI_RGB:` Entweder 0 oder die Größe der Bilddaten in Byte. • Ansonsten: Größe der Bilddaten in Byte.
10	uint32_t	4 Byte	Horizontale Auflösung des Zielausgabegerätes in Pixel pro Meter; wird aber für BMP-Dateien meistens auf 0 gesetzt.
10	uint32_t	4 Byte	Vertikale Auflösung des Zielausgabegerätes in Pixel pro Meter; wird aber für BMP-Dateien meistens auf 0 gesetzt.
10	uint32_t	4 Byte	• Wenn biBitCount=1: 0. • Wenn biBitCount=4 oder 8: die Anzahl der Einträge der Farbtabelle; 0 bedeutet die maximale Anzahl (2, 16 oder 256). • Ansonsten: Die Anzahl der Einträge der Farbtabelle (0=keine Farbtabelle). Auch wenn sie in diesem Fall nicht notwendig ist, kann dennoch eine für die Farbquantisierung empfohlene Farbtabelle angegeben werden.

Offset	Format	Größe	Inhalt
10	uint32_t	4 Byte	• Wenn biBitCount=1, 4 oder 8: Die Anzahl sämtlicher im Bild verwendeten Farben; 0 bedeutet alle Farben der Farbtabelle. • Ansonsten: ◦ Wenn eine Farbtabelle vorhanden ist und diese sämtliche im Bild verwendeten Farben enthält: deren Anzahl. ◦ Ansonsten: 0.

Quelle: [@wiki:bmp]

Funktionszeiger in C

Bisher haben wir Zeiger verwendet, die auf Variablen oder Datenbereiche verweisen. Die Programmiersprache C bietet allerdings auch die Möglichkeit, Zeiger auf Funktionen zu deklarieren. Mittels eines solchen Zeigers kann anschließend die Funktion aufgerufen werden. Solche Zeiger lassen sich auch als Parameter an andere Funktionen übergeben.

Die Deklaration eines solchen Funktionszeigers ist wie folgt definiert:

```
< return type > (*ptr_name)(< argument list >)
```

Dabei ist die Verwendung der Klammern zwingend, da es sich ansonsten um einen Rückgabewert vom Typ Zeiger handelt.

```
int *f;     /* Zeiger auf eine Integer-Variable */
int *f();   /* Funktion, die einen Zeiger auf eine Integer-
               Variable zurückliefert
            */
int (*f)(); /* Zeiger auf Funktion mit Rückgabe Integer */
```

Lediglich bei der letzten Deklaration handelt es sich um einen Funktionszeiger. Der Umgang mit Funktionszeigern fühlt sich anfänglich etwas sperrig an. Ein Beispiel für die Verwendung von Funktionszeigern ist die Funktion `qsort` aus der Standardbibliothek.

```
#include <stdlib.h>
void qsort(void *base,
    size_t nel,
    size_t width,
    int (*compar)(const void *, const void *));
```

Dabei stellt `(*compar)` einen Zeiger auf eine Funktion dar, die zwei Zeiger vom Typ `void` als Parameter hat. Als Rückgabewert liefert diese Funktion einen Integer. Dieser Integer ist gleich Null, wenn die Parameter gleich groß sind. Ist der erste Parameter kleiner als der zweite, liefert diese Funktion einen Wert kleiner Null. Im anderen Fall einen Wert größer Null.

Die Schreibweise des Funktionszeigers kann mittels einer Typdefinition erleichtert werden:

```
typedef int (*my_func)(int x, int y);
void use_my_function(my_func f, int a, int b)
{
    printf("pow(%d,%d) = %d\n", a, b, f(a,b));
}

int pow(int a, int b)
{
    for (int i = 0; i < b; i++)
    {
        a *= a;
    }
    return a;
}

//...
int a = 2;
int b = 4;
use_my_function(pow, a, b);
```

Alles hat einen Zustand

Alle Dinge haben einen Zustand. Dieser Zustand kann durch eine Änderung der Umgebung geändert werden. So wechselt Wasser bei Zuführung von Wärme vom flüssigen in den gasförmigen Zustand.

In der Informationsverarbeitung werden Änderungen von Zuständen mithilfe sogenannter Zustandsautomaten (engl. finite state machine, kurz FSM) modelliert. Ein solcher Zustandsautomat ist eine abstrakte Maschine, die über einen internen Zustand verfügt. Die Maschine kann zur Laufzeit in definierte Zustände wechseln. In der theoretischen Informatik wird ein Zustandsautomat auch als endlicher Automat bezeichnet.

Der interne Zustand beinhaltet alle Parameter, die zum Betrieb dieser Maschine benötigt werden. Die Maschine „arbeitet“ nur bei einem Zustandswechsel. Das heißt: Es werden nur Aktionen bei einem Zustandswechsel ausgeführt. Der Folgezustand ergibt sich aus dem momentanen Zustand und einem Ereignis von außen. Ein Ereignis kann beispielsweise eine Temperaturänderung oder ein Tastendruck sein.

Hat der endliche Automat einen Zustand stabil eingenommen, wird erst bei einem neuen Ereignis wieder eine Aktion ausgeführt, die einen Zustandswechsel bedingt.

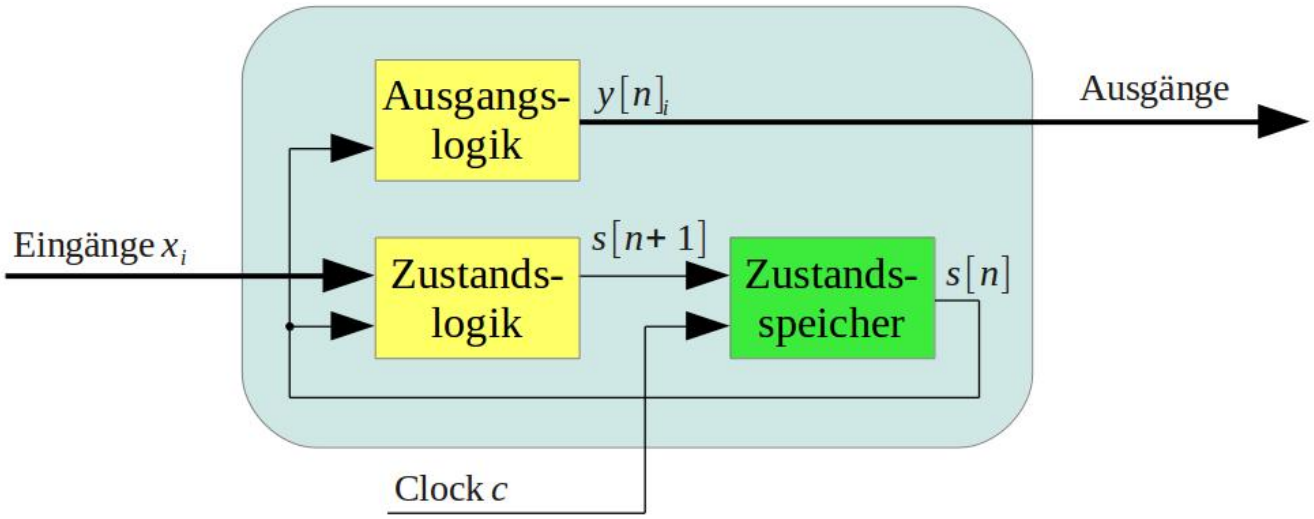
Der Zustandsautomat selbst wird über eine Taktung angetrieben. Er kann also nicht in beliebig kurzen Zeitspannen auf Ereignisse reagieren. In jedem Takt wird anhand des vorliegenden Zustands und des Status der Eingabekanäle entschieden, welcher Zustand als nächstes vorliegen soll und welche Aktionen auszuführen sind.

Die Modellierung eines Zustandsautomaten

In der Informationsverarbeitung unterscheidet man in der Regel zwischen Moore-Automaten und dem Mealy-Automaten. Diese beiden Automaten unterscheiden sich im Wesentlichen dadurch, wie das Ergebnis erzeugt wird. Beim Moore-Automaten hängt die Ausgabe ausschließlich vom Zustand ab. Beim Erreichen eines Zustandes wird eine Ausgabe erzeugt. Diese ist unabhängig vom Übergang in diesen Zustand. Beim Mealy-Automaten ist die Ausgabe vom Zustand und von der Eingabe abhängig.

Der Moore-Automat

Der Moore-Automat ist benannt nach dem Mathematiker Edward F. Moore (1925–2003). Er besteht im Wesentlichen aus drei Teilen:



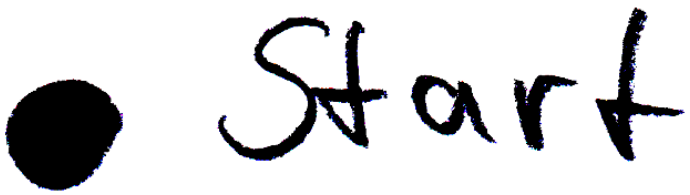
- Zustandsspeicher – speichert den aktuellen Zustand S[n]
- Zustandslogik – bestimmt den Folgezustand S[n+1]. Dieser wird bei einer steigenden Flanke am Takt übernommen.
- Ausgangslogik – bestimmt aufgrund des Zustands S[n] die Ausgänge

Dieses Modell kann auf die Softwareentwicklung übertragen werden. Die Eingänge werden dabei zu Ereignissen. Quellen von Ereignissen können zum Beispiel Taster, Timer oder auch Schnittstellen wie eine UART sein. Der Takt entspricht einem Verarbeitungsaufruf des Zustandsautomaten. Dabei ist es nicht zwingend, dass dieser Aufruf in einem Ablauf mit fester Periode erfolgt. Der Aufruf kann auch so oft wie nötig erfolgen. Die Ausgänge können wiederum LED Anzeigen, eine Kommunikationsschnittstelle oder ein digitaler Ausgang sein (siehe Abbildung).

Die grafische Modellierung des Zustandsautomaten

Der Startzustand

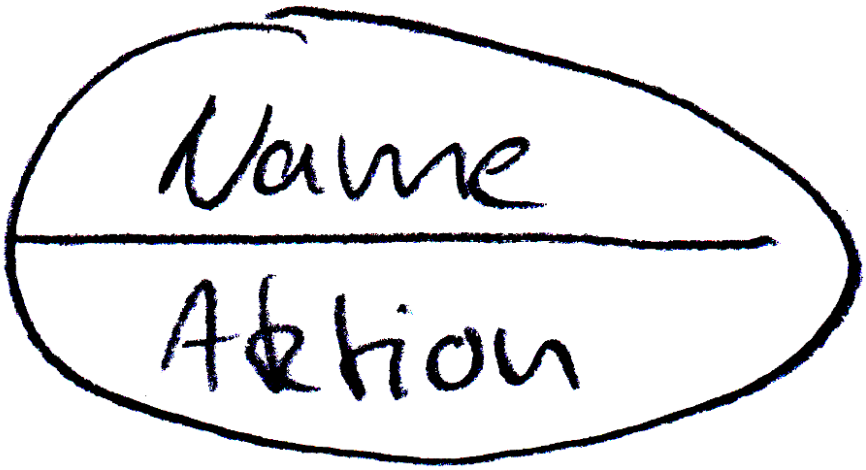
Der Startzustand (siehe Abbildung) beschreibt den Einstiegspunkt in den Zustandsautomaten. Der Startzustand wird durch einen ausgefüllten Kreis symbolisiert, zur Verdeutlichung kann auch "Start" hinzugefügt werden.



Zustände

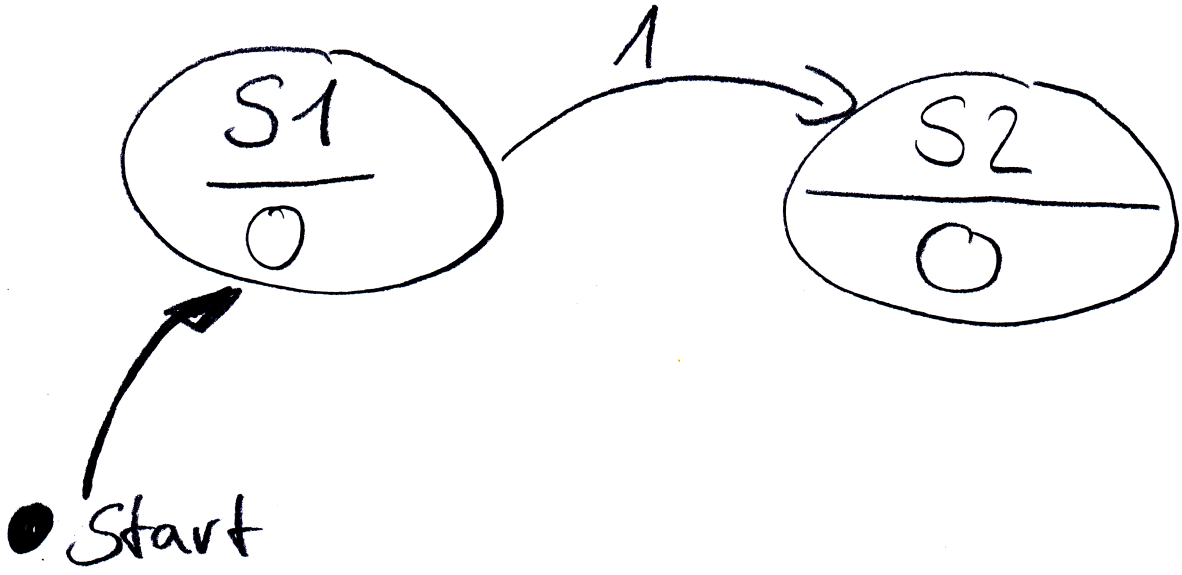
Zustände werden mittels eines zweigeteilten Ovals wie in Abbildung dargestellt. In der oberen

Hälfte befindet sich der Name des Zustands, in der unteren Hälfte die Aktion, die beim Erreichen des Zustands ausgeführt wird.



Übergänge

Ein Übergang, auch Transition genannt, wird mittels eines Pfeils dargestellt. Das Ereignis, das zu dieser Transition führt, wird an den Pfeil geschrieben. Lediglich die Transition aus dem Zustand Start erhält keine Bedingung. Diese erfolgt unmittelbar, und es kann auch nur einen Übergang geben.



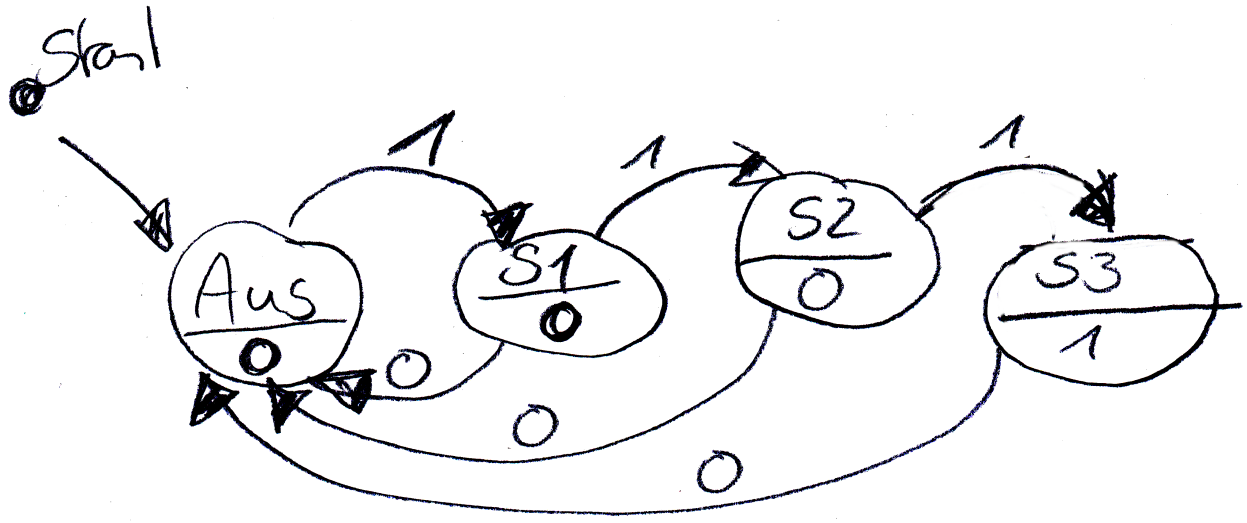
Tastenentprellen

Mechanische Schalter wie Relais neigen beim Ein- und Ausschalten zum Prellen. Dabei wechselt das Ausgangssignal unkoordiniert zwischen den Zuständen an und aus hin und her. Dies wird durch den mechanischen Aufbau der Schalter verursacht. Das vereinfacht dargestellte Signal sieht

wie in der Abbildung dargestellt aus:



Entwickeln Sie einen Zustandsautomaten, der den Ausgang auf logisch "1" setzt, nachdem er dreimal hintereinander eine stabile "1" gelesen hat. Es ist nur der Wechsel von 0 zu 1 zu entprellen. Wie in Abbildung illustriert.



Übungsaufgabe I

Garagentorantrieb

Es soll eine Garagentorsteuerung für eine Fernbedienung, wie sie in Abbildung abgebildet ist, modelliert werden. Die Fernbedienung verfügt über eine einzelne Taste. Beim Anschalten öffnet das Garagentor sich automatisch. Ein Druck auf die Taste ändert die Fahrtrichtung. Das Tor stoppt jeweils an den Endschaltern in den Positionen "unten" und "oben".



Die Implementierung in C

Es gibt viele unterschiedliche Möglichkeiten, einen Zustandsautomaten in C zu implementieren. Eine der am weitesten verbreiteten Arten ist die Switch-Case- Anweisung unter Verwendung eines Aufzählungstyps (enum). Für die Tastenentprellung könnte der Zustandsautomat wie folgt aussehen:

```
extern int getInput(void);

typedef enum {
  DB_STATE_OFF = 0,
  DB_STATE_S1 = 1,
  DB_STATE_S2 = 2,
  DB_STATE_ON = 3,
  DB_STATE_MAX
} DebounceState_t;

DebounceState_t state  = DB_STATE_OFF;

void stateMachine()
{
  switch(state)
  {
    case DB_STATE_OFF:
        if(1 == getInput())
        {
          state = DB_STATE_S1;
        }
    break;
    case DB_STATE_S1:
        if(1 == getInput())
        {
          state = DB_STATE_S2;
        }
```

```
        else
        {
          state = DB_STATE_OFF;
        }
    break;
    case DB_STATE_S2:
        if(1 == getInput())
        {
          state = DB_STATE_ON;
        }
        else
        {
          state = DB_STATE_OFF;
        }

    break;
    case DB_STATE_ON:
        if(1 == getInput())
        {
          state = DB_STATE_ON;
        }
        else
        {
          state = DB_STATE_OFF;
        }
    default:
        state = DB_STATE_OFF;
    break;

  }
}

int main(void)
{
  ....

  while( 1==1 )
  {
    stateMachine();
    sleep_ms(100);
  }
}
```

Übungsaufgabe

Modellierung

Modellieren Sie einen Zustandsautomaten, der das Format der Eingabedaten aus der Übungsaufgabe von letzter Woche korrekt verarbeiten kann. Das Format der Daten sieht wie folgt aus:

```
A:<48656> B:
```

Achten Sie bei Ihrer Implementierung auf die korrekte Verarbeitung des Zeilenumbruchs (0x0a, \n) und des Dateiendes (EOF).

Implementierung

Implementieren Sie Ihren modellierten Zustandsautomaten in der Programmiersprache C.

- Verwenden Sie zum Zeichenweisen einlesen die Funktion `fgetc`
- Verwenden Sie zur Umwandlung der Zeichenketten in einen Integer die Funktion `atoi`
- Kapseln Sie Ihren Zustandsautomaten in ein eigenes C-Modul (Header und C-Datei)
- Schreiben Sie für jeden Zustand eine eigene Funktion. Damit wird Ihre Switch Case-Anweisung übersichtlicher.

Anhang A: Linker-Skript des STM32F103

```
/*
******************************************************************************
**
**  File        : LinkerScript.ld
**
**  Author      : Auto-generated by Ac6 System Workbench
**
**  Abstract    : Linker script for STM32F103RBTx Device from STM32F1 series
**                20Kbytes RAM
**                128Kbytes ROM
**
**                Set heap size, stack size and stack location according
**                to application requirements.
**
**                Set memory bank area and size if external memory is used.
**
**  Target      : STMicroelectronics STM32
**
**  Distribution: The file is distributed as is, without any warranty
**                of any kind.
**
*****************************************************************************
** @attention
**
** <h2><center>&copy; COPYRIGHT(c) 2014 Ac6</center></h2>
**
** Redistribution and use in source and binary forms, with or without modification,
** are permitted provided that the following conditions are met:
**   1. Redistributions of source code must retain the above copyright notice,
**      this list of conditions and the following disclaimer.
**   2. Redistributions in binary form must reproduce the above copyright notice,
```

```
**      this list of conditions and the following disclaimer in the documentation
**      and/or other materials provided with the distribution.
**   3. Neither the name of Ac6 nor the names of its contributors
**      may be used to endorse or promote products derived from this software
**      without specific prior written permission.
**
** THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS"
** AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
** IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
** DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE
** FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
** DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
** SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER
** CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY,
** OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
** OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
**
*****************************************************************************
*/

/* Entry Point */
ENTRY(Reset_Handler)

/* Highest address of the user mode stack */
_estack = 0x20005000;    /* end of RAM */

_Min_Heap_Size = 0;      /* required amount of heap  */
_Min_Stack_Size = 0x400; /* required amount of stack */

/* Memories definition */
MEMORY
{
  RAM (xrw)     : ORIGIN = 0x20000000, LENGTH = 20K
  ROM (rx)      : ORIGIN = 0x8000000, LENGTH = 128K
}

/* Sections */
SECTIONS
{
  /* The startup code into ROM memory */
  .isr_vector :
  {
    . = ALIGN(4);
    KEEP(*(.isr_vector)) /* Startup code */
    . = ALIGN(4);
  } >ROM

  /* The program code and other data into ROM memory */
  .text :
  {
    . = ALIGN(4);
```

```
  *(.text)           /* .text sections (code) */
  *(.text*)          /* .text* sections (code) */
  *(.glue_7)         /* glue arm to thumb code */
  *(.glue_7t)        /* glue thumb to arm code */
  *(.eh_frame)

  KEEP (*(.init))
  KEEP (*(.fini))

  . = ALIGN(4);
  _etext = .;        /* define a global symbols at end of code */
} >ROM

/* Constant data into ROM memory*/
.rodata :
{
  . = ALIGN(4);
  *(.rodata)         /* .rodata sections (constants, strings, etc.) */
  *(.rodata*)        /* .rodata* sections (constants, strings, etc.) */
  . = ALIGN(4);
} >ROM

.ARM.extab   : {
  . = ALIGN(4);
  *(.ARM.extab* .gnu.linkonce.armextab.*)
  . = ALIGN(4);
} >ROM

.ARM : {
  . = ALIGN(4);
  __exidx_start = .;
  *(.ARM.exidx*)
  __exidx_end = .;
  . = ALIGN(4);
} >ROM

.preinit_array     :
{
  . = ALIGN(4);
  PROVIDE_HIDDEN (__preinit_array_start = .);
  KEEP (*(.preinit_array*))
  PROVIDE_HIDDEN (__preinit_array_end = .);
  . = ALIGN(4);
} >ROM

.init_array :
{
  . = ALIGN(4);
  PROVIDE_HIDDEN (__init_array_start = .);
  KEEP (*(SORT(.init_array.*)))
  KEEP (*(.init_array*))
```

```
  PROVIDE_HIDDEN (__init_array_end = .);
  . = ALIGN(4);
} >ROM

.fini_array :
{
  . = ALIGN(4);
  PROVIDE_HIDDEN (__fini_array_start = .);
  KEEP (*(SORT(.fini_array.*)))
  KEEP (*(.fini_array*))
  PROVIDE_HIDDEN (__fini_array_end = .);
  . = ALIGN(4);
} >ROM

/* Used by the startup to initialize data */
_sidata = LOADADDR(.data);

/* Initialized data sections into RAM memory */
.data :
{
  . = ALIGN(4);
  _sdata = .;        /* create a global symbol at data start */
  *(.data)           /* .data sections */
  *(.data*)          /* .data* sections */

  . = ALIGN(4);
  _edata = .;        /* define a global symbol at data end */
} >RAM AT> ROM


/* Uninitialized data section into RAM memory */
. = ALIGN(4);
.bss :
{
  /* This is used by the startup in order to initialize the .bss secion */
  _sbss = .;         /* define a global symbol at bss start */
  __bss_start__ = _sbss;
  *(.bss)
  *(.bss*)
  *(COMMON)

  . = ALIGN(4);
  _ebss = .;         /* define a global symbol at bss end */
  __bss_end__ = _ebss;
} >RAM

/* User_heap_stack section, used to check that there is enough RAM left */
._user_heap_stack :
{
  . = ALIGN(8);
  PROVIDE ( end = . );
```

```
    PROVIDE ( _end = . );
    . = . + _Min_Heap_Size;
    . = . + _Min_Stack_Size;
    . = ALIGN(8);
  } >RAM

  /* Remove information from the compiler libraries */
  /DISCARD/ :
  {
    libc.a ( * )
    libm.a ( * )
    libgcc.a ( * )
  }

  .ARM.attributes 0 : { *(.ARM.attributes) }
}
```

Anhang B: Start-up-Code des STM32F103

```
/**
  ******************************************************************************
  * @file      startup_stm32.s
  * @author    Ac6
  * @version   V1.0.0
  * @date      12-June-2014
  ******************************************************************************
  */

  .syntax unified
  .cpu cortex-m3
  .thumb

.global g_pfnVectors
.global Default_Handler

/* start address for the initialization values of the .data section.
defined in linker script */
.word   _sidata
/* start address for the .data section. defined in linker script */
.word   _sdata
/* end address for the .data section. defined in linker script */
.word   _edata
/* start address for the .bss section. defined in linker script */
.word   _sbss
/* end address for the .bss section. defined in linker script */
.word   _ebss

.equ  BootRAM,        0xF1E0F85F
/**
```

```
 * @brief  This is the code that gets called when the processor first
 *          starts execution following a reset event. Only the absolutely
 *          necessary set is performed, after which the application
 *          supplied main() routine is called.
 * @param  None
 * @retval : None
*/

    .section    .text.Reset_Handler
    .weak   Reset_Handler
    .type   Reset_Handler, %function
Reset_Handler:

/* Copy the data segment initializers from flash to SRAM */
  movs  r1, #0
  b LoopCopyDataInit

CopyDataInit:
    ldr r3, =_sidata
    ldr r3, [r3, r1]
    str r3, [r0, r1]
    adds    r1, r1, #4

LoopCopyDataInit:
    ldr r0, =_sdata
    ldr r3, =_edata
    adds    r2, r0, r1
    cmp r2, r3
    bcc CopyDataInit
    ldr r2, =_sbss
    b   LoopFillZerobss
/* Zero fill the bss segment. */
FillZerobss:
    movs r3, #0
    str  r3, [r2]
    adds r2, r2, #4

LoopFillZerobss:
    ldr r3, = _ebss
    cmp r2, r3
    bcc FillZerobss

/* Call the clock system intitialization function.*/
    bl  SystemInit
/* Call static constructors */
    bl __libc_init_array
/* Call the application's entry point.*/
    bl  main

LoopForever:
    b LoopForever
```

```
.size   Reset_Handler, .-Reset_Handler

/**
 * @brief  This is the code that gets called when the processor receives an
 *         unexpected interrupt.  This simply enters an infinite loop, preserving
 *         the system state for examination by a debugger.
 *
 * @param  None
 * @retval : None
*/
    .section    .text.Default_Handler,"ax",%progbits
Default_Handler:
Infinite_Loop:
    b   Infinite_Loop
    .size   Default_Handler, .-Default_Handler
/*******************************************************************************
*
* The minimal vector table for a Cortex-M.  Note that the proper constructs
* must be placed on this to ensure that it ends up at physical address
* 0x0000.0000.
*
*******************************************************************************/
    .section    .isr_vector,"a",%progbits
    .type   g_pfnVectors, %object
    .size   g_pfnVectors, .-g_pfnVectors

g_pfnVectors:
    .word   _estack
    .word   Reset_Handler
    .word   NMI_Handler
    .word   HardFault_Handler
    .word   MemManage_Handler
    .word   BusFault_Handler
    .word   UsageFault_Handler
    .word   0
    .word   0
    .word   0
    .word   0
    .word   SVC_Handler
    .word   DebugMon_Handler
    .word   0
    .word   PendSV_Handler
    .word   0  //  .word SysTick_Handler
    .word   0  //  .word WWDG_IRQHandler
    .word   0  //  .word PVD_IRQHandler
    .word   0  //  .word TAMPER_IRQHandler
    .word   0  //  .word RTC_IRQHandler
    .word   0  //  .word FLASH_IRQHandler
    .word   0  //  .word RCC_IRQHandler
    .word   0  //  .word EXTI0_IRQHandler
```

```
    .word   0  //  .word EXTI1_IRQHandler
    .word   0  //  .word EXTI2_IRQHandler
    .word   0  //  .word EXTI3_IRQHandler
    .word   0  //  .word EXTI4_IRQHandler
    .word   0  //  .word DMA1_Channel1_IRQHandler
    .word   0  //  .word DMA1_Channel2_IRQHandler
    .word   0  //  .word DMA1_Channel3_IRQHandler
    .word   0  //  .word DMA1_Channel4_IRQHandler
    .word   0  //  .word DMA1_Channel5_IRQHandler
    .word   0  //  .word DMA1_Channel6_IRQHandler
    .word   0  //  .word DMA1_Channel7_IRQHandler
    .word   0  //  .word ADC1_2_IRQHandler
    .word   0  //  .word USB_HP_CAN1_TX_IRQHandler
    .word   0  //  .word USB_LP_CAN1_RX0_IRQHandler
    .word   0  //  .word CAN1_RX1_IRQHandler
    .word   0  //  .word CAN1_SCE_IRQHandler
    .word   0  //  .word EXTI9_5_IRQHandler
    .word   0  //  .word TIM1_BRK_IRQHandler
    .word   0  //  .word TIM1_UP_IRQHandler
    .word   0  //  .word TIM1_TRG_COM_IRQHandler
    .word   0  //  .word TIM1_CC_IRQHandler
    .word   0  //  .word TIM2_IRQHandler
    .word TIM3_IRQHandler
    .word   0  //  .word TIM4_IRQHandler
    .word   0  //  .word I2C1_EV_IRQHandler
    .word   0  //  .word I2C1_ER_IRQHandler
    .word   0  //  .word I2C2_EV_IRQHandler
    .word   0  //  .word I2C2_ER_IRQHandler
    .word   0  //  .word SPI1_IRQHandler
    .word   0  //  .word SPI2_IRQHandler
    .word   0  //  .word USART1_IRQHandler
    .word   0  //  .word USART2_IRQHandler
    .word   0  //  .word USART3_IRQHandler
    .word   0  //  .word EXTI15_10_IRQHandler
    .word   0  //  .word RTC_Alarm_IRQHandler
    .word   0  //  .word USBWakeUp_IRQHandler
    .word   0  //  .word 0
    .word   0  //  .word 0
    .word   0  //  .word 0
    .word   0  //  .word 0
    .word   0  //  .word 0
    .word   0  //  .word 0
    .word   0  //  .word 0
    .word   0  //  .word BootRAM          /* @0x108. This is for boot in RAM mode for
    .word   0  //                            STM32F10x Medium Density devices. */
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
```

```
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0
    .word   0  //   .word   0

/*******************************************************************************
*
* Provide weak aliases for each Exception handler to the Default_Handler.
* As they are weak aliases, any function with the same name will override
* this definition.
*
*******************************************************************************/

    .weak   NMI_Handler
    .thumb_set NMI_Handler,Default_Handler

    .weak   HardFault_Handler
    .thumb_set HardFault_Handler,Default_Handler

    .weak   MemManage_Handler
    .thumb_set MemManage_Handler,Default_Handler

    .weak   BusFault_Handler
    .thumb_set BusFault_Handler,Default_Handler

    .weak   UsageFault_Handler
    .thumb_set UsageFault_Handler,Default_Handler

    .weak   SVC_Handler
    .thumb_set SVC_Handler,Default_Handler
```

```
    .weak   DebugMon_Handler
    .thumb_set DebugMon_Handler,Default_Handler

    .weak   PendSV_Handler
    .thumb_set PendSV_Handler,Default_Handler

    .weak   SysTick_Handler
    .thumb_set SysTick_Handler,Default_Handler

    .weak   SystemInit

/*********************** (C) COPYRIGHT Ac6 *****END OF FILE****/
```

Anhang C: Versionsverwaltung

Die Versionsverwaltung dient der Verwaltung und Erfassung von Änderungen an Dokumenten oder Dateien.

Wozu benötigen wir eine Versionsverwaltung?

- Protokollierung von Änderungen: Es kann jederzeit nachvollzogen werden, wer wann was geändert hat.
- Wiederherstellung alter Stände einzelner Dateien: Somit können versehentliche Änderungen jederzeit wieder rückgängig gemacht werden.
- Archivierung der einzelnen Stände eines Projektes: Dadurch ist es jederzeit möglich, auf alle Versionen zuzugreifen.
- Koordinierung des gemeinsamen Zugriffs von mehreren Entwicklern auf die Dateien.
- Gleichzeitige Entwicklung mehrerer Entwicklungszweige (engl. Branches) eines Projektes.

Fachbegriffe

- Ein Branch, zu Deutsch Zweig, ist eine Abspaltung von einer anderen Version, so dass unterschiedliche Versionen parallel weiterentwickelt werden können.
- Mergen: Änderungen können dabei von einem Branch auch in einen anderen wieder einfließen, das wird dann als Merging, zu Deutsch verschmelzen, bezeichnet.
- Trunk: Oft wird der Hauptentwicklungszweig als Trunk (z. B. bei Subversion) oder master (z. B. bei Git) bezeichnet. Branches können zum Beispiel für neue Hauptversionen einer Software erstellt werden oder für Entwicklungszweige für unterschiedliche Betriebssysteme oder aber auch, um experimentelle Versionen zu haben.
- Forks sind in der Regel Branches.
- Tags: Ein bestimmter Stand kann auch mit einem Tag (einem frei wählbaren Bezeichner) gekennzeichnet werden.
- Repository (englisch: Behälter): Beinhaltet die Dateien des Quellcodeverwaltungssystems.
- Auschecken (fetch): Laden der aktuellen Kopie aus dem Versionsverwaltungssystem.

- Einchecken (commit): Eintragen der lokalen Änderungen in das Quellcodeverwaltungssystem.
- Arbeitskopie: Lokale Kopie der Daten aus dem Repository. Entsteht durch Auschecken.

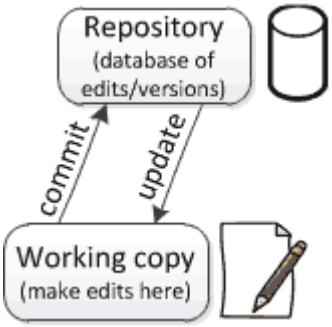


Abbildung 10. Repository Quelle: [1]

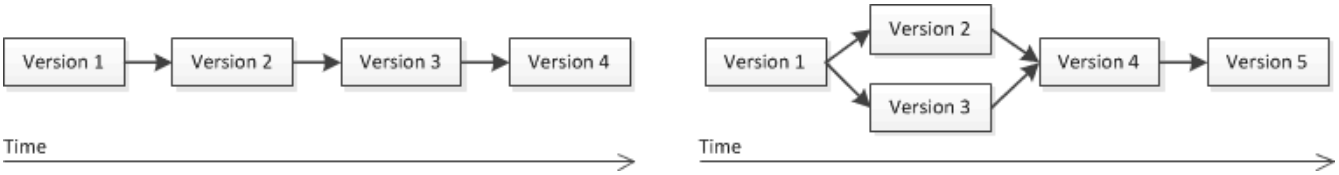


Abbildung 11. Konzept Versionierung Quelle: [2]

Lokale Versionsverwaltung

Bei der lokalen Versionsverwaltung wird oft nur eine einzige Datei versioniert. diese Variante wurde mit Werkzeugen wie SCCS und RCS umgesetzt. Sie findet auch heute noch Verwendung in Büroanwendungen, die Versionen eines Dokumentes in der Datei des Dokuments selbst speichern. Auch in technischen Zeichnungen werden Versionen zum Beispiel durch einen Änderungsindex verwaltet.

Zentrale Versionsverwaltung

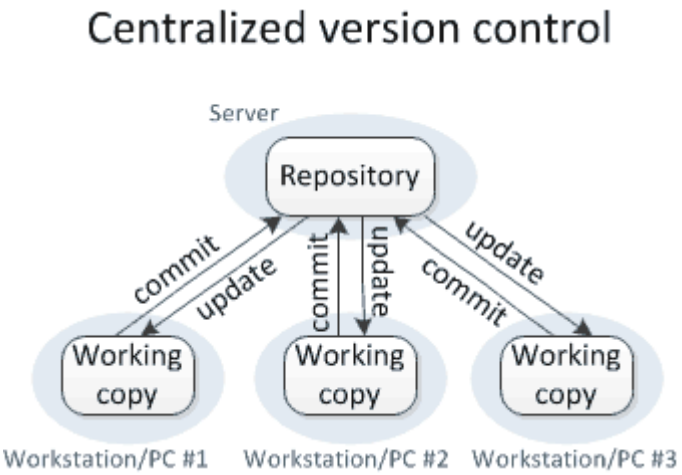


Abbildung 12. Zentrale Versionsverwaltung Quelle: [3]

Diese Art ist als Client-Server-System aufgebaut, sodass der Zugriff auf ein Repository auch über ein Netzwerk erfolgen kann. Durch eine Rechteverwaltung wird dafür gesorgt, dass nur berechtigte Personen neue Versionen in das Archiv legen können. Die Versionsgeschichte ist hierbei nur im Repository vorhanden. Dieses Konzept wurde vom Open Source Projekt Concurrent Versions System (CVS) populär gemacht, mit Subversion (SVN) neu implementiert und von vielen kommerziellen Anbietern verwendet.

Verteilte Versionsverwaltung

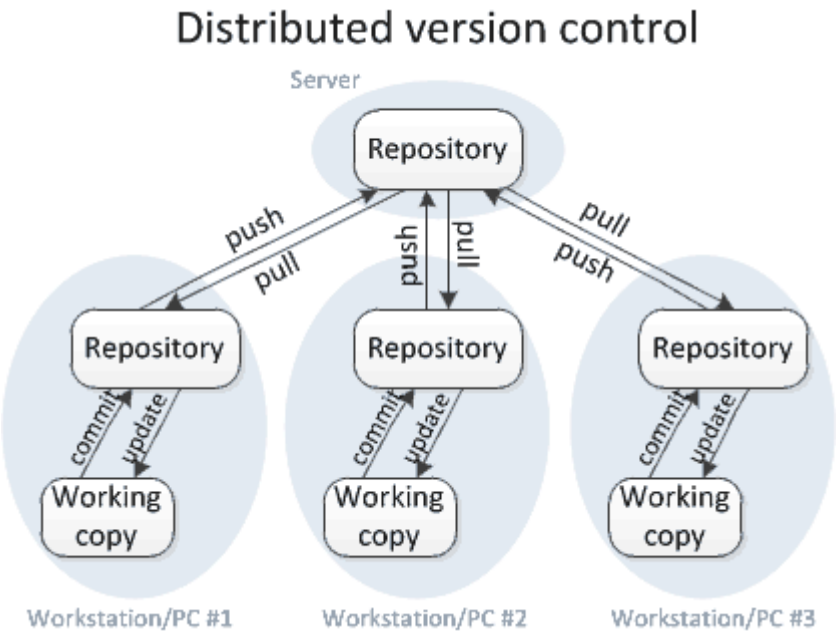


Abbildung 13. Dezentrale Versionsverwaltung Quelle: [4]

Die verteilte Versionsverwaltung verwendet kein zentrales Repository mehr. Jeder, der an dem verwalteten Projekt arbeitet, hat sein eigenes Repository und kann dieses mit jedem beliebigen anderen Repository abgleichen. Die Versionsgeschichte ist dadurch genauso verteilt. Änderungen können lokal verfolgt werden, ohne eine Verbindung zu einem Server aufbauen zu müssen.

Im Gegensatz zur zentralen Versionsverwaltung kommt es nicht zu einem Konflikt, wenn mehrere Benutzer dieselbe Version einer Datei ändern. Die sich widersprechenden Versionen existieren zunächst parallel und können weiter geändert werden. Sie können später in eine neue Version zusammengeführt werden. Dadurch entsteht ein gerichteter azyklischer Graph (Polyhierarchie) anstatt einer Kette von Versionen. In der Praxis werden bei der Verwendung in der Softwareentwicklung meist einzelne Features oder Gruppen von Features in separaten Versionen entwickelt und diese bei größeren Projekten von Personen mit einer Integrator-Rolle überprüft und zusammengeführt.

Systembedingt bieten verteilte Versionsverwaltungen keine Locks. Da wegen der höheren Zugriffsgeschwindigkeit die Granularität der gespeicherten Änderungen viel kleiner sein kann, können sie sehr leistungsfähige, weitgehend automatische Merge-Mechanismen zur Verfügung stellen.

Konzepte

Lock Modify Write

Quelle: Wikipedia Diese Arbeitsweise eines Versionsverwaltungssystems wird auch als Lock-Modify-Unlock bezeichnet. Die zugrunde liegende Philosophie wird pessimistische Versionsverwaltung genannt. Einzelne Dateien müssen vor einer Änderung durch den Benutzer gesperrt und nach Abschluss der Änderung wieder freigegeben werden. Während sie gesperrt sind, verhindert das System Änderungen durch andere Benutzer. Der Vorteil dieses Konzeptes ist, dass kein Zusammenführen von Versionen erforderlich ist, da nur immer ein Entwickler eine Datei ändern kann. Der Nachteil ist, dass man unter Umständen auf die Freigabe eines Dokuments

warten muss, um eine eigene Änderung einzubringen. Zu beachten ist, dass Binärdateien (im Gegensatz zu Quelltextdateien) in der Regel diese Arbeitsweise erfordern, da das Versionsverwaltungssystem verteilte Änderungen nicht automatisch synchronisieren kann. Ältester Vertreter dieser Arbeitsweise ist das Revision Control System, ebenso bekannt ist Visual SourceSafe. Verteilte Versionsverwaltungssysteme kennen diese Arbeitsweise systembedingt nicht.

Copy Modify Merge

Ein solches System lässt gleichzeitige Änderungen durch mehrere Benutzer an einer Datei zu. Anschließend werden diese Änderungen automatisch oder manuell zusammengeführt (Merge). Somit wird die Arbeit des Entwicklers wesentlich erleichtert, da Änderungen nicht im Voraus angekündigt werden müssen. Insbesondere, wenn viele Entwickler räumlich getrennt arbeiten, wie es beispielsweise bei Open-Source-Projekten häufig der Fall ist, ermöglicht dies erst effizientes Arbeiten, da kein direkter Kontakt zwischen den Entwicklern benötigt wird. Problematisch bei diesem System sind Binärdateien, da diese oft nicht automatisch zusammengeführt werden können, sofern kein passendes Werkzeug verfügbar ist. Manche Vertreter dieser Gattung unterstützen daher auch alternativ das Lock-Modify-Write-Konzept für bestimmte Dateien.

Die zugrunde liegende Philosophie wird als optimistische Versionsverwaltung bezeichnet und wurde entwickelt, um die Schwächen der pessimistischen Versionsverwaltung zu beheben. Alle modernen zentralen und verteilten Systeme setzen dieses Verfahren um.

Best Practice

- Tools zur Verwaltung von Quellcode sind für die Verwaltung von Quellcode im ASCII-Textformat optimiert. Deshalb sollten Binärdateien nur in Ausnahmen eingecheckt werden. Checken Sie auch keine ausführbaren Dateien ein, die durch Ihren Quellcode generiert wurden.
- Keine Dateien einchecken, die automatisch durch Entwicklungswerkzeuge erzeugt werden. Minimalprinzip!
- Aktualisieren Sie regelmäßig Ihre Arbeitskopie.
- Checken Sie Ihre Änderungen möglichst zügig in die Quellcodeverwaltung ein damit die Änderungen den anderen Teammitgliedern zur Verfügung stehen.
- Verwenden Sie Commit-Kommentare. Beschreiben Sie nicht, was Sie getan haben, sondern warum. Was Sie getan haben, kann man einfach nachvollziehen.
- Machen Sie kleine Commits. Beschränken Sie Commits auf Features. Ein Commit sollte so wenig Änderungen als möglich enthalten.
- Achten Sie beim Committen auf die Reihenfolge. Dies gilt insbesondere bei großen Änderungen, die Sie in kleinere Commits unterteilt haben.
- Falls Sie ein Bug-Tracker einsetzen und sich eine Änderung auf eine Bug-ID bezieht, referenzieren Sie diese im Commit-Kommentar.

Git

Bei Git handelt es sich um ein dezentrales und quelloffenes Versionsverwaltungssystem.

Geschichte

Quelle: Wikipedia Durch eine Lizenzänderung des bis dahin genutzten, proprietären BitKeeper-Systems konnten die Linux-Kernel-Entwickler dieses nicht mehr kostenlos verwenden und somit blieb vielen Entwicklern der Zugang verwehrt. Daher begann Linus Torvalds im April 2005 mit der Entwicklung einer neuen Quellcode-Management-Software und präsentierte bereits wenige Tage nach deren Ankündigung eine erste Version. Altes Logo

Torvalds wünschte sich ein verteiltes System, das wie BitKeeper genutzt werden konnte und folgende Anforderungen erfüllte:

- Unterstützung verteilter, BitKeeper-ähnlicher Arbeitsabläufe
- Sehr hohe Sicherheit gegen sowohl unbeabsichtigte als auch böswillige Verfälschung
- Hohe Effizienz

Links und Hinweise

- Pro Git

Erste Schritte: ein Repository anlegen

Eine ausführliche Beschreibung kann hier gefunden werden: Git-Grundlagen: ein Git-Repository anlegen

Festlegung der eigenen Identität

Damit Änderungen am Quellcode eindeutig nachvollzogen werden können, muss die Identität festgelegt werden:

```
$ git config --global user.name "John Doe"
$ git config --global user.email johndoe@example.com
```

Einrichten eines GitHub-Zugangs

Zur Veranschaulichung wird ein Zugang auf gitlab.com eingerichtet.

Erstellen eines Git-Repositories aus bereits vorhandenem Quellcode

Es wird ein einfaches Hello-World-Programm erstellt und in Git importiert.

Ein leeres Git-Repository erzeugen: `$ git init`

Erstellen eines kleinen Projektes

Hello World unter Versionsverwaltung stellen: `$ git add hello.c`

Den aktuellen Status prüfen

```
$ git status
```

Eine neue Datei

```
$ gcc  -Wall -o test hello.c # Alternativ Visual Studio
$ git commit
$ git log
```

Anhang D: Testen

Weshalb muss getestet werden?

- Komplexe Systeme führen häufig zu Fehlern.
- Programmiersprachen sind fehleranfällig.

Fehlerursachen in komplexen Systemen

- Kommunikationsprobleme zwischen Menschen. Jede Transformation von Wissen kann Informationsverlust bedeuten. Eine Transformation findet beispielsweise beim Schreiben von Anforderungen statt. Der Anforderungsingenieur beschreibt seine Vorstellungen textuell; dabei können Details verloren gehen.
- Sprachliche Unsicherheit, z. B. wenn Spezifikationen auf Englisch geschrieben werden
- Gedächtnisprobleme, kein Mensch kann sich alles merken.
- Fachliches Verständnis fehlt. In der Regel verfügt ein Programmierer nicht über das Fachwissen der Anwender. Soll etwa eine Software für einen Schreinereibetrieb erstellt werden so sind dem Programmierer die Abläufe in diesem Handwerk nicht geläufig. Dem Schreiner fehlt im Gegenzug die Kenntnis zum Erstellen komplexer Softwaresysteme. Ein weiteres Problem ist die jeweilige Fachsprache der verschiedenen Gewerke.
- Eine hohe Komplexität der Softwarelösung. Es ist einfach, ein kompliziertes System zu erschaffen, aber sehr schwierig, eine einfache und verständliche Lösung eines Problems zu erschaffen.

Programmiersprachen sind fehleranfällig

Quelle Wikipedia: Die Ariane 5 startete am 4. Juni 1996 zu ihrem Erstflug V88 mit den vier Cluster-Satelliten als Nutzlast. Nach 37 Sekunden stellte sich die Rakete plötzlich quer, brach durch die Luftkräfte auseinander und sprengte sich selbst. Dabei kamen keine Menschen ums Leben, doch der materielle Schaden belief sich auf etwa 370 Millionen US-Dollar. Verkürzte Darstellungen nennen den Fehlstart einen der teuersten Softwarefehler der Geschichte.

Gehen Sie auf die Fehlermeldungen Ihres Compilers ein und ignorieren Sie sie nicht. Auch wenn Ihnen die Fehler sehr trivial vorkommen. Es kann schnell passieren, dass Sie eine wichtige Warnung durch die Flut der „unwichtigen“ übersehen.

Verwenden Sie niemals die folgende Anweisung in C++

using namespace std;

Was ist Qualität?

Quelle Uwe Vingenschow:

Qualität ist Leistung im Verhältnis zur Erwartung.

Damit wird es wichtig, die Erwartungen des Auftraggebers genau zu kennen.

Funktionalität

- Angemessenheit: geeignete Realisierung der Anforderungen; kein Schießen mit Kanonen auf Spatzen schießen.
- Richtigkeit: korrekte Realisierung gemäß Spezifikation
- Interoperabilität: Zusammenspiel des Testsystems mit anderen vorgegeben Systemen.
- Ordnungsmäßigkeit: Erfüllung von anwendungsspezifischen Normen, gesetzlichen Bestimmungen oder sonstigen Vereinbarungen.
- Sicherheit: Zugriffs- und Datensicherheit gegenüber versehentlichem oder vorsätzlichem unberechtigtem Zugriff.

Aktuell spricht in der Industrie nahezu jeder von Industrie 4.0. Allerdings machen sich die wenigsten Gedanken über das Thema Sicherheit. Bei Industrie 4.0 oder auch Internet of Things geht es in der Regel um eine Vernetzung von Systemen. Dabei wird die Wartung und Pflege solcher Produkte leider nicht ausreichend beachtet. So sollten diese Systeme regelmäßig ein Software-Update erfahren. Dabei sind sowohl die Hersteller wie auch die Anwender gefragt.

Beispiel USB Keyboard Hack

Snake on a Keyboard

- YouTube Video: https://www.youtube.com/watch?v=enzU9u328Fc
- Blog: http://spritesmods.com/?art=rapidisnake&page=2

Weshalb ist ein solcher Hack so gefährlich?

- Was passiert, wenn Sie auf einem solchen Keyboard Ihre Bankgeschäfte erledigen?
- Was passiert, wenn ein Virus die Firmware Ihres Keyboards ändert?
- Das Keyboard kann Ihren PC vollkommen autonom steuern.

Zuverlässigkeit

- Reife: Wie häufig erfolgt ein Versagen der Software durch Fehlerzustände?
- Fehlertoleranz: Wie verhält sich das System, nachdem ein Problem zu einer Fehlerwirkung geführt hat. Beispiel Trigger mittels Relais am O3D300.
- Wiederherstellbarkeit: Wie einfach und schnell kann das geforderte Leistungsniveau nach einem Versagen oder Ausfall wiederhergestellt werden? Beispiel Dead Chicken, O2D2xx Norwegen Bug.

Benutzbarkeit

- Verständlichkeit
- Erlernbarkeit
- Bedienbarkeit

Effizienz

- Zeitverhalten, Beispiel: 25 Hz beim O3D3xx
- Ressourcenverbrauch, z. B. Flash beim Diagnose-Controller

Änderbarkeit

- Analysierbarkeit
- Modifizierbarkeit
- Stabilität
- Prüfbarkeit

Ein Softwaresystem muss testbar und erweiterbar sein, da ein solches Produkt lebt und einige Anforderungen erst mit dem Erleben des Produktes entstehen.

Übertragbarkeit

- Anpassbarkeit
- Installierbarkeit - Abhängigkeit zu Systemen die nur schwer zu beschaffen sind
- Austauschbarkeit

[1] Das Repository: http://homes.cs.washington.edu/~mernst/advice/version-control.html

[2] Das Repository Konzept: http://homes.cs.washington.edu/~mernst/advice/version-control.html

[3] Zentrale Versionsverwaltung: http://homes.cs.washington.edu/~mernst/advice/version-control.html

[4] Dezentrale Versionsverwaltung: http://homes.cs.washington.edu/~mernst/advice/version-control.html